Чернівецький Національний університет імені Юрія Федьковича
Факультет математики та інформатики

В. Косован
О. Матвій
Г. Мельник
В. Мельник

Інтелектуальний Аналіз Даних

Навчальний посібник

Чернівці, 2026

УДК 004.738.5

Рекомендовано до друку методичною радою факультету математики та інформатики
Чернівецького національного університету імені Юрія Федьковича
(протокол № від 2026 року)

Укладачі:
Василь Косован, кандидат фізико-математичних наук, асистент кафедри математичного моделювання.
Олександр Матвій, кандидат фізико-математичних наук, доцент кафедри математичного моделювання;
Галина Мельник, кандидат економічних наук, доцент кафедри прикладної математики та інформаційних технологій;
Василь Мельник, кандидат фізико-математичних наук, асистент кафедри математичного моделювання;

Інтелектуальний Аналіз Даних : навчальний посібник.. Укл.: Василь Косован, Олександр Матвій, Галина Мельник, Василь Мельник – Чернівці: Чернівецький національний університет ім. Юрія Федьковича, 2026. – с.

Посібник висвітлює базові принципи інтелектуального аналізу даних. Матеріали посібника підготовлені на основі методологічних досліджень авторів та відповідного курсу лекцій, містять основи теорії і практики аналізу даних.

Рецензенти:
Міночкін Дмитро Анатолійович, канд. щу сфері телекомунікацій, доцент кафедрителекомунікацій, Національний технічний університет України "Київський політехнічний інститут імені Ігоря Сікорського"
Семенов Віталій, сеніор інженер, SoftHouseGroup.

Для студентів, що здобувають освіту в галузі знань „Інформаційні технології” для спеціальностей 122 - “Комп’ютерні науки”, 124 - “Системний аналіз”, 113 - ”Прикладна математика” та інших спеціальностей, що пов’язані із вивченням комп’ютерних інформаційних технологій. УДК 004.738.5

© Косован В.М., Матвій О.В., Мельник Г.В., Мельник В.С.

© Чернівецький національний університет, 2026

Зміст

1 Збір даних. 7

1.1 Збір даних з веб-джерел. . . . 7
1.2 RESTful API . . . 8
1.3 CSV-файли. . . . 9
1.4 Парсинг JSON в Python. . . . 10
1.5 Парсинг XML/HTML в Python. . . . 11
1.6 Регулярні вирази. . . . 13
1.6.1 Відповідність декількох потенційних символів. . . . 14
1.6.2 Групування. . . . 14
1.6.3 Заміни. . . . 15
1.6.4 Порядок і жадібне зіставлення. . . . 15

2 Обробка даних. 18

2.1 Pandas . . . 18
2.2 Методи очищення даних та обробка відсутніх значень. . . . 23
2.2.1 Видалення відсутніх даних. . . . 24
2.2.2 Заповнення середнім. . . . 25
2.2.3 Заміщення медіанним значенням. . . . 25
2.2.4 Заміщення модою. . . . 26
2.3 Дублікати. . . . 26
2.4 Масштабування та нормалізація даних. . . . 27
2.4.1 Викиди. . . . 27
2.4.2 Мінімально-максимальне масштабування. . . . 28
2.4.3 Нормалізація (стандартизація) за Z-балом. . . . 29
2.4.4 Надійне масштабування. . . . 30
2.5 Вступ до конструювання ознак. . . . 31
2.5.1 Створення нових ознак. . . . 31
2.5.2 Кодування категоріальних змінних. . . . 32
2.5.3 Функція взаємодії. . . . 33
2.5.4 Поліноміальні ознаки. . . . 33

3 Візуалізація даних. 37

3.1 Типи даних. . . . 37
3.2 1D дані . . . 38
3.2.1 Стовпчасті діаграми — категоріальні дані . . . 38
3.2.2 Кругові діаграми. . . . 39
3.2.3 Гістограми — числові дані . . . 40
3.3 2D дані . . . 41
3.3.1 Точкові діаграми — числові x числові. . . . 41

3.3.2 Лінійна діаграма - числові x числові. 42
3.3.3 Коробкова і скрипкові діаграми — категоріальні x числові 43
3.3.4 Теплова карта та бульбашкові діаграми — категоріальні x категоріальні . . 45
3.4 3D дані 47
3.4.1 Матриці розсіювання (точкові матриці). 47
3.4.2 Бульбашкові діаграми. 48

4 Лінійна алгебра та бібліотека NumPy. 52
4.1 Основи лінійної алгебри. 52
4.1.1 Порядок рядків і стовпців. 52
4.1.2 Основні операції. 53
4.2 Масиви NumPy. 53
4.2.1 Індексація в масивах NumPy. 55
4.2.2 Основні операції над масивами в NumPy. 56
4.2.3 Трансляція NumPy. 58
4.2.4 Операції лінійної алгебри в NumPy. 58
4.2.5 Складність операцій. 60
4.2.6 Розріджені матриці. 60

5 Обробка Природної Мови. 63
5.1 Попередня обробка тексту. 63
5.1.1 Очищення тексту. 63
5.1.2 Токенізація, стемінг, лематизація, нормалізація. 65
5.2 Чисельне представлення тексту. 68
5.2.1 Хмара слів та мішок слів. 68
5.2.2 TFIDF та косинусна подібність тексту. 69
5.2.3 Векторне подання слів. 73

6 Теорія ймовірності в контексті аналізу даних. 77
6.1 Основи теорії ймовірності. 77
6.1.1 Закон великих чисел. 78
6.2 Використання NumPy для роботи з розподілами ймовірностей. 79
6.2.1 Рівномірний розподіл. 79
6.2.2 Гаусівський розподіл. 79
6.2.3 Дискретні розподіли. 80
6.3 Теорема Байєса. 81

7 Вступ в машинне навчання. 84
7.1 Градієнтний спуск. 86
7.1.1 Трохи більше про лінійну регресію. 92
7.1.2 Біблотека scikit-learn. 93
7.2 Трохи більше про машинне навчання. 95
7.2.1 Приклади алгоритмів машинного навчання. 96

8 Лінійна Класифікація та Логістична Регресія. 98
8.1 Лінійна Класифікація. 98
8.1.1 Приклад лінійної класифікації: класифікація раку молочної залози 98
8.1.2 Функція втрат. 99
8.2 Логістична регресія. 100
8.2.1 Метрики моделей машинного навчання. 103

9 Нелінійні Алгоритми Машинного Навчання. 106
9.1 Поліноміальна Регресія. . . . 108
9.1.1 Узагальнення та перенавчання. . . . 110
9.1.2 k-кратна перехресна валідація. . . . 110
9.2 Поліноміальна класифікація. . . . 110
9.3 Метод Опорних Векторів. . . . 113
9.4 Наївний баєсів класифікатор. . . . 117
9.5 Дерева прийняття рішень. . . . 119

10 Ансамблеве навчання. 125
10.1 Випадковий ліс. . . . 125
10.2 XGBoost. . . . 128
10.2.1 Пошук оптимальних гіперпараметрів. . . . 130
10.3 Компроміс зсуву та дисперсії 130

11 Навчання без учителя. 133
11.1 Кластеризація методом к–середніх (k-means clustering). . . . 133
11.1.1 Вибір k: метод ліктя. . . . 135
11.2 Метод головних компонент. . . . 136

12 Перевірка статистичних гіпотез. 142
12.1 Ключові поняття в тестуванні гіпотез 142
12.1.1 Перевірка гіпотез у машинному навчанні 143
12.1.2 Практичний приклад: A/B-тестування з перевіркою гіпотез 144
12.2 Приклади статистичних перевірок. . . . 144
12.2.1 Одновибірковий t-критерій. . . . 144
12.2.2 Однофакторний дисперсійний аналіз. . . . 145
12.2.3 Одновибірковий критерій хі-квадрат. . . . 145
12.2.4 Критерій незалежності хі-квадрат. . . . 146
12.2.5 Коли вибрати який тест? 146

13 Рекомендаційні Системи. 148
13.1 Колаборативна фільтрація. . . . 148
13.1.1 Фільтрація на основі подібності користувачів. . . . 149
13.1.2 Холодний старт. . . . 151
13.2 Матрична факторизація. . . . 152
13.3 Метрики рекомендаційних систем. . . . 152
13.4 Етичні проблеми та сучасні підходи. . . . 153

14 Часові Ряди. 155
14.1 Основи роботи з часовими рядами. Стаціонарність часового ряду. . . . 155
14.1.1 Доповнений тест Дікі - Фуллера (ADF). . . . 158
14.2 Автокореляція. . . . 159
14.3 Функції втрати. . . . 160
14.4 ARIMA. . . . 161
14.5 Передбачення із використанням методів машинного навчання. . . . 164
14.6 Експоненційне зглажування. . . . 165

15 Моделювання Кризових Явищ. **169**
15.1 Передбачення фінансових криз. 169
15.2 Моделювання розповсюдження епідемій. 173

16 Використання нейронних мереж для аналізу даних. **178**
16.1 Вступ до нейронних мереж. Архітектура нейронної мережі. 178
16.2 Навчання нейронної мережі. 180
16.3 Згорткові нейронні мережі (CNN). 180
16.4 Transformer-моделі. 180
16.5 Переваги та недоліки нейронних мереж для аналізу даних. Сучасні напрями розвитку нейронних мереж. 180

Розділ 1

Збір даних.

Першим етапом практично будь-якого проєкту з інтелектуального аналізу даних, машинного навчання або аналітики є збір даних (Data Collection). Якість отриманих результатів значною мірою залежить від якості та повноти вихідних даних. Навіть найскладніші алгоритми машинного навчання не здатні компенсувати відсутність важливої інформації або низьку якість даних, тому процес їх отримання є одним із найважливіших етапів підготовки аналітичного проєкту.

Сучасні дані можуть надходити з великої кількості різноманітних джерел. Це можуть бути реляційні бази даних, електронні таблиці, CSV-файли, журнали подій (logs), сенсори Інтернету речей (IoT), соціальні мережі, веб-сайти, а також численні програмні інтерфейси (API), що надають доступ до інформаційних сервісів. У деяких випадках дані вже доступні у вигляді готових файлів і можуть бути безпосередньо завантажені для подальшого аналізу. В інших ситуаціях необхідно виконувати запити до баз даних або зовнішніх інформаційних систем для отримання актуальної інформації.

Автоматизований збір даних є значно швидшим, дозволяє обробляти великі обсяги інформації, зменшує кількість людських помилок і забезпечує регулярне оновлення даних без постійного втручання користувача. Основними труднощами під час збору даних є різні формати даних, неоднакова структура, дублікати, пропущені значення, обмеження доступу, необхідність автентифікації та різна якість отриманої інформації.

В цій темі ми поговоримо про збір даних з двох основних джерел:

- запит до API (більшість з яких сьогодні є веб-орієнтованими); та
- вилучення даних з веб-сторінки за допомогою технологій веб-скрапінгу (Web Scraping).

1.1 Збір даних з веб-джерел.

Веб-джерело даних — це будь-який ресурс у мережі Інтернет, з якого можна отримати інформацію, наприклад веб-сторінка, REST API або онлайн-база даних. Для збору даних можуть використовуватися веб-сайти, REST API, XML- та JSON-сервіси, відкриті набори даних, хмарні сервіси та веб-додатки.

Сьогодні переважна більшість автоматизованих систем обміну даними використовує протокол HTTP (HyperText Transfer Protocol). Хоча спочатку HTTP був створений для передачі вебсторінок між браузерами та веб-серверами, з часом він став універсальним механізмом взаємодії між програмними системами. Саме на основі HTTP функціонує більшість сучасних веб-сервісів, REST API та хмарних платформ.

Під час збору даних клієнтська програма надсилає HTTP-запит до віддаленого сервера, а сервер повертає відповідь, яка може містити HTML-документ, JSON-структуру, XML-файл, зображення або інші типи даних. Розуміння принципів роботи HTTP є необхідною передумовою

для ефективного використання веб-джерел даних, тому в наступних розділах буде розглянуто основні типи HTTP-запитів, структуру відповідей сервера та способи обробки отриманих даних у Python.

HTTP-відповідь — це повідомлення, яке сервер надсилає клієнту після обробки HTTP-запиту. Вона містить код стану, заголовки та тіло відповіді, у якому можуть передаватися HTML, JSON, XML або інші дані.

Веб-скрапінг — це автоматизований процес отримання та вилучення інформації з HTML-сторінок веб-сайтів для її подальшого аналізу або обробки. Процес веб-скрапінгу включає отримання HTML-сторінки, аналіз її структури, пошук потрібних елементів, вилучення необхідних даних та їх подальше збереження або обробку.

Процес веб-скрапінгу складається із надсилання запиту, одержання відповіді, парсингу тексту, витягу очищення та збереження даних.

```
import requests
response = requests.get("https://fmi.chnu.edu.ua/")

print("Status Code:", response.status_code)
print("Headers:", response.headers)

print(response.text)
```

Результат:

```
Status Code: 200
Headers: {'Date': 'Fri, 12 Sep 2025 05:44:43 GMT', 'Content-Type': 'text/html; charset=utf-8' ...
```

Коди стану HTTP повідомляють клієнту про результат обробки запиту сервером. Код 200 (OK) означає, що запит успішно виконано. Код 404 (Not Found) вказує на те, що запитуваний ресурс не знайдено. Код 500 (Internal Server Error) свідчить про виникнення внутрішньої помилки на сервері під час обробки запиту.

HTTP GET є найпоширенішим методом, але існують також методи PUT, POST, DELETE, які змінюють деякі параметри на сервері.

Параметри URL можна вказати за допомогою бібліотеки запитів, як показано нижче:

```
params = {"sa":"t", "rct":"j", "q":"", "esrc":"s",
"source":"web", "cd":"9", "cad":"rja", "uact":"8"}
response = requests.get("http://www.google.com/url", params=params)

print("Final URL:", response.url)
print(response.text)
```

Результат:

```
Final URL: http://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=9&cad=rja&uact=8
<html lang="uk"><head><meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1"><title>&#1055
    ...
```

1.2 RESTful API

Якщо ви перейдете від простого запиту веб-сторінок до веб-API, ви, швидше за все, зіткнетеся з REST API (Representational State Transfer). REST — це скоріше архітектура дизайну, але ось

кілька ключових моментів:

- Використовує стандартний інтерфейс і методи HTTP (GET, PUT, POST, DELETE)
- Stateless – сервер не запам'ятовує, що ви робили
- Практичне правило: якщо ви надсилаєте ключ свого облікового запису разом із кожним викликом API, ви, ймовірно, використовуєте REST API

REST API забезпечує стандартизований, швидкий і надійний доступ до актуальних даних, дозволяє автоматизувати їх отримання та легко інтегрується з різними програмними системами. Ви запитуєте REST API подібно до стандартних HTTP-запитів, але майже завжди потрібно включати параметри. Параметри можуть передаватися як частина URL-адреси, через рядок запиту (Query Parameters), у заголовках HTTP або в тілі запиту залежно від використовуваного HTTP-методу.

Автентифікація API — це процес підтвердження особи користувача або програми перед наданням доступу до ресурсів сервера. Найпоширенішими способами є API Key, Bearer Token (JWT), Basic Authentication та OAuth 2.0.

Отримайте власний токен доступу на https://github.com/settings/tokens/new GitHub API використовує GET/PUT/DELETE, щоб ви могли автоматично запитувати або оновлювати елементи у вашому обліковому записі GitHub.

☞ Сервер не запам'ятовує ваші останні запити, ви завжди повинні включати свій токен доступу, якщо використовуєте його таким чином.

Приклад REST запиту:

```
import os
from dotenv import load_dotenv

load_dotenv()
token = os.getenv("GITHUB_TOKEN")

headers = {'Authorization': 'Bearer '+token}
response = requests.get("https://api.github.com/user", headers=headers)
print(response.content)
```

Результат:

```
b'{"login":"maxvonlancaster","id":38358502,"node_id":"MDQ6VXNlcjM4MzU4NTAy","avatar_url": ...
```

REST-архітектура базується на взаємодії клієнта і сервера, використанні стандартних HTTP-методів, відсутності збереження стану між запитами (stateless), єдиному інтерфейсі доступу до ресурсів та представленні ресурсів за допомогою URI. Дані найчастіше передаються у форматах JSON або XML.

HTTP-заголовки використовуються для передачі службової інформації між клієнтом і сервером. Вони можуть містити тип передаваних даних, параметри автентифікації, інформацію про кешування, мову, кодування, тип клієнта та інші параметри, необхідні для коректної обробки запиту або відповіді.

1.3 CSV-файли.

CSV (Comma-Separated Values): Відноситься до будь-якого текстового файлу з роздільниками (не завжди розділеного комами).

Якщо самі значення містять коми, ви можете взяти їх у лапки (наш реєстратор, очевидно, завжди так робить, щоб бути впевненим).

Імпортуємо дані із csv-файлу, що містить дані про історичну ціну біткоїна, та зобразимо ці дані, за допомогою наступного коду.

```
import pandas as pd

dataframe = pd.read_csv("../resources/bitcoin.csv", delimiter=',', quotechar='"')
dataframe.head(10)
```

Результат:

```
Date  Open  High  Low   Close Volume   Currency
0  2010-07-18  0.0   0.1   0.1   0.1   75     USD
1  2010-07-19  0.1   0.1   0.1   0.1   574       USD
2  2010-07-20  0.1   0.1   0.1   0.1   262       USD
3  2010-07-21  0.1   0.1   0.1   0.1   575       USD
4  2010-07-22  0.1   0.1   0.1   0.1   2160  USD
5  2010-07-23  0.1   0.1   0.1   0.1   2403  USD
6  2010-07-24  0.1   0.1   0.1   0.1   496       USD
7  2010-07-25  0.1   0.1   0.1   0.1   1551  USD
8  2010-07-26  0.1   0.1   0.1   0.1   877       USD
9  2010-07-27  0.1   0.1   0.1   0.1   3374  USD
```

Формат CSV є простим, компактним і зрозумілим, підтримується більшістю програм для роботи з даними та легко обробляється різними мовами програмування. Він забезпечує швидке зчитування й запис табличних даних, що робить його одним із найпоширеніших форматів для обміну інформацією.

☞ Формат CSV не підтримує вкладені структури даних, типи даних, метадані та форматування. Крім того, можуть виникати проблеми через використання різних роздільників, кодувань символів або символів нового рядка, що іноді ускладнює обмін даними між різними програмами.

1.4 Парсинг JSON в Python.

JSON (JavaScript Object Notation) — це легкий текстовий формат обміну даними, який використовується для передачі структурованої інформації між програмами. Незважаючи на назву, JSON сьогодні є незалежним від JavaScript і підтримується практично всіма сучасними мовами програмування.

JSON широко використовується у REST API, веб-сервісах, мобільних застосунках та хмарних системах. Основними перевагами JSON є компактність, простота читання людиною та легкість автоматичної обробки.

Вбудована бібліотека для читання/запису об'єктів Python з/у файли JSON, імпортується за допомогою команди 'import json'. Нижче наведено приклад роботи із json-даними в python. REST-запит вертає json дані, які ми потім відображаємо.

```
import json
# load json response from REST API

headers = {'Authorization': 'token ' + token}
response = requests.get("https://api.github.com/user", headers=headers)
data = json.loads(response.content)

#json.load(file) # load json from file
```

```
json.dumps(data) # return json string
#json.dump(obj, file) # write json to file
```

Результат:

```
'{"login": "maxvonlancaster", "id": 38358502, "node_id": "MDQ6VXNlcjM4MzU4NTAy", "avatar_url": ...
```

JSON підтримує рядки, числа, логічні значення, масиви, об'єкти, а також спеціальне значення ‘null‘. JSON-файл можна зчитати за допомогою стандартного модуля ‘json‘, використовуючи функцію ‘json.load()‘, або за допомогою бібліотеки ‘pandas‘, якщо дані мають табличну структуру.

Серіалізація JSON — це процес перетворення об'єкта Python у JSON-рядок або JSON-файл для його збереження чи передачі. Десеріалізація JSON — це процес перетворення JSON-рядка або JSON-файлу у відповідні об'єкти Python.

Після десеріалізації json в python значення вкладених полів отримують шляхом послідовного звернення до вкладених словників або списків за допомогою ключів та індексів.

Нижче наведено приклад роботи з вкладеними структурами в JSON:

```
import json

json_string = '''
{
    "employee":
    {
        "name":"John",
        "department":
        {
            "name":"IT",
            "floor":4
        }
    }
}
'''

obj = json.loads(json_string)

print(obj["employee"]["department"]["name"])
```

Результат:

```
IT
```

1.5 Парсинг XML/HTML в Python.

XML (eXtensible Markup Language) — це мова розмітки, призначена для зберігання, структурування та обміну даними між різними інформаційними системами. XML-документ складається з декларації, кореневого елемента, вкладених тегів, атрибутів та текстового вмісту. XML використовує теги для опису структури даних і є більш громіздким, тоді як JSON має компактніший синтаксис, легше читається людиною та швидше обробляється програмами.

Існує ряд XML/HTML-парсерів для Python, але для науки про дані добре підходить бібліотека BeautifulSoup (спеціально призначена для вилучення даних з XML/HTML-файлів). Нижче наведено приклад завантаження html-розмітки із сайту, та код, що знаходить всі anchor-посилання всередені елементів ‘ ‘.

```
from bs4 import BeautifulSoup
import requests

response = requests.get("https://mathmod.chnu.edu.ua/")

root = BeautifulSoup(response.content)
root.find("div").findAll("a")


item = root.find(class_="flex-item")
print(item.text)


titles = root.select("div.flex-item h2")

for t in titles:
    print(t.text)


link = root.find("a")

print(link["href"])
print(link.text)


table = root.find("table")

rows = table.find_all("tr")

for row in rows:
    cells = row.find_all("td")
    print([cell.text.strip() for cell in cells])
```

В цьому коді ми спочатку: отримуємо всі посилання на веб-сторінці, потім шукаємо елемент за класом, CSS селектором, отримуємо атрибути тега та, в кінці, отримуємо всі рядки таблиці.

Результат:

```
[<a href="/cdn-cgi/l/email-protection#77141b105a1a16031f37141f190259121302590216"><span><span
    class="__cf_email__" data-cfemail="a0c3ccc78dcdc1d4c8e0c3c8ced58ec5c4d58ed5c1">[email
    protected]</span></span></a>,
 <a href="https://www.facebook.com/fmi.org.ua/" rel="noopener" target="_blank" title=".."><span
    class="iconify-icon footer-social-link-icon icon--fa-brands icon--fa-brands--facebook-f"></span></a>,
 <a href="https://www.instagram.com/m._i_.f/" rel="noopener" target="_blank" title=".."><span
    class="iconify-icon footer-social-link-icon icon--fa-brands icon--fa-brands--instagram"></span></a>,
 <a href="https://youtube.com/@MathTube_" rel="noopener" target="_blank" title="Youtube"><span
    class="iconify-icon footer-social-link-icon icon--fa-brands icon--fa-brands--youtube"></span></a>]
```

HTML-документ — це текстовий файл, який містить структуру веб-сторінки, описану за допомогою HTML-тегів. HTML є основним джерелом інформації під час веб-скрапінгу, оскільки саме в ньому містяться текст, таблиці, посилання, зображення та інші елементи веб-сторінки.

HTML-елемент за тегом можна знайти за допомогою методів `find()` або `find_all()`, вказавши назву потрібного тегу. Елементи знаходять, вказуючи значення атрибутів, наприклад "class "id"або інших, використовуючи параметри методів `find()` чи `find_all()`.

DOM (Document Object Model) — це деревоподібне представлення HTML-документа, у якому кожен HTML-елемент є окремим вузлом, доступним для аналізу та модифікації.

1.6 Регулярні вирази.

Після завантаження даних (або якщо вам потрібно створити парсер для завантаження даних іншого формату) часто доводиться шукати певні елементи в даних.

Регулярні вирази (Regular Expressions, скорочено RegEx або Regex) — це спеціальна мова шаблонів, призначена для пошуку, перевірки, виділення та заміни текстових фрагментів. Регулярні вирази широко застосовуються під час збору, очищення та попередньої обробки даних, оскільки дозволяють швидко знаходити рядки, що відповідають заданому шаблону. У задачах інтелектуального аналізу даних регулярні вирази використовуються для пошуку даних (електронна пошта, номери телефонів, дати/час), перевірки правильності введених даних, очищення тексту від службових символів, та попередньої підготовки текстів перед їх аналізом.

Наприклад, знайти перше входження рядка «data science»

```
import re
text = "This course will introduce the basics of data science"
match = re.search(r"data science", text)
print(match.start())
print(re.match(r"This", text))

match = re.match(r"data science", text)
match = re.search(r"data science", text)

all_matches = re.findall(r"a", text)
print(all_matches)
```

Тут метод `re.findall(r"a", text)` повертає всі збіги в тексті.

Результат:

```
41
<re.Match object; span=(0, 4), match='This'>
['a', 'a', 'a']
```

Під час збору та підготовки даних регулярні вирази використовуються для автоматичного вилучення структурованої інформації з текстів. Типовими прикладами є пошук електронних адрес, телефонних номерів, URL-адрес, IP-адрес, поштових індексів, дат, часу, грошових сум та інших шаблонних даних.

Крім пошуку інформації, регулярні вирази широко застосовуються для очищення текстів перед аналізом: видалення HTML-тегів, спеціальних символів, зайвих пробілів, службових позначок та інших небажаних елементів. Вони також використовуються для перевірки правильності введених даних (валідації) та підготовки текстових корпусів до подальшої обробки методами обробки природної мови (Natural Language Processing, NLP).

Завдяки простоті використання та високій швидкодії регулярні вирази є одним із базових інструментів інженера даних і спеціаліста з інтелектуального аналізу даних.

1.6.1 Відповідність декількох потенційних символів.

Справжня сила регулярних виразів полягає в можливості збігу декількох можливих послідовностей символів. Спеціальні символи в регулярних виразах: `.^$*+?{}\[]()|` (якщо ви хочете, щоб ці символи збігалися точно, вам потрібно їх екранувати: `\$`)

Відповідність наборів символів:

- Відповідність символу «a»: `a`
- Відповідність символів «a», «b» або «c»: `[abc]`
- Будь-який символ, крім «a», «b» або «c»: `[\^$abc]`
- Відповідність будь-якої цифри: `\d` (те саме, що ‘[0-9]‘)
- Відповідність будь-якого алфавітно-цифрового символу: `\w` (те саме, що `[a-zA-z0-9_]`)
- Відповідність пробілу: `\s` (те саме, що `[ \t\n\r\f\v]`)
- Відповідність будь-якого символу: ‘.‘ (включно з новою лінією з re.DOTALL)

Може збігатися з одним або декількома випадками символу (або набору символів)

Деякі поширені модифікатори:

- Відповідність символу «a» рівно один раз: ‘a‘
- Відповідність символу «a» нуль або один раз: ‘a?‘
- Відповідність символу «a» нуль або більше разів: ‘a*‘
- Відповідність символу «a» один або більше разів: ‘a+‘
- Відповідність символу «a» рівно n разів: ‘an‘

Можна поєднувати їх із збігом декількох символів:

- Відповідність усіх випадків «‘<something>‘ science», де ‘<something>‘ — це алфавітно-цифровий рядок, що містить принаймні один символ: `‘\w+\s+science‘`

1.6.2 Групування.

Часто нам потрібно отримати більше інформації, ніж просто те, чи знайшли ми збіг чи ні (наприклад, ми можемо хотіти знати, який текст збігся).

Групування: оберніть частини регулярного виразу в дужки, щоб «запам’ятати» ці частини збігу

`‘(\w+)\s([Aa]nalysis)‘`.

```
text = "This course will introduce the basics of data analysis"
match = re.search(r"(\w+)\s([Aa]nalysis)", text)
print(match.start(), match.groups())
```

Чому перед рядком стоїть «r»? Це дозволяє уникнути необхідності подвійного екранування рядків

Результат:

```
41 ('data', 'analysis')
```

1.6.3 Заміни.

Регулярні вирази надають механізм для заміни одного тексту іншим текстом.

```
better_text = re.sub(r"data analysis", r"schmada analysis", text)

better_text = re.sub(r"(\w+)\s([Aa])nalysis", r"\1 \2shnalysis", text)
print(better_text)
```

Щоб включити текст, який було збережено під час пошуку за допомогою груп, використовуйте екрановані послідовності \1, \2, ... у тексті заміни

Результат:

```
This course will introduce the basics of data ashnalysis
```

1.6.4 Порядок і жадібне зіставлення.

У регулярних виразах існує порядок операцій. ‘abc|def‘ відповідає рядкам «abc» або «def», а не «ab(c або d)ef». Ви можете обійти це, використовуючи дужки, наприклад ‘a(bc|de)f‘. Це також створює групу, використовуйте ‘a(?:bc|de)f‘, якщо ви не хочете її захоплювати.

За замовчуванням регулярні вирази намагаються захопити якомога більше тексту (жадібне співставлення, greedy matching). ‘<(.*)>‘, застосоване до ‘<a>text‘, буде відповідати всьому виразу. Якщо ви хочете захопити якомога менше тексту, використовуйте ‘<(.*?)>‘, це буде відповідати тільки терміну ‘<a>‘.

✎ **Питання для самоконтролю.**

1. Що таке збір даних (Data Collection) та яке його значення в процесі інтелектуального аналізу даних?
2. Які основні джерела даних використовуються в Data Mining?
3. Які переваги має автоматизований збір даних порівняно з ручним?
4. Які труднощі можуть виникати під час збору даних з різних джерел?
5. Що таке веб-джерело даних?
6. Які типи веб-джерел можуть використовуватися для збору даних?
7. Що таке веб-скрапінг (Web Scraping)?
8. Які етапи процесу веб-скрапінгу?
9. Які бібліотеки Python найчастіше використовуються для отримання веб-сторінок?
10. Для чого використовується бібліотека ‘requests‘ ?
11. Що таке HTTP-запит?
12. Які основні HTTP-методи використовуються при роботі з веб-ресурсами?
13. Що таке HTTP-відповідь?
14. Яке призначення кодів стану HTTP?
15. Що означають коди стану 200, 404 та 500?

16. Що таке RESTful API?
17. Які принципи лежать в основі REST-архітектури?
18. Чим REST API відрізняється від звичайної веб-сторінки?
19. Які переваги використання REST API для збору даних?
20. Як передаються параметри в REST API-запитах?
21. Що таке JSON-відповідь REST API?
22. Для чого використовуються HTTP-заголовки (headers)?
23. Що таке автентифікація API?
24. Які способи автентифікації API ви знаєте?
25. Що таке CSV-файл?
26. Які особливості структури CSV-файлів?
27. Які переваги має формат CSV?
28. Які недоліки має формат CSV?
29. Як зчитати CSV-файл у Python?
30. Для чого використовується бібліотека ‘pandas‘ при роботі з CSV?
31. Що таке JSON?
32. Які типи даних підтримує формат JSON?
33. Як зчитати JSON-файл у Python?
34. Що таке серіалізація JSON?
35. Що таке десеріалізація JSON?
36. Як отримати значення вкладених полів у JSON-структурі?
37. Що таке XML?
38. Які основні елементи XML-документа?
39. Чим XML відрізняється від JSON?
40. Як виконати парсинг XML у Python?
41. Що таке HTML-документ?
42. Яку роль HTML відіграє у веб-скрапінгу?
43. Для чого використовується бібліотека BeautifulSoup?
44. Як знайти HTML-елемент за тегом?
45. Як знайти HTML-елемент за атрибутом або класом?
46. Що таке DOM-структура веб-сторінки?
47. Що таке регулярний вираз (Regular Expression)?

48. Для яких задач використовуються регулярні вирази під час збору даних?
49. Які основні метасимволи регулярних виразів ви знаєте?
50. Як за допомогою регулярних виразів знаходити та витягувати потрібні фрагменти тексту?

Розділ 2

Обробка даних.

Обробка даних (Data Processing) — це сукупність методів і процедур, спрямованих на підготовку даних до подальшого аналізу та побудови моделей машинного навчання. Основними етапами обробки є очищення даних, усунення пропущених значень і дублікатів, виявлення викидів, перетворення типів даних, а також масштабування та нормалізація ознак. Якість виконання цих етапів безпосередньо впливає на точність, швидкість і надійність моделей машинного навчання, оскільки алгоритми навчаються саме на підготовлених даних. «Сирі» дані часто містять пропущені значення, дублікати, помилки введення, викиди, некоректні формати та неоднорідні масштаби ознак, що може призводити до погіршення якості аналізу та зниження точності прогнозування.

2.1 Pandas

Існує ряд бібліотек Python, які обробляють реляційні дані, зазвичай написані як інтерфейси до декількох різних систем управління реляційними базами даних (таких як PostgreSQL, MySQL та інші).

Примітка: слід зазначити, що програмне забезпечення, таке як PostreSQL і MySQL, правильніше називати системами управління реляційними базами даних (RDBMS), а не базами даних. База даних — це фактичні таблиці та записи, що визначають фактичну сукупність даних.

Однак у цьому курсі ми будемо працювати з реляційними даними переважно за допомогою двох бібліотек: Pandas і SQLite. Це особливо прості бібліотеки, якщо говорити про реальні бази даних: Pandas, безумовно, не є реальною системою реляційних баз даних (хоча вона надає функції, що віддзеркалюють деякі їхні можливості), тоді як SQLite є «реальною» RDBMS, але надзвичайно простою, без стандартної архітектури клієнт/сервер, яка є практично у всіх реальних виробничих базах даних. Проте для багатьох задач у галузі науки про дані вони будуть достатніми, тому ми зосередимося на них тут.

Ми вже коротко розглянули Panda, коли обговорювали збір даних, і вона виявилася однією з найкорисніших бібліотек Python для науки про дані. Як ми вже згадували вище (але будемо повторювати цей факт багато разів), Pandas — це не бібліотека реляційних баз даних, а бібліотека «даних-рам» (DataFrame). Ви можете уявити собі фрейм даних як 2D-масив, за винятком того, що записи у фреймі даних можуть бути будь-яким типом об'єкта Python (і мати змішані типи в масиві), а рядки/стовпці можуть мати «мітки» замість цілочисельних індексів, як у стандартному масиві.

Давайте подивимося, як спочатку створити фрейм даних у Pandas, який відображає таблицю Person із записами про деяких людей.

```
import pandas as pd
```

```
df = pd.DataFrame([(1, 'Ivanov', 'Ivan'),
                   (2, 'Petrov', 'Petro'),
                   (3, 'Kosovan', 'Vasyl'),
                   (4, 'Cherevko', 'Igor'),
                   (5, 'Kravtsov', 'Kyryl'),
                   (6, 'Rarenko', 'Anastasiya'),],
                  columns=["id", "last_name", "first_name"])
df
```

Результат:

```
   id   last_name   first_name
0  1  Ivanov   Ivan
1  2  Petrov   Petro
2  3  Kosovan  Vasyl
3  4  Cherevko Igor
4  5  Kravtsov Kyryl
5  6  Rarenko  Anastasiya
```

Ви, напевно, пам'ятаєте, що таке первинний ключ в базі даних. Якщо ні, можна коротко згадати, що це одне (або декілька) полів (або стовпців) у таблиці бази даних, які унікально ідентифікують кожен її рядок.

«Індекс» для Pandas насправді означає щось більше, ніж «первинний ключ» у таблиці бази даних (хоча б з тим винятком, що можливі дублікати записів).

Тобто індекс (якщо він зроблений правильно, без дублікатів) є ідентифікатором для кожного рядка в базі даних. Ми можемо встановити індекс для одного з існуючих стовпців за допомогою виклику `.set_index()`.

Але тут потрібно бути дуже обережним з одним моментом. За замовчуванням більшість операцій Pandas, таких як `.set_index()` та багато інших, не виконуються на місці. Тобто, хоча виклик `df.set_index(«id»)` вище повертає копію фрейму даних df з індексом, встановленим для стовпця id (пам'ятайте, що Jupyter notebook відображає значення, яке повертається в останньому рядку комірки), оригінальний об'єкт `df` насправді тут не змінюється.

Якщо ми хочемо фактично змінити сам об'єкт `df`, потрібно використовувати прапор `inplace=True` для цих функцій (або присвоїти оригінальний об'єкт результату функції, але це не так чітко).

```
df.set_index("id", inplace=True)
df
```

Результат:

```
   last_name   first_name
id
1  Ivanov   Ivan
2  Petrov   Petro
3  Kosovan  Vasyl
4  Cherevko Igor
5  Kravtsov Kyryl
6  Rarenko  Anastasiya
```

Series є одновимірною структурою, тоді як DataFrame є двовимірною. Наприклад, один стовпець таблиці: `df['last_name']` повертає об'єкт Series.

Якщо ви захочете завантажити дані із CSV-файлу у DataFrame, це можна зробити наступним чином:

```
import pandas as pd

data = pd.read_csv('users.csv')
```

Для отримання загальної інформації про структуру DataFrame використовується метод info(). Він показує кількість рядків, назви стовпців, кількість непорожніх значень і типи даних. Наприклад, за його допомогою можна швидко визначити, у яких стовпцях є пропущені значення.

```
df.info()
```

Результат:

```
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 1 to 6
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   last_name   6 non-null      str
 1   first_name  6 non-null      str
dtypes: str(2)
memory usage: 303.0 bytes
```

Для отримання розміру таблиці використовують атрибут shape: `df.shape`. Він повертає кортеж із кількістю рядків і стовпців.

Для статистичного опису числових даних використовується метод `df.describe()`. Він автоматично обчислює основні статистичні характеристики числових стовпців, зокрема кількість значень, середнє значення, стандартне відхилення, мінімальне та максимальне значення, а також квартилі.

Розглянемо іще деякі методи для роботи із фреймами в Pandas.

Для перегляду перших записів використовується метод `head()`. Наприклад, перші три рядки:

```
df.head(3)
```

Результат:

```
   last_name   first_name
id
1  Ivanov   Ivan
2  Petrov   Petro
3  Kosovan  Vasyl
```

Для перегляду останніх записів використовується метод `tail()`. Наприклад, останні три рядки:

```
df.tail(3)
```

Результат:

```
   last_name   first_name
id
4  Cherevko Igor
5  Kravtsov Kyryl
6  Rarenko  Anastasiya
```

Доступ до окремих елементів можна отримати за допомогою нотації `.loc[row, column]`, де row позначає індекс, який ви шукаєте, а column - назву стовпця.

```
df.loc[1, "last_name"]
```

Результат:

```
'Ivanov'
```

Якщо ми хочемо отримати доступ до всіх прізвищ (або до всіх елементів у певному рядку), ми використовуємо символ-замінник :.

```
df.loc[:, "last_name"]
```

Результат:

```
id
1      Ivanov
2      Petrov
3     Kosovan
4    Cherevko
5    Kravtsov
6     Rarenko
Name: last_name, dtype: str
```

Ми можемо передати список потрібних стовпців, щоб отримати об'єкт типу DataFrame.

```
df.loc[:, ["last_name"]]
```

Результат:

```
    last_name
id
1  Ivanov
2  Petrov
3  Kosovan
4  Cherevko
5  Kravtsov
6  Rarenko
```

Те саме можна зробити з індексами рядків.

```
df.loc[[1,2],:]
```

Результат:

```
   last_name   first_name
id
1  Ivanov   Ivan
2  Petrov   Petro
```

Крім того, ми можемо використовувати ‘.loc‘ для зміни вмісту існуючих записів.

```
df.loc[6,"last_name"] = "Vikovan"
df
```

Результат:

```
   last_name   first_name
id
1  Ivanov   Ivan
```

```
2  Petrov   Petro
3  Kosovan  Vasyl
4  Cherevko Igor
5  Kravtsov Kyryl
6  Vikovan  Anastasiya
```

Ми можемо навіть додати додаткові рядки/стовпці.

```
df.loc[7,:] = ('Vikovan', 'Valentyn')
df
```

Результат:

```
last_name   first_name
id
1  Ivanov   Ivan
2  Petrov   Petro
3  Kosovan  Vasyl
4  Cherevko Igor
5  Kravtsov Kyryl
6  Vikovan  Anastasiya
7  Vikovan  Valentyn
```

☞ Наостанок пам'ятайте, що '.loc' завжди індексує на основі «індексу» (тобто, фактично, первинного ключа) фрейму даних разом з назвою стовпця. Якщо ви хочете отримати доступ на основі позиційного індексу (тобто, використовуючи лічильники з нульовим індексом як для рядків, так і для стовпців), ви можете скористатися властивістю `.iloc`.

```
df.iloc[4,1]
```

Результат:

```
'Kyryl'
```

Рядки DataFrame можна відфільтрувтаи за певною умовою:

```
filtered = df[df['first_name'].str.contains('I')]
display(filtered)
```

Результат:

```
   last_name   first_name
id
1  Ivanov   Ivan
4  Cherevko Igor
```

Можна також комбінувати декілька умов.

Новий стовпець можна створити за допомогою оператора присвоєння:

```
df['user_name'] = df['last_name'] + df['first_name']
display(df)
```

Результат:

```
   last_name   first_name  user_name
id
1  Ivanov   Ivan  IvanovIvan
2  Petrov   Petro PetrovPetro
```

```
3  Kosovan  Vasyl KosovanVasyl
4  Cherevko Igor  CherevkoIgor
5  Kravtsov Kyryl KravtsovKyryl
6  Vikovan  Anastasiya  VikovanAnastasiya
7  Vikovan  Valentyn VikovanValentyn
```

Для видалення стовпця використовується метод drop():

```
df = df.drop('user_name', axis=1)
```

Параметр axis=1 означає, що видаляється стовпець. Для видалення рядка можна використати його індекс та параметр axis=0.

Для перейменування використовують метод rename():

```
df = df.rename(columns={
    'last_name': 'Last_Name',
})
```

Для сортування використовується метод `sort_values()`:

```
df = df.sort_values('first_name')
```

2.2 Методи очищення даних та обробка відсутніх значень.

У реальному світі необроблені дані рідко бувають ідеальними. Вони часто містять помилки, невідповідності та відсутні значення.

Навіть найкращий алгоритм чи штучний інтелект не зможе побудувати хорошу модель, якщо навчальні дані містять помилки або пропуски. Наприклад, прогноз погоди буде ускладненим, якщо серед показників термометра за попередні дні затесалися такі дані як -500 градусів за Цельсієм, чи просто багато пропусків.

Очищення даних — це процес виявлення та виправлення цих проблем для поліпшення якості даних.

- *Дублікати*: це повторювані записи в наборі даних. Дублікати рядків можуть спотворити ваш аналіз або модель, особливо якщо вони представляють одну й ту саму подію або об'єкт.
- *Неправильні типи даних*: дані, які введені неправильно (наприклад, числа, збережені як рядки), можуть призвести до помилок в аналізі та навчанні моделі.
- *Випадкові значення*: екстремальні значення, які значно відрізняються від решти даних. Вони можуть спотворити результати статистичного аналізу або моделей машинного навчання.

Відсутність даних може відбуватися з різних причин, таких як помилки введення даних, несправності обладнання або проблеми з конфіденційністю. Важливо обережно поводитися з відсутніми значеннями, оскільки неправильне поводження може призвести до упереджених результатів.

Типи відсутніх даних:

- *MCAR* (Missing Completely at Random, повністю випадкова відсутність): ймовірність відсутності даних не залежить від самих даних.
- *MAR* (відсутність випадкова): ймовірність відсутності даних пов'язана з іншими спостережуваними даними, але не з самими відсутніми даними.

- *MNAR* (відсутність невипадкова): відсутність пов'язана з самими неспостережуваними даними.

2.2.1 Видалення відсутніх даних.

Видалення рядків або стовпців з відсутніми значеннями є найпростішим підходом, але може призвести до втрати цінної інформації.

У наступному прикладі метод `df.dropna()` видаляє всі рядки, у яких є хоча б одне відсутнє значення, а метод `df.dropna(subset=['feature1', 'feature2'])` видаляє рядки, у яких є хоча б одне відсутнє значення в числових стовпцях feature1 та feature2.

```
import pandas as pd

df = pd.read_csv('../resources/data.csv')
print(df, '\n')

df_cleaned = df.dropna()
df_feature_cleaned = df.dropna(subset=['feature1', 'feature2'])
print(df_cleaned, '\n')
print(df_feature_cleaned, '\n')
```

Результат:

```
   feature1  feature2 categorical_column  target
0       1.2       3.2                  A      10
1       2.3       NaN                  B      15
2       3.1       1.1                  C       7
3       4.7       4.8                  A      18
4       NaN       2.7                  B      11
5       5.1       4.2                  C      19
6       2.8       NaN                  A      13
7       4.1       3.9                  B      16
8       5.3       4.1                NaN      14
9       3.3       2.1                  C      12

   feature1  feature2 categorical_column  target
0       1.2       3.2                  A      10
2       3.1       1.1                  C       7
3       4.7       4.8                  A      18
5       5.1       4.2                  C      19
7       4.1       3.9                  B      16
9       3.3       2.1                  C      12

   feature1  feature2 categorical_column  target
0       1.2       3.2                  A      10
2       3.1       1.1                  C       7
3       4.7       4.8                  A      18
5       5.1       4.2                  C      19
7       4.1       3.9                  B      16
8       5.3       4.1                NaN      14
9       3.3       2.1                  C      12
```

За допомогою методу `df.isna()` можна виявити пропущенні значення у DataFrame. Щоб отримати кількість пропусків у кожному стовпці, зручно використати `df.isna().sum()`

☞ Видалення рядків доцільне тоді, коли кількість пропущених записів невелика і їхнє видалення не призведе до істотної втрати інформації. Наприклад, якщо з 100 000 записів лише 20 містять пропущені значення, їх можна видалити без значного впливу на розмір вибірки. Водно-

час якщо пропуски присутні у значній частині даних або стосуються важливої ознаки, краще розглянути імпутацію. Особливо обережно потрібно працювати з пропусками у невеликих наборах даних, де видалення навіть декількох відсотків записів може суттєво змінити результати аналізу.

2.2.2 Заповнення середнім.

Імпутація даних (Data Imputation) — це процес заміни пропущених значень оціненими значеннями.

При заповненні середнім (Mean Imputation) ми замінюємо відсутні значення середнім значенням стовпця.

```
df['feature1'].fillna(df['feature1'].mean(), inplace=True)
df['feature2'].fillna(df['feature2'].mean(), inplace=True)
df
```

Результат:

```
feature1 feature2 categorical_column   target
0  1.200000 3.2000   A  10
1  2.300000 3.2625   B  15
2  3.100000 1.1000   C  7
3  4.700000 4.8000   A  18
4  3.544444 2.7000   B  11
5  5.100000 4.2000   C  19
6  2.800000 3.2625   A  13
7  4.100000 3.9000   B  16
8  5.300000 4.1000   NaN   14
9  3.300000 2.1000   C  12
```

2.2.3 Заміщення медіанним значенням.

При заміщенні медіанним значенням (Median Imputation) ми заміщуємо відсутні значення медіанним значенням. Цей метод є корисним, коли дані є викривленими.

```
df = pd.read_csv('../resources/data.csv')
df['feature1'].fillna(df['feature1'].median(), inplace=True)
df
```

Результат:

```
   feature1 feature2 categorical_column   target
0  1.2   3.2   A  10
1  2.3   NaN   B  15
2  3.1   1.1   C  7
3  4.7   4.8   A  18
4  3.3   2.7   B  11
5  5.1   4.2   C  19
6  2.8   NaN   A  13
7  4.1   3.9   B  16
8  5.3   4.1   NaN   14
9  3.3   2.1   C  12
```

2.2.4 Заміщення модою.

Заміщення модою (Mode Imputation) часто використовується для категоріальних даних.

```
df['categorical_column'].fillna(df['categorical_column'].mode()[0], inplace=True)
df
```

Результат:

```
feature1 feature2 categorical_column   target
0  1.2   3.2   A  10
1  2.3   NaN   B  15
2  3.1   1.1   C  7
3  4.7   4.8   A  18
4  3.3   2.7   B  11
5  5.1   4.2   C  19
6  2.8   NaN   A  13
7  4.1   3.9   B  16
8  5.3   4.1   A  14
9  3.3   2.1   C  12
```

☞ Інші поширені методи заповнення даних:

Імпутування K-найближчих сусідів (KNN): оцінює відсутні значення на основі значень найближчих сусідів.

Багатовимірне імпутування (підстановка): враховує взаємозв'язки між ознаками для імпутування відсутніх значень.

Створення індикаторних змінних: інший підхід полягає у створенні бінарної індикаторної змінної, яка позначає наявність відсутніх даних.

2.3 Дублікати.

Дублікати — це записи, які повністю або частково повторюють інші записи у наборі даних. Дублікати можуть виникати під час повторного завантаження даних, об'єднання декількох джерел або помилкового введення інформації.

Для перевірки дубльованих рядків використовується метод duplicated():

```
df.duplicated()
```

Він повертає True для рядків, які є дубльованими.

Щоб отримати самі дублікати:

```
df[df.duplicated()]
```

Кількість дублікатів можна визначити так:

```
df.duplicated().sum()
```

Для видалення дубльованих рядків використовується метод `drop_duplicates()`:

```
df = df.drop_duplicates()
```

Можна визначати дублікати лише за певними стовпцями.

```
df = df.drop_duplicates(subset=['feature1', 'feature2'])
```

2.4 Масштабування та нормалізація даних.

Масштабування та нормалізація даних (Data Scaling and Normalization) є важливими етапами попередньої обробки, які можуть суттєво вплинути на ефективність ваших моделей.

Масштабування даних гарантує, що всі ознаки однаково впливають на процес навчання моделі. Коли ознаки мають дуже різні діапазони, ті, що мають більші діапазони, можуть домінувати в обчисленнях відстані або кроках градієнта, що призводить до неоптимальних моделей.

Наприклад, якщо Age має значення від 18 до 80, а Salary — від 20 000 до 200 000, то без масштабування більші за величиною значення Salary можуть непропорційно впливати на роботу моделі.

Алгоритми на основі відстані: В алгоритмах, таких як K-найближчі сусіди (KNN) або машини опорних векторів (SVM), відстань між точками даних має вирішальне значення. Якщо одна ознака має набагато більший діапазон, ніж інші, вона непропорційно впливатиме на метрику відстані, що може спотворити результати.

Градієнтний спуск: Алгоритми, такі як лінійна регресія та нейронні мережі, використовують градієнтний спуск для мінімізації функції вартості. Якщо ознаки не масштабуються, алгоритм градієнтного спуску може потребувати більше часу для збіжності або може збігатися до неоптимального рішення через нерівномірні кроки, зроблені в просторі параметрів.

Масштабування - це загальна назва перетворень, які змінюють масштаб числових ознак. Воно може включати як приведення значень до певного інтервалу, так і стандартизацію.

Нормалізація у контексті підготовки даних часто означає перетворення значень до визначеного діапазону, наприклад від 0 до 1.

Тому мінімально-максимальна нормалізація є одним із видів масштабування, тоді як стандартизація є іншим способом масштабування.

2.4.1 Викиди.

Викид (Outlier) це спостереження, значення якого істотно відрізняється від більшості інших значень у вибірці.

Наприклад, якщо зарплати працівників знаходяться переважно в межах від 20 000 до 80 000 грн, значення 1 000 000 грн може бути потенційним викидом.

Викид не обов'язково є помилкою. Він може бути результатом помилки вимірювання або введення даних, але також може відображати реальне рідкісне явище. Тому викиди не слід автоматично видаляти.

Одним із найпоширеніших статистичних методів є правило міжквартильного розмаху (IQR). Значення вважається потенційним викидом, якщо воно знаходиться нижче $Q_1 - 1.5 \cdot IQR$ або вище $Q_3 + 1.5 \cdot IQR$, де Q_1 і Q_3 - перший та третій квартилі, а $IQR = Q_3 - Q_1$.

Також викиди можна виявляти графічно за допомогою boxplot, гістограм або scatter plot, а для складніших багатовимірних даних — за допомогою алгоритмів кластеризації та спеціальних методів виявлення аномалій. про це ми поговоримо в наступних розділах.

Викиди можуть суттєво змінювати статистичні характеристики вибірки. Наприклад, один надзвичайно великий показник може значно збільшити середнє значення.

Вони також можуть впливати на параметри деяких моделей машинного навчання. Особливо чутливими до викидів можуть бути методи, які використовують відстані або мінімізують квадрати помилок, наприклад лінійна регресія та k-середніх.

Однак видалення викидів також може бути небезпечним. Якщо викид є реальним явищем, його видалення означатиме втрату важливої інформації. Тому спочатку потрібно з'ясувати природу аномального значення.

2.4.2 Мінімально-максимальне масштабування.

Мінімально-максимальне масштабування (min-max normalization) перемасштабовує дані до фіксованого діапазону, зазвичай `[0, 1]`. Ця техніка особливо корисна, коли ви знаєте, що ваші дані відповідають розподілу з чіткими верхньою та нижньою межами.

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- Використовуйте мінімально-максимальне масштабування, коли ви хочете, щоб усі ваші ознаки мали однаковий масштаб (наприклад, в алгоритмах, таких як KNN, SVM).
- Особливо корисно, коли дані розподілені по відомому і фіксованому діапазону.

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

data = {'feature': [10, 50, 100, 150, 200]}
df = pd.DataFrame(data)

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)

print(scaled_data)
```

Результат:

```
[[0.       ]
 [0.21052632]
 [0.47368421]
 [0.73684211]
 [1.       ]]
```

Ми можемо вибрати окремі стовпці для масштабування через `numerical_scaled = scaler.fit_transfo`

В наступному прикладі ми спочатку вибираємо числові стовпці для масштабування через `numerical_cols = df[['feature1', 'feature2']]`, потім розділяємо стовпці категорій та цільові стовпці `categorical_cols = df[['categorical_column', 'target']]`, ініціалізуємо масштабування `scaler = MinMaxScaler()`, підготовуємо та перетворюємо числові стовпці `numerical_scaled =` перетворюємо нормалізовані дані назад у DataFrame `numerical_scaled_df = pd.DataFrame(numerical_sc` та об'єднуємо нормалізовані числові дані з категоріальними стовпцями `df_scaled = pd.concat([numerica`

```
df = pd.read_csv('../resources/data.csv')

numerical_cols = df[['feature1', 'feature2']]

categorical_cols = df[['categorical_column', 'target']]

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

numerical_scaled = scaler.fit_transform(numerical_cols)
```

```
numerical_scaled_df = pd.DataFrame(numerical_scaled,
    columns=numerical_cols.columns)

df_scaled = pd.concat([numerical_scaled_df,
    categorical_cols.reset_index(drop=True)], axis=1)

print(df_scaled)
```

Результат:

```
   feature1  feature2 categorical_column  target
0  0.000000  0.567568                 A      10
1  0.268293       NaN                 B      15
2  0.463415  0.000000                 C       7
3  0.853659  1.000000                 A      18
4       NaN  0.432432                 B      11
5  0.951220  0.837838                 C      19
6  0.390244       NaN                 A      13
7  0.707317  0.756757                 B      16
8  1.000000  0.810811               NaN      14
9  0.512195  0.270270                 C      12
```

☞ Основним недоліком Min-Max нормалізації є її чутливість до викидів. Якщо одна ознака містить надзвичайно велике або мале значення, x_{min} або x_{max} можуть сильно змінитися, внаслідок чого більшість нормалізованих значень опиниться у вузькому діапазоні. Крім того, якщо у майбутніх даних з'являться значення, що виходять за межі початкового діапазону, вони можуть отримати значення менше 0 або більше 1.

2.4.3 Нормалізація (стандартизація) за Z-балом.

Нормалізація (стандартизація) за Z-балом (Z-Score Normalization or standardization) перемасштабовує дані так, щоб середнє значення дорівнювало 0, а стандартне відхилення — 1. Ця техніка корисна, коли дані мають гаусівський (нормальний) розподіл, але не обов'язково обмежені.

$$X' = \frac{X - \nu}{\sigma}$$

- Використовуйте нормалізацію за Z-балом, коли ваші дані мають нормальний розподіл.
- Це стандартний вибір для багатьох моделей машинного навчання, особливо тих, що передбачають нормальний розподіл даних.

```
from sklearn.preprocessing import StandardScaler
import pandas as pd

data = {'feature': [10, 50, 100, 150, 200]}
df = pd.DataFrame(data)

scaler = StandardScaler()
standardized_data = scaler.fit_transform(df)

print(standardized_data)
```

Результат:

```
[[-1.35411306]
 [-0.76536825]
 [-0.02943724]
 [ 0.70649377]
 [ 1.44242478]]
```

Нормалізація дозволяє привести ознаки до єдиного діапазону, що особливо корисно для алгоритмів, які використовують відстані.

Вона запобігає ситуації, коли ознака з великими числовими значеннями домінує над ознаками з меншим масштабом. Крім того, нормалізація може покращити стабільність та швидкість оптимізації деяких моделей.

2.4.4 Надійне масштабування.

Надійне масштабування (Robust Scaling) використовує медіану та міжквартильний розмах (IQR) для масштабування, що робить його надійним щодо винятків. На відміну від мінімально-максимального масштабування та нормалізації Z-балу, надійне масштабування менше піддається впливу екстремальних значень у даних.

$$X' = \frac{X - \text{медіана}(X)}{\text{IQR}(X)}$$

Де $\text{IQR}(X)$ — діапазон між 25-м процентилем ($Q1$) і 75-м процентилем ($Q3$).

Використовуйте надійне масштабування, якщо ваші дані містять винятки або мають негаусівський розподіл. Воно ідеально підходить для ситуацій, коли дані є асиметричними або містять екстремальні значення, які не повинні сильно впливати на масштабування.

```python
from sklearn.preprocessing import RobustScaler
import pandas as pd

data = {'feature': [10, 50, 90, 100, 110, 150, 160, 200]}
df = pd.DataFrame(data)

scaler = RobustScaler()
robust_scaled_data = scaler.fit_transform(df)

print(robust_scaled_data)
```

Результат:

```
[[-1.42857143]
 [-0.85714286]
 [-0.28571429]
 [-0.14285714]
 [ 0.        ]
 [ 0.57142857]
 [ 0.71428571]
 [ 1.28571429]
 [34.14285714]]
```

☞ Для яких випадків підходить той чи інший метод масштабування?

Мінімально-максимальне масштабування: використовується, коли відомі межі даних або коли алгоритм вимагає даних у певному діапазоні.

Нормалізація за Z-балом: підходить для нормально розподілених даних або коли потрібно стандартизувати ознаки, щоб вони мали однакову важливість.

Надійне (Robust) масштабування: найкраще підходить для роботи з даними, що м істять винятки.

2.5 Вступ до конструювання ознак.

Інженерія ознак (Feature Engineering) є важливим етапом у процесі попередньої обробки даних. Вона передбачає перетворення необроблених даних на ознаки, які краще відображають основну проблему для моделей прогнозування. Якісна інженерія ознак може значно покращити ефективність моделей машинного навчання, надаючи їм найбільш релевантні та інформативні вхідні дані.

Для прикладу, маючи такі фізичні дані про людину як її вага та зріст, ми можемо сконструювати нову ознаку - "індекс маси тіла що буде значно користнішим для нашої моделі, ніж дві інші ознаки кожна окремо.

Інженерія ознак - це процес відбору, модифікації або створення нових ознак з необроблених даних для покращення ефективності моделей машинного навчання. Метою є створення ознак, які ефективніше відображають основні закономірності в даних, ніж вихідні необроблені дані. Правильна інженерія ознак може призвести до спрощення моделей, підвищення точності та скорочення часу навчання.

2.5.1 Створення нових ознак.

Створення нових ознак це виведення нових змінних з існуючих, які краще відображають основну структуру даних. Це може включати математичні перетворення, агрегації або операції, специфічні для певної області.

В наступному прикладі, маючи такі дані як вага та ріст людини, ми створюємо нову ознаку: BMI (body mass index, індекс маси тіла).

```
import pandas as pd

data = {
    'height': [1.60, 1.75, 1.82, 1.90],
    'weight': [55, 80, 72, 90]
}
df = pd.DataFrame(data)

df

df['BMI'] = df['weight'] / (df['height'] ** 2)

df
```

Результат:

```
   height   weight   BMI
0  1.60  55 21.484375
1  1.75  80 26.122449
2  1.82  72 21.736505
3  1.90  90 24.930748
```

2.5.2 Кодування категоріальних змінних.

Алгоритми машинного навчання зазвичай вимагають числових даних. Тому категоріальні змінні необхідно перетворити в числовий формат. Існує кілька методів кодування категоріальних змінних:

Кодування *One-hot* (унітарне кодування) перетворює кожну категорію в новий бінарний стовпець. Цей метод підходить для випадків, коли кількість категорій обмежена.

```
data = {'color': ['red', 'blue', 'green']}
df = pd.DataFrame(data)

# One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=['color'])

print(df_encoded)
```

Результат:

```
   color_blue  color_green  color_red
0           0            0          1
1           1            0          0
2           0            1          0
```

Кодування міток (Label Encoding) присвоює кожної категорії унікальне ціле число. Це простий метод, але він передбачає порядкові відносини між категоріями, що може бути недоречним у деяких випадках.

```
from sklearn.preprocessing import LabelEncoder

data = {'color': ['red', 'blue', 'green']}
df = pd.DataFrame(data)

# Label Encoding
encoder = LabelEncoder()
df['color_encoded'] = encoder.fit_transform(df['color'])

df
```

Результат:

```
   color color_encoded
0  red   2
1  blue  0
2  green 1
```

Цільове кодування (Target Encoding) передбачає заміну кожної категорії середнім значенням цільової змінної для цієї категорії. Ця техніка є корисною, коли категоріальна змінна має велику кількість рівнів.

```
data = {
    'color': ['red', 'blue', 'green', 'red', 'blue', 'green'],
    'target': [1, 0, 0, 1, 1, 0]
}
df = pd.DataFrame(data)
```

```
# Target Encoding
df['color_encoded'] = df.groupby('color')['target'].transform('mean')

print(df)
```

Результат:

```
   color  target  color_encoded
0    red       1            1.0
1   blue       0            0.5
2  green       0            0.0
3    red       1            1.0
4   blue       1            0.5
5  green       0            0.0
```

2.5.3 Функція взаємодії.

Функція взаємодії створюються шляхом поєднання двох або більше змінних для відображення взаємодії між ними. Ці функції можуть виявити взаємозв'язки, які не є очевидними при розгляді змінних окремо.

Приклад:

Припустимо, що ви маєте такі дані, як вік і дохід користувачів. Термін взаємодії можна створити шляхом множення цих двох властивостей, щоб відобразити, як дохід змінюється з віком.

В наступному прикладі ми створюємо функцію взаємодії, яка включає в себе параметри вік та дохід.

```
data = {
    'age': [25, 35, 45, 55],
    'income': [30000, 50000, 70000, 90000]
}
df = pd.DataFrame(data)

df['age_income_interaction'] = df['age'] * df['income']

df
```

Результат:

```
   age   income   age_income_interaction
0  25 30000 750000
1  35 50000 1750000
2  45 70000 3150000
3  55 90000 4950000
```

2.5.4 Поліноміальні ознаки.

Створюються шляхом генерації нових ознак, які є поліноміальними комбінаціями існуючих ознак. Це може бути особливо корисно при моделюванні нелінійних взаємозв'язків.

Приклад:

Припустимо, у вас є одна ознака x. Ви можете створити поліноміальні ознаки, такі як x^2, x^3 тощо, які можуть допомогти у виявленні нелінійних взаємозв'язків у даних.

```
from sklearn.preprocessing import PolynomialFeatures

data = {'x': [2, 3, 4]}
df = pd.DataFrame(data)

poly = PolynomialFeatures(degree=3, include_bias=False)
poly_features = poly.fit_transform(df)

df_poly = pd.DataFrame(poly_features, columns=poly.get_feature_names_out(['x']))

df_poly
```

Результат:

```
   x  x^2   x^3
0  2.0   4.0   8.0
1  3.0   9.0   27.0
2  4.0   16.0  64.0
```

☞ Після завершення обробки необхідно переконатися, що дані не містять пропущених значень, дублікатів, некоректних типів даних та аномальних викидів, а всі ознаки мають очікуваний формат і масштаб. Для цього використовують статистичний аналіз, перевірку структури даних, візуалізацію та засоби бібліотеки pandas, такі як `info()`, `describe()`, `isnull()` і `duplicated()`.

☞ Якісно підготовлені дані дозволяють моделям машинного навчання точніше виявляти закономірності, швидше навчатися, бути менш чутливими до шуму та краще узагальнювати знання на нових даних, що безпосередньо підвищує точність, стійкість і надійність прогнозування. Особливо важливо уникати витоку інформації під час підготовки даних. Параметри імпутації, масштабування та інших перетворень потрібно визначати на навчальній вибірці, а потім застосовувати до тестових даних. Це забезпечує чесну оцінку реальної здатності моделі працювати з новими спостереженнями.

✎ **Питання для самоконтролю.**

1. Що таке обробка даних (Data Processing) та яке її значення в процесі інтелектуального аналізу даних?
2. Які основні етапи включає процес обробки даних?
3. Чому якість обробки даних впливає на результати машинного навчання?
4. Які основні проблеми можуть бути присутні у «сирих» даних?
5. Що таке бібліотека pandas та для чого вона використовується?
6. Які основні структури даних підтримує бібліотека pandas?
7. Що таке DataFrame?
8. Чим DataFrame відрізняється від Series?
9. Як створити DataFrame у Python?
10. Як зчитати дані у DataFrame з CSV-файлу?

11. Як переглянути перші та останні записи DataFrame?
12. Як отримати інформацію про структуру DataFrame?
13. Як отримати статистичний опис числових даних за допомогою pandas?
14. Як вибрати окремий стовпець або кілька стовпців DataFrame?
15. Яка різниця між методами ‘loc[]‘ та ‘iloc[]‘ ?
16. Як відфільтрувати рядки DataFrame за певною умовою?
17. Як додати новий стовпець до DataFrame?
18. Як видалити стовпець або рядок із DataFrame?
19. Як перейменувати стовпці DataFrame?
20. Як відсортувати дані за одним або кількома стовпцями?
21. Що таке очищення даних (Data Cleaning)?
22. Які основні причини виникнення неякісних даних?
23. Що таке пропущені значення (Missing Values)?
24. Які причини появи пропущених значень у наборах даних?
25. Як виявити пропущені значення у DataFrame?
26. Які методи використовуються для видалення пропущених значень?
27. Які методи використовуються для заповнення пропущених значень?
28. У яких випадках доцільно видаляти рядки з пропущеними значеннями?
29. Що таке імпутація даних (Data Imputation)?
30. Які статистичні методи використовуються для заповнення пропущених значень?
31. Що таке дублікати даних?
32. Як знайти дублікати у DataFrame?
33. Як видалити дублікати за допомогою pandas?
34. Що таке викиди (Outliers)?
35. Якими методами можна виявляти викиди?
36. Чому викиди можуть негативно впливати на результати аналізу даних?
37. Що таке масштабування даних (Data Scaling)?
38. Для чого виконується масштабування ознак перед навчанням моделей?
39. У чому полягає різниця між масштабуванням та нормалізацією даних?
40. Що таке мінімально-максимальна нормалізація?
41. Як обчислюється мінімально-максимальна нормалізація?
42. Що таке стандартизація (Standardization)?

43. Як обчислюється стандартизація за Z-оцінкою (Z-score)?
44. Для яких алгоритмів машинного навчання масштабування є особливо важливим?
45. Які переваги має нормалізація даних?
46. Які недоліки має мінімально-максимальна нормалізація?
47. Які переваги має стандартизація порівняно з мінімально-максимальною нормалізацією?
48. Які інструменти бібліотеки scikit-learn використовуються для масштабування та нормалізації даних?
49. Як перевірити, що дані були успішно очищені та підготовлені до аналізу?
50. Яким чином правильна обробка даних впливає на точність, стійкість і узагальнювальну здатність моделей машинного навчання?

Розділ 3

Візуалізація даних.

Людський мозок значно краще сприймає візуальні образи, ніж набори сухих чисел. За лічені секунди можна помітити тенденцію на графіку, яку не можна було б побачити в таблиці в екселі чи в базі даних. Саме тому надзвичайно важливим етапом аналізу даних є візуалізація.

Важливо підкреслити різницю між двома дуже різними поняттями, які люди описують, використовуючи загальний термін "візуалізація". Перше з них — "візуалізація для дослідження даних": використання візуалізації для розуміння (тобто для власного розуміння) сукупності даних з метою подальшого аналізу. Коротко кажучи, мета цього типу візуалізації — "знайти істину"в даних. Альтернативою є "візуалізація для презентації": виокремлення певного аспекту даних у вигляді зрозумілої діаграми або графіку, що передає широкій аудиторії конкретний аспект даних, який ви хочете донести. Цей тип візуалізації має на меті "переконати інших людей у правдивості ваших висновків".

Візуалізація VS статистика: Візуалізація майже завжди надає більш інформативний (хоча і менш кількісний) огляд ваших даних, ніж статистика (підхід, а не галузь науки в цілому).

3.1 Типи даних.

Перед тим, як приступити до візуалізації, варто зрозуміти, з яким типом даних ви маєте справу. Це потрібно для того, щоб, у першу чергу, не помилитися із вибором методу візуалізації.

Чотири основні типи даних:

- *Номінальні дані (Nominal data)*: категоріальні дані без внутрішнього порядку між категоріями. Наприклад, змінна "тип домашньої тварини"може складатися з класів собака, кіт, кролик, і між цими двома типами немає відносного порядку, це просто різні дискретні значення.
- *Порядкові дані (Ordinal data)*: категоріальні дані з внутрішньою послідовністю, але "відмінності"між категоріями не мають суто числового значення. Канонічним прикладом тут є відповіді на опитування з такими відповідями: категорично не згоден, трохи не згоден, нейтральний, трохи згоден, категорично згоден. Важливою особливістю тут є те, що, хоча між цими типами існує чітка послідовність, немає сенсу вважати, що різниця між "трохи згоден"і "категорично згоден"є "такою ж як різниця між "нейтральний"і "трохи згоден".
- *Інтервальні дані (Interval data)*: числові дані, тобто дані, які можна відобразити на "числовій прямій"; важливим аспектом на відміну від порядкових даних є не "дискретна проти безперервної диференціації"(наприклад, цілочисельні значення можна вважати інтервальними даними), а той факт, що відносні відмінності в інтервальних даних мають значення. Класичним прикладом є температура (у Фаренгейті або Цельсії, що ми підкреслимо трохи пізніше): тут різниці між температурами мають значення: 10 і 15 градусів розділені такою

ж величиною, як 15 і 20 (ця властивість настільки властива числовим даним, що майже дивно її підкреслювати). З іншого боку, інтервальні дані охоплюють випадки, коли нульова точка "не має реального значення"; на практиці це означає, що співвідношення між двома точками даних не має значення. Двадцять градусів за Фаренгейтом не є "вдвічі гарячішими"в будь-якому значущому сенсі, ніж 10 градусів; і, звичайно, не нескінченно гарячішими, ніж нуль градусів.

- *Дані про співвідношення (Ratio data)*: також числові дані, але співвідношення між вимірами має певне значення. Класичним прикладом тут є температура за Кельвіном. Очевидно, що, як і температура за Фаренгейтом або Цельсієм, це описує основне явище температури, але на відміну від попередніх випадків, нуль за Кельвіном має значення з точки зору молекулярної енергії в речовині (тобто її відсутність). Це означає, що співвідношення мають реальне значення: речовина при 20 градусах Кельвіна має вдвічі більше кінетичної енергії на молекулярному рівні, ніж та сама речовина при 10 градусах Кельвіна.

✎ "Допустимі операції які ми можемо виконувати над двома елементами різних типів:

Номінальні дані: $=,\neq$. Все, що ми можемо зробити з номінальними даними, це порівняти два дані та перевірити, чи вони рівні.

Порядкові дані: $=,\neq,<,>$. Окрім перевірки на рівність, ми також можемо порівнювати порядок різних точок даних.

Інтервальні дані: $=,\neq,<,>,-$. Ми маємо всі операції з порядковими даними, але також можемо обчислювати точні числові різниці між двома точками даних. Залежно від контексту, ми можемо сказати, що додавання також дозволено, але іноді додавання фактично передбачає нульову точку і тому застосовується тільки до даних про співвідношення.

Дані співвідношення: $=,\neq,<,-,+,\div$. Ми маємо всі операції з інтервальними даними, але додавання тепер практично завжди дозволено, і ми можемо додатково виконувати ділення (для визначення співвідношень) між різними точками даних.

Щоб бути більш формальними, далі ми припускаємо, що маємо доступ до набору даних, який можна позначити як

$\{x^{(1)}, x^{(2)}, \ldots, x^{(m)}\}$

де кожне $x^{(i)}$ є деякою точкою даних. $x^{(i)} = (a_1^i, a_2^i, a_3^i, ...)$

Для побудови всіх цих графіків ми будемо використовувати бібліотеку matplotlib, яка добре інтегрується з Jupyter notebook.

3.2 1D дані

3.2.1 Стовпчасті діаграми — категоріальні дані

Якщо ваші дані є одновимірними категоріальними, єдиним реальним варіантом для їх візуалізації є використання стовпчастої діаграми (Bar charts). Зверніть увагу, що, виходячи з нашого припущення про те, що порядок точок даних у нашому наборі даних не має значення, єдиною значущою інформацією, якщо ми маємо набір даних типу

$\{student, teacher, student, professor, teacher, student, \ldots\}$

буде кількість повторень кожного елемента. Таким чином, ми можемо ефективно узагальнити дані, включивши кількість повторень кожного значення даних (це легко зробити за допомогою класу ‘collections.Counter‘).

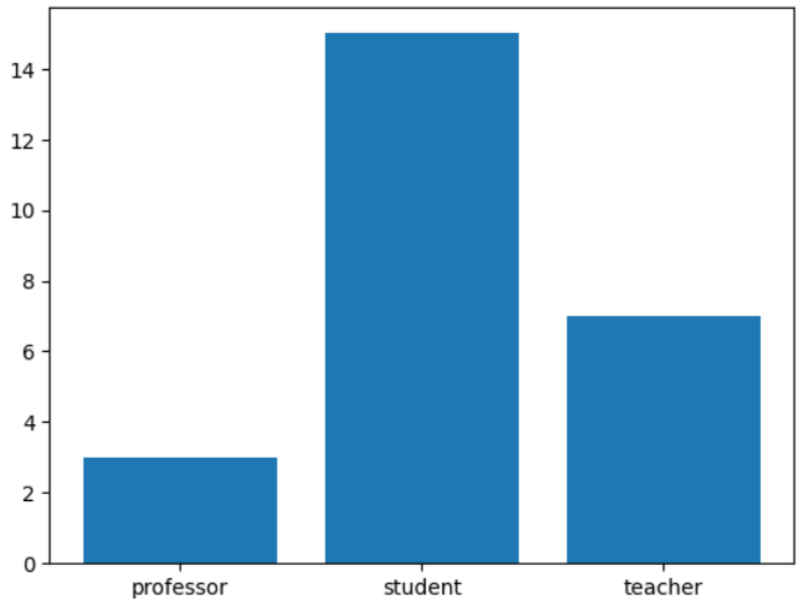
Примітка: незважаючи на те, що ви, можливо, бачили на деяких рисунках, зверніть увагу, що включення ліній між різними стовпчиками однозначно не має сенсу.

```
import collections
import matplotlib.pyplot as plt
```

```
import numpy as np

data = np.random.permutation(np.array(["student"]*15 + ["teacher"]*7 +
    ["professor"]*3))
# print(data)
counts = collections.Counter(data)
plt.bar(range(len(counts)), list(counts.values()), tick_label=list(counts.keys()))
```

Результат:



3.2.2 Кругові діаграми.

Кругові діаграми (Pie charts) призначені для представлення даних, а не для їх дослідження!

Люди не дуже добре оцінюють площу або кути (наприклад, важко визначити, чи один сегмент становить 23 %, а інший — 27 %). Стовпчасті діаграми набагато простіші, оскільки люди точніше порівнюють довжини.

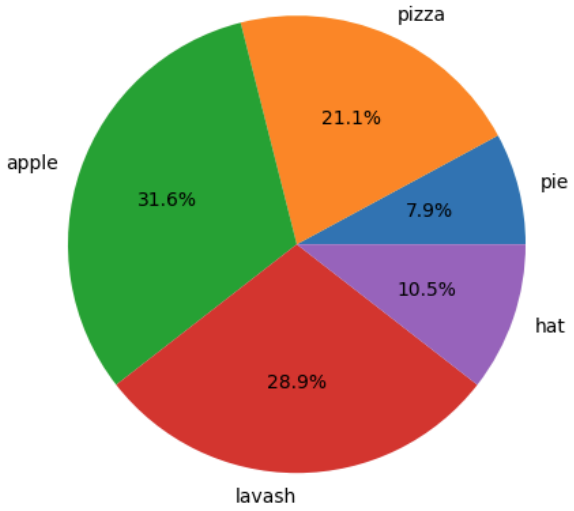
Порівняти дві різні кругові діаграми (наприклад, два періоди часу) майже неможливо. Неможливо легко додати смуги похибки, опорні лінії або точні порівняння.

Якщо ж ви, з якоїсь причини, вирішили побудувати кругову діаграму (наприклад, щоб прпродемонструвати одемонструвтаи клієнту), ось приклад того, як це можна зробити:

```
data = {"pie": 3,
        "pizza": 8,
        "apple": 12,
        "lavash": 11,
        "hat": 4}

plt.pie(data.values(), labels=data.keys(), autopct='%1.1f%%')
plt.axis('equal')
```

Результат:



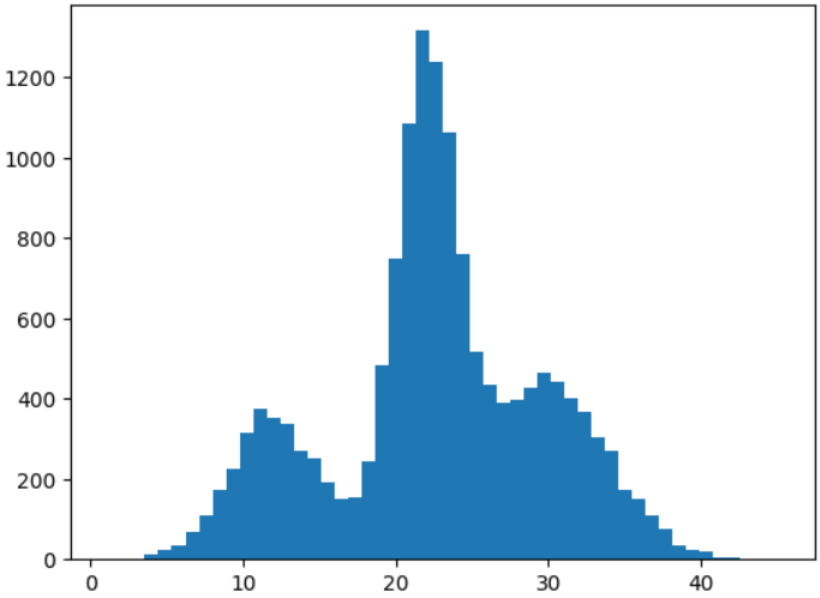
3.2.3 Гістограми — числові дані

Гістограма (Histogram) — це графік, який відображає розподіл числових даних, показуючи кількість спостережень у кожному інтервалі (біні).

```
np.random.seed(0)
data = np.concatenate([30 + 4*np.random.randn(5000),
                       22 + 2*np.random.randn(7000),
                       12 + 3*np.random.randn(3000)])
plt.hist(data, bins=50)


# data = np.concatenate([10 + 5*np.random.randn(10000)])
# plt.hist(data, bins=50);
```

Результат:



Гістограма показує розподіл числових даних, дозволяючи оцінити частоту появи значень у

різних інтервалах. Бін (Bin) — це інтервал значень, у межах якого підраховується кількість спостережень. Занадто мала кількість інтервалів приховує деталі розподілу, тоді як занадто велика може зробити гістограму шумною та ускладнити її аналіз.

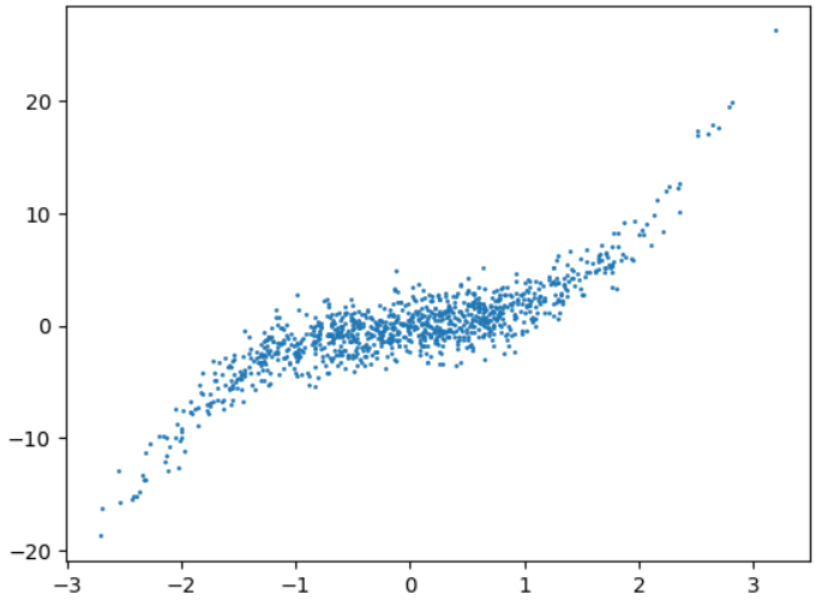
3.3 2D дані

3.3.1 Точкові діаграми — числові x числові.

Якщо обидва виміри даних є числовими, найприроднішим першим типом діаграми, який слід розглянути, є точкова діаграма (scatter plot): нанесення точок, які просто відповідають різним координатам даних. (Діаграма розсіювання)

```
x = np.random.randn(1000)
y = 0.8*x**3 + x + 1.5*np.random.randn(1000)
plt.scatter(x,y,s=1)
```

Результат:



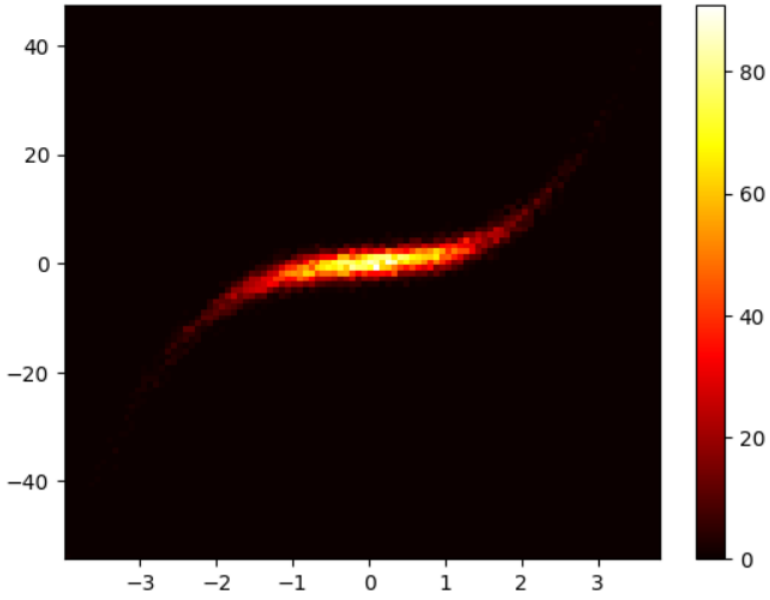
Однак існує також природний режим відмови, коли точок занадто багато, щоб їх можна було чітко розділити, і графік втрачає здатність показувати щільність даних. Наприклад, якщо у нас в 10 разів більше точок, графік вже не так чітко показує щільність у внутрішній частині "щільності".

У цьому випадку надлишку даних ми також можемо створити 2D гістограму даних (яка групує дані за обома вимірами) і вказати "висоту"кожного блоку за допомогою кольорової карти. Такі графіки можуть чіткіше вказувати щільність точок у регіонах, які в оригінальному графіку розсіювання мають суцільний колір. Ці 2D гістограми іноді також називають *тепловими картами* (heat maps), але ця назва часто конфліктує з подібними версіями, що використовуються для побудови 2D категоріальних даних, тому тут ми будемо використовувати термін *2D гістограма*.

```
x = np.random.randn(10000)
y = 0.8*x**3 + x + 1.5*np.random.randn(10000)
#plt.scatter(x,y,s=10)
```

```
plt.hist2d(x,y,bins=100);
plt.colorbar();
plt.set_cmap('hot')
```

Результат:



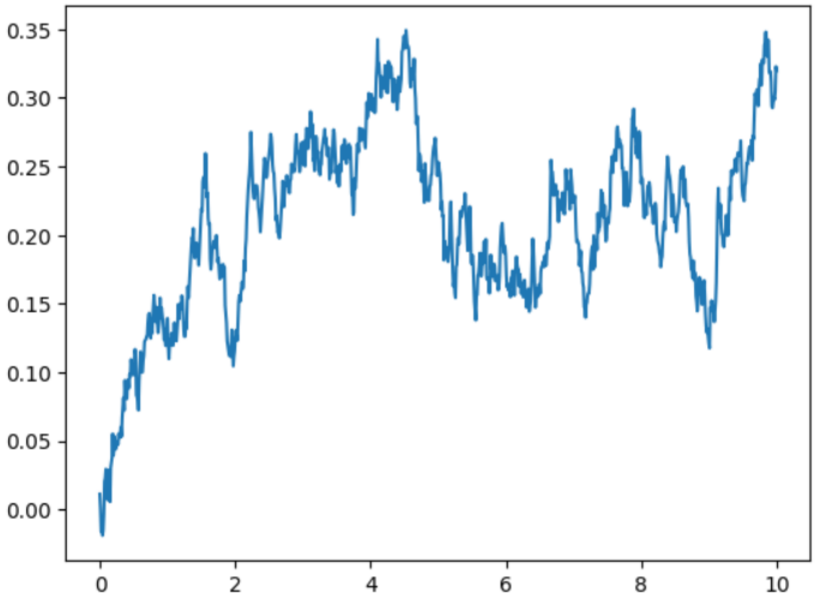
3.3.2 Лінійна діаграма - числові x числові.

Відомий всім зі школи і зрозумілий приклад візуалізації двовимірних числових даних, це лінійна діаграма (або графік, line plot). Це тип візуалізації, який відображає дані у вигляді точок, з'єднаних відрізками прямих.

```
x = np.linspace(0,10,1000)
y = np.cumsum(0.01*np.random.randn(1000))
plt.plot(x,y)
```

Результат:

```
[<matplotlib.lines.Line2D at 0x1957fbbcb50>]
```



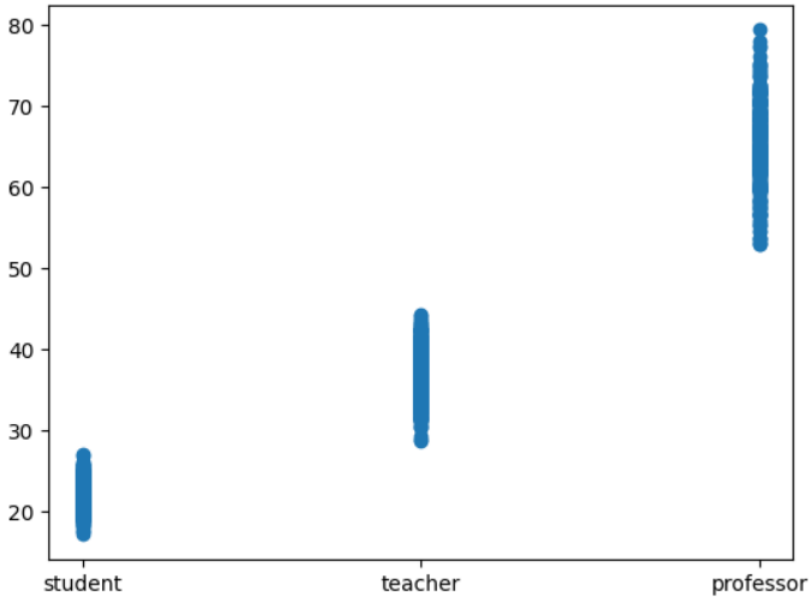
3.3.3 Коробкова і скрипкові діаграми — категоріальні х числові

Розглянемо простий приклад, де (тільки з вигаданими даними) ми будуємо графік учасників навчального процесу та їхній вік.

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.linalg as la

data= {"student": 22 + 1.4*np.random.randn(1000),
       "teacher": 37 + 2.5*np.random.randn(900),
       "professor": 65 + 5.5*np.random.randn(300)}
plt.scatter(np.concatenate([i*np.ones(len(x)) for i,x in
    enumerate(data.values())]),
            np.concatenate(list(data.values())))
plt.xticks(range(len(data)), data.keys());
```

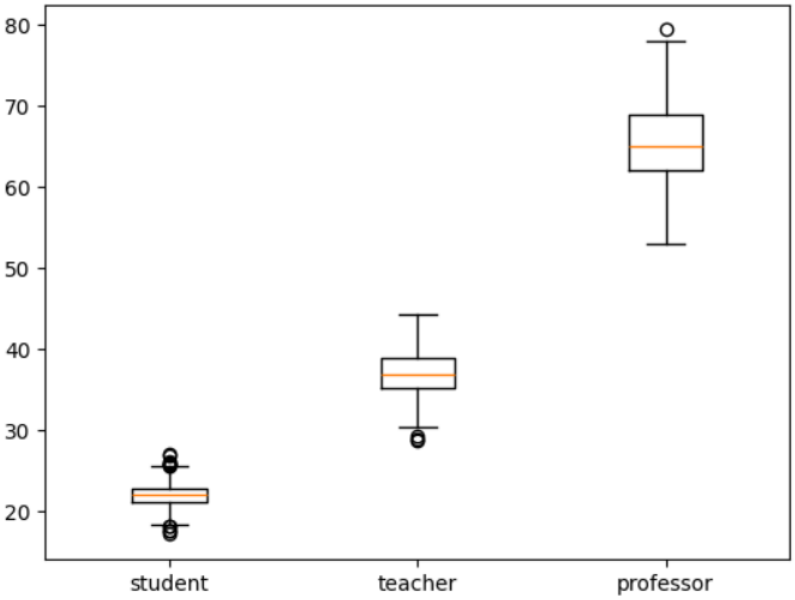
Результат:



Очевидно, що, дивлячись тільки на цей графік, можна отримати дуже мало, оскільки в щільній лінії точок недостатньо інформації, щоб дійсно зрозуміти розподіл числової змінної для кожної точки. Поширеною стратегією тут є використання графіка коробкова, або коробка з вусами (box and whiskers), яка відображає медіану даних (як лінію в середині коробки), 25-й і 75-й процентилі даних (як нижня і верхня межі коробки), "вуса"встановлюються за допомогою ряду різних можливих угод (за замовчуванням Matplotlib використовує 1,5-кратний інтерквартильний розмах, відстань між 25-м і 75-м процентилем), а будь-які точки поза цим діапазоном ("випадкові значення або "викиди") відображаються окремо.

```
plt.boxplot(data.values())
plt.xticks(range(1,len(data)+1), data.keys())
```

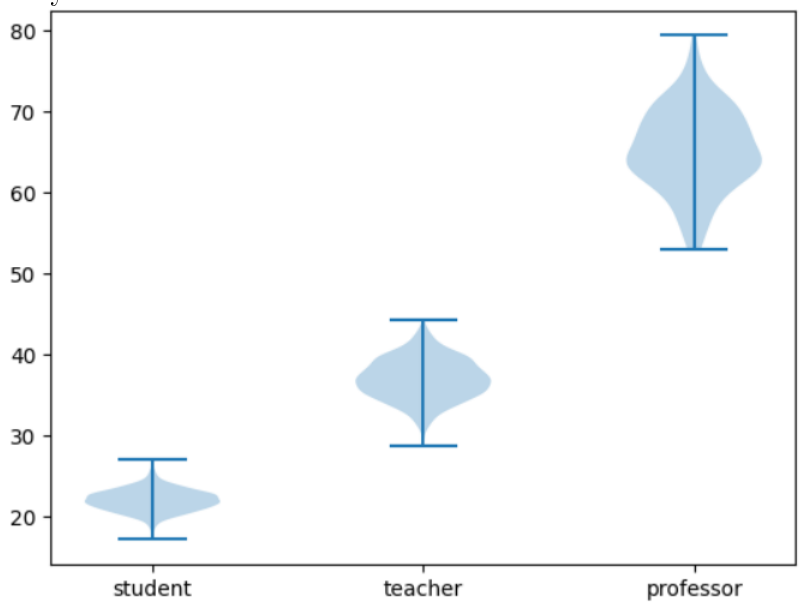
Результат:



☞ Звичайно, так само як середні значення та стандартні відхилення не описують повністю набір даних, статистика бокс-плат і вусів не відображає повністю розподіл даних. З цієї причини також часто використовуються скрипкові діаграми (violin plots), які створюють міні-гістограми.

```
plt.violinplot(data.values())
plt.xticks(range(1,len(data)+1), data.keys())
```

Результат:



3.3.4 Теплова карта та бульбашкові діаграми — категоріальні x категоріальні

Теплова карта (Heatmap) — це спосіб візуалізації даних, у якому значення відображаються за допомогою кольорів, що дозволяє швидко оцінити їхню величину та закономірності. За допомогою теплових карт найчастіше візуалізують матриці кореляцій, статистичні таблиці, матриці відстаней, частоти спостережень та інші двовимірні числові дані. Теплова карта кореляцій використовується для наочного відображення сили та напряму кореляції між числовими ознаками, що допомагає виявити сильно пов'язані або незалежні змінні.

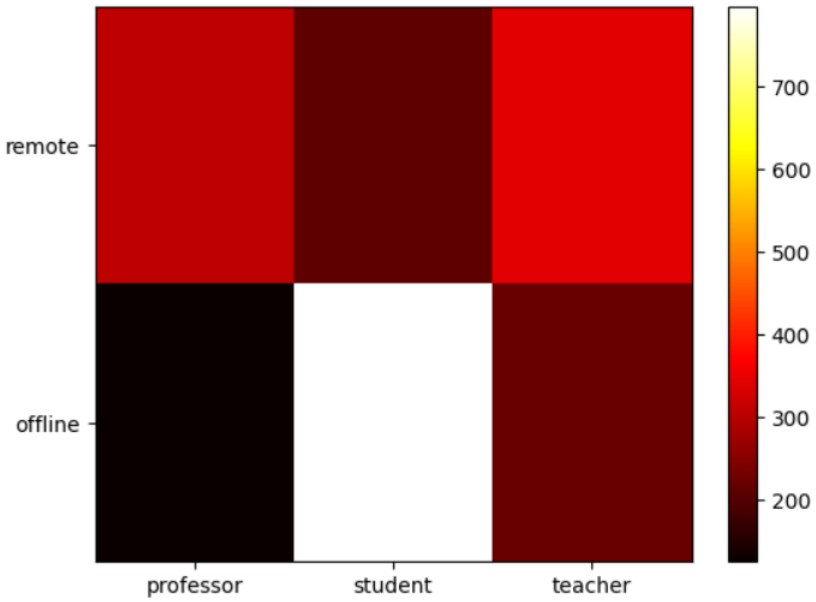
Коли обидва виміри наших 2D-даних є категоріальними, ми маємо ще менше інформації для використання. Знову ж таки, метою буде надання певної інформації про загальну кількість усіх можливих комбінацій між двома наборами даних. Наприклад, розглянемо вигаданий набір даних про учасників навчального процесу та їхній улюблений спосіб навчання.

```
types = np.array([('student', 'offline'), ('student', 'remote'),
                  ('teacher', 'offline'), ('teacher', 'remote'),
                  ('professor', 'offline'), ('professor', 'remote')])
data = types[np.random.choice(range(6), 2000, p=[0.4, 0.1, 0.12, 0.18, 0.05,
    0.15]),:]

# print(data[100:])

label_x, x = np.unique(data[:,0], return_inverse=True)
label_y, y = np.unique(data[:,1], return_inverse=True)
M, xt, yt, _ = plt.hist2d(x,y, bins=(len(label_x), len(label_y)))
plt.xticks((xt[:-1]+xt[1:])/2, label_x)
plt.yticks((yt[:-1]+yt[1:])/2, label_y)
plt.colorbar()
```

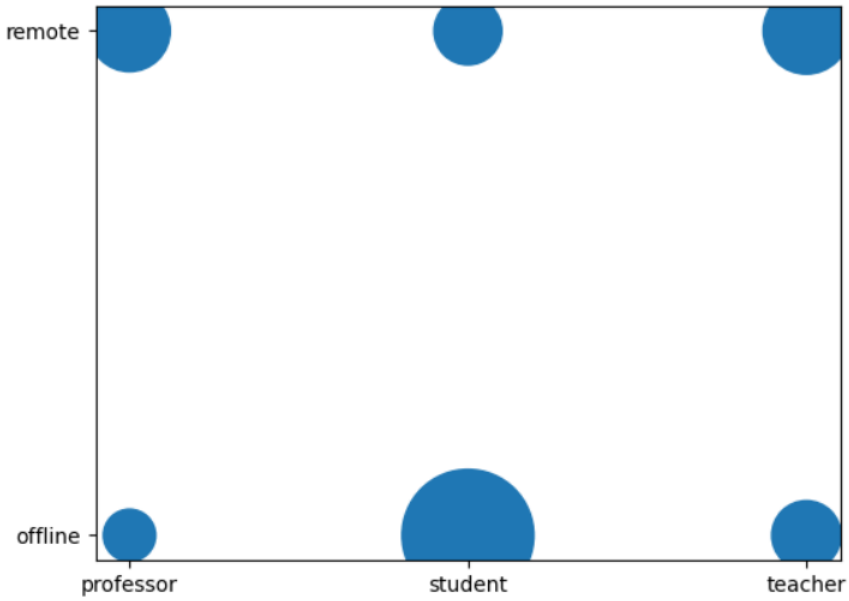
Результат:



Хоча це може бути дещо корисним, діапазон кольорів, безперечно, не є дуже інформативним у деяких налаштуваннях, тому більш доречним може бути діаграма розсіювання з розмірами, пов'язаними з кожним типом даних (це також називається бульбашковою діаграмою).

```
xy, cnts = np.unique((x,y), axis=1, return_counts=True)
plt.scatter(xy[0], xy[1], s=cnts*5)
plt.xticks(range(len(label_x)), label_x)
plt.yticks(range(len(label_y)), label_y)
```

Результат:



Бульбашкова діаграма (bubble plot) додатково використовує розмір маркерів для відображення ще однієї числової змінної. Розмір бульбашок може відображати величину показника, наприклад населення, прибуток, обсяг продажів або іншу числову характеристику.

3.4 3D дані

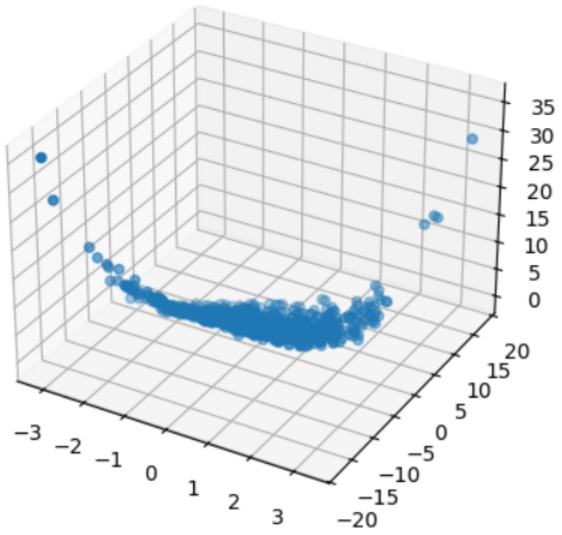
Як тільки ми виходимо за межі двох вимірів, ефективна візуалізація стає набагато складнішою.

3D точкові графіки (3D scatter plots), зображення точок в 3D, слід уникати, якщо, звичайно, не можна взаємодіяти з 3D-графіком:

```
x = np.random.randn(1000)
y = 0.4*x**3 + x + 1.7*np.random.randn(1000)
z = 0.5 + 0.1*(y-1)**2 + 0.4*np.random.randn(1000)

from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(x,y,z)
```

Результат:



3.4.1 Матриці розсіювання (точкові матриці).

Універсальним засобом для візуалізації високорозмірних даних є матриця розсіювання (scatter matrices).

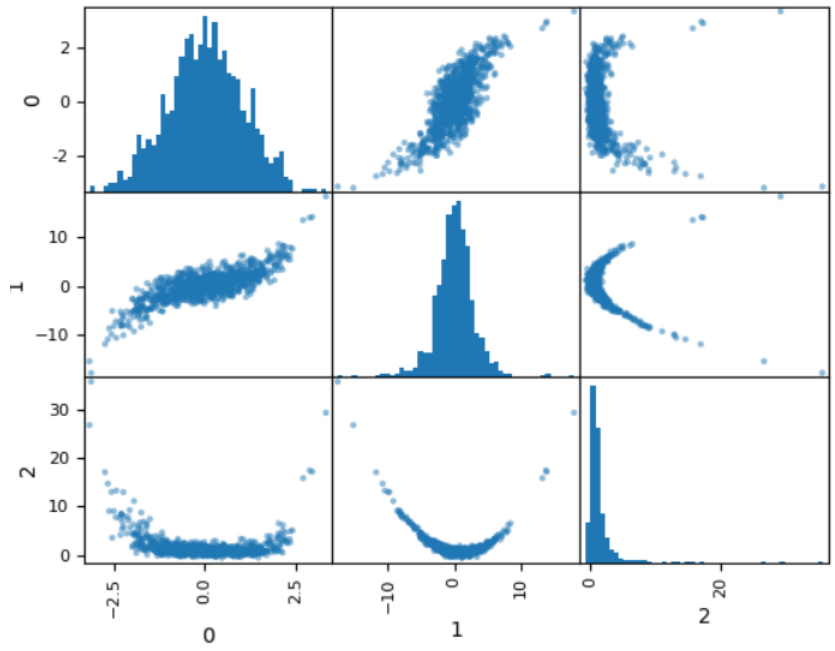
По-суті, це сітка графіків, яка дозволяє візуально оцінити взаємозв'язок між усіма парами змінних у наборі даних. Тобто, для кожного набору двох із трьох аргументів, ми будуємо графік, враховуючи лише відповідні два агрументи, та ігноруючи третій.

☞ Щодо матриць розсіювання слід зазначити один важливий момент: не намагайтеся використовувати їх для представлення даних.

```
import pandas as pd
df = pd.DataFrame([x,y,z]).transpose()
pd.plotting.scatter_matrix(df, hist_kwds={'bins':50})
```

Результат:

```
array([[<Axes: xlabel='0', ylabel='0'>, <Axes: xlabel='1', ylabel='0'>,
        <Axes: xlabel='2', ylabel='0'>],
       [<Axes: xlabel='0', ylabel='1'>, <Axes: xlabel='1', ylabel='1'>,
        <Axes: xlabel='2', ylabel='1'>],
       [<Axes: xlabel='0', ylabel='2'>, <Axes: xlabel='1', ylabel='2'>,
        <Axes: xlabel='2', ylabel='2'>]], dtype=object)
```



Матриця розсіювання використовується для одночасного аналізу взаємозв'язків між кількома числовими ознаками. За допомогою матриці розсіювання можна виявити кореляції, кластери, викиди (outliers) та можливі нелінійні залежності між ознаками.

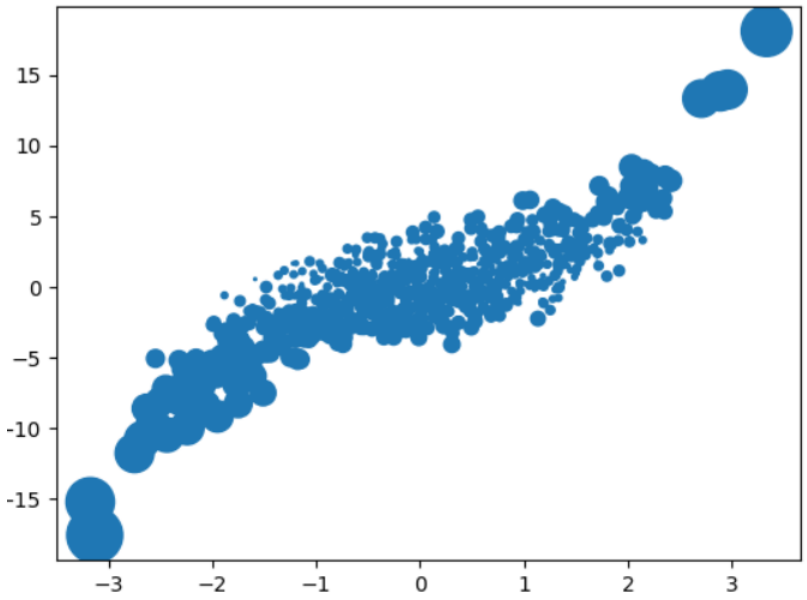
3.4.2 Бульбашкові діаграми.

Бульбашкова діаграма (bubble plots) з попереднього розділу також може бути одним із засобів розуміння тривимірних даних. У цій діаграмі та її варіантах ми побудуємо двовимірну точкову діаграму, але додамо третій вимір за допомогою іншого атрибуту, наприклад розміру точок. Для наведених вище даних це буде виглядати наступним чином.

```
plt.scatter(x,y,s=z*20)
```

Результат:

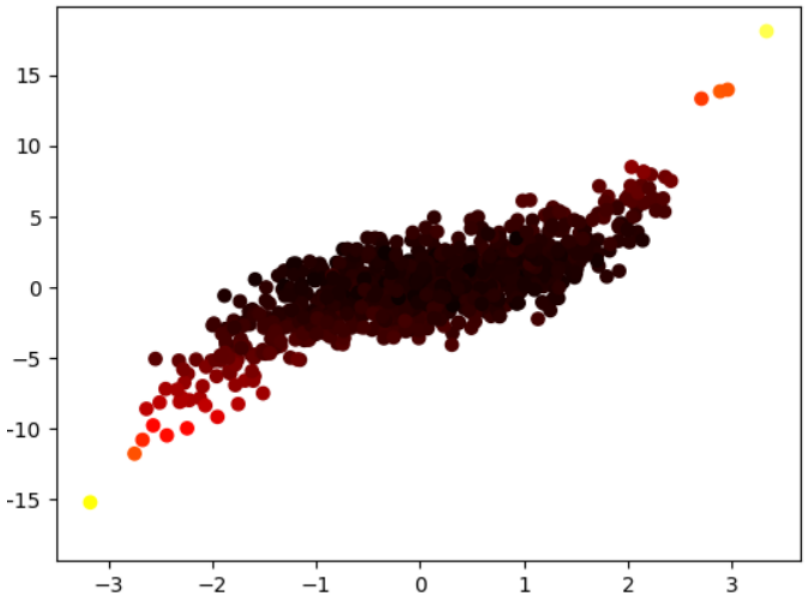
```
<matplotlib.collections.PathCollection at 0x2bfbdcfcd10>
```




```
plt.scatter(x,y,c=z)
```

Результат:

```
<matplotlib.collections.PathCollection at 0x2bfbb610650>
```



Тип діаграми обирають залежно від типу даних, кількості змінних, мети аналізу та закономірностей, які необхідно виявити. Правильно обрана візуалізація робить дані зрозумілішими, дозволяє швидко знаходити закономірності, виявляти аномалії та оцінювати взаємозв'язки між змінними.

✎ **Питання для самоконтролю.**

1. Що таке візуалізація даних і яке її значення в інтелектуальному аналізі даних?
2. Які основні цілі візуалізації даних?
3. Чому візуалізація є важливим етапом дослідницького аналізу даних (EDA)?

4. Які переваги має графічне представлення даних порівняно з табличним?
5. Які бібліотеки Python найчастіше використовуються для побудови графіків?
6. Які типи даних виділяють за шкалою вимірювання?
7. Що таке номінальні дані?
8. Наведіть приклади номінальних даних.
9. Що таке порядкові дані?
10. Наведіть приклади порядкових даних.
11. Що таке інтервальні дані?
12. Чим інтервальні дані відрізняються від даних співвідношення?
13. Що таке дані співвідношення (Ratio Data)?
14. Який тип діаграми найкраще підходить для номінальних даних?
15. Для яких типів даних доцільно використовувати гістограми?
16. Що таке стовпчаста діаграма (Bar Chart)?
17. Для яких задач використовується стовпчаста діаграма?
18. Чим стовпчаста діаграма відрізняється від гістограми?
19. Що таке кругова діаграма (Pie Chart)?
20. У яких випадках доцільно використовувати кругову діаграму?
21. Які недоліки має кругова діаграма?
22. Що таке гістограма?
23. Яку інформацію дозволяє отримати гістограма?
24. Що таке бін (Bin) у гістограмі?
25. Як вибір кількості інтервалів впливає на вигляд гістограми?
26. Що таке точкова діаграма (Scatter Plot)?
27. Для яких задач використовують точкові діаграми?
28. Як за допомогою точкової діаграми можна виявити кореляцію між змінними?
29. Що таке лінійна діаграма (Line Chart)?
30. Для яких типів даних найчастіше використовують лінійні діаграми?
31. Що таке коробкова діаграма (Box Plot)?
32. Які статистичні характеристики відображає коробкова діаграма?
33. Як за допомогою Box Plot можна виявити викиди?
34. Що таке скрипкова діаграма (Violin Plot)?
35. Чим скрипкова діаграма відрізняється від коробкової?

36. Що таке теплова карта (Heatmap)?
37. Які дані найчастіше візуалізують за допомогою теплової карти?
38. Для чого використовується теплова карта кореляцій?
39. Що таке бульбашкова діаграма (Bubble Chart)?
40. Чим бульбашкова діаграма відрізняється від звичайної точкової діаграми?
41. Яку інформацію може відображати розмір бульбашок?
42. Що таке матриця розсіювання (Scatter Matrix)?
43. Для чого використовується матриця розсіювання під час аналізу даних?
44. Які закономірності можна виявити за допомогою матриці розсіювання?
45. Що таке тривимірна (3D) візуалізація даних?
46. Які типи 3D-графіків підтримуються сучасними бібліотеками Python?
47. Які переваги має використання 3D-візуалізації?
48. Які недоліки та обмеження має 3D-візуалізація?
49. За якими критеріями слід обирати тип діаграми для конкретного набору даних?
50. Яким чином правильний вибір способу візуалізації допомагає виявляти закономірності, аномалії та взаємозв'язки у даних?

Розділ 4

Лінійна алгебра та бібліотека NumPy.

Лінійна алгебра — це розділ математики, що вивчає вектори, матриці та операції над ними. Вона є математичною основою більшості алгоритмів машинного навчання, аналізу даних, комп'ютерного зору та нейронних мереж. Методи лінійної алгебри використовуються для представлення даних у вигляді матриць, зменшення розмірності даних, побудови моделей машинного навчання, кластеризації, регресії, обробки зображень та аналізу текстів.

Вектори та матриці відіграють центральну роль в сучасному інтелектуальному аналізі даних. Матриці, це очевидний спосіб зберігання табличних даних. Сьогодні основа лінійної алгебри, є мовою всіх алгоритмів аналізу даних (машинне навчання, штучний інтелект та ін.).

4.1 Основи лінійної алгебри.

Скаляр - окреме числове значення, яке характеризується лише величиною і не має напрямку.
Вектор - 1D масив даних.

$$\mathbf{v} = \begin{bmatrix} x_1 \\ x_2 \\ ... \\ x_n \end{bmatrix}$$

Матриця - 2D масив даних.

$$\mathbf{A} = \begin{bmatrix} x_{11} & x_{12} & ... & x_{1m} \\ x_{21} & x_{22} & ... & x_{2m} \\ ... & ... & ... & ... \\ x_{n1} & x_{n2} & ... & x_{nm} \end{bmatrix}$$

Існують також узагальнення матриць вищого порядку (так звані *тензори*), які представляють тривимірні або вищі масиви значень. Вони досить широко використовуються в сучасній науці про дані, хоча зазвичай (але, звичайно, не завжди) тензори використовуються лише в значенні «багатовимірного масиву», а не в їхньому справжньому лінійно-алгебраїчному значенні. Тензори як лінійні оператори, що діють, наприклад, на матриці або інші тензори вищого порядку, дещо рідше використовуються в базовій науці про дані.

4.1.1 Порядок рядків і стовпців.

Матриці можуть бути розміщені в пам'яті за рядками або за стовпцями. Для того, щоб писати ефективний код для роботи із матричними даними, варто цей розуміти порядок запису.

$$\mathbf{A} = \begin{bmatrix} 100 & 80 \\ 60 & 80 \\ 100 & 100 \end{bmatrix}$$

За рядками: 100, 80, 60, 80, 100, 100. За стовпцями: 100, 60, 100, 80, 80, 100.

☞ По замовчуванню в NumPy порядок за рядками.

4.1.2 Основні операції.

Додавання: Дві матриці A та B, $[n \times m]$, $A + B = C$ визначається: $C_{ij} = A_{ij} + B_{ij}$

Транспонування: Для матриці A, $[n \times m]$, транспонована матриця $C = A^T$ визначається: $C_{ij} = A_{ji}$

Множення: Для матриць A ($[n \times m]$) та B ($[m \times p]$) добуток $C = AB$ ($[n \times p]$) визначається: $C_{ij} = \sum_{k=1}^{m} A_{ik}B_{kj}$

Множення матриць є асоціативним ($(AB)C = A(BC)$), дистрибутивним $A(B+C) = AB+AC$, але не комутативним ($AB \neq BA$).

Одинична матриця: Одинична матриця I це квадратна матриця з одиницями на головній діагоналі та нулями в інших позиціях:

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & ... & 0 \\ 0 & 1 & ... & 0 \\ ... & ... & ... & ... \\ 0 & 0 & ... & 1 \end{bmatrix}$$

Для будь-якої матриці A: $AI = IA = A$

Обернена матриця: Для квадратної матриці A, обернена матриця A^{-1} це така матриця, що

$A^{-1}A = AA^{-1} = I$

Не для кожної квадратної матриці існує обернена.

Деякі рівняння: $(AB)^T = B^T A^T$, $(AB)^{-1} = B^{-1}A^{-1}$

Скалярний добуток векторів: $x * y = x^T y = \sum_{i=1}^{n} x_i y_i$;

Норма вектора: $||x||_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$

4.2 Масиви NumPy.

Створення масиву за допомогою команди `numpy.array` повертає тип `ndarray`. Ви також можете створювати масиви нулів, одиниць або випадкових чисел (у цьому випадку `np.randon.randn` створює матрицю зі стандартними випадковими нормальними елементами, а `np.random.rand` створює рівномірні випадкові елементи).

Спочатку імпортуйте модуль NumPy за допомогою наступної команди:

```
import numpy as np
```

Приклад створення вектора та масиву в NumPy:

```
b = np.array([-13,9])
A = np.array([[4,-5], [-2,3]])
print(b, "\n")
print(A, "\n")
```

Результат:

```
[-13   9]

[[ 4 -5]
 [-2  3]]
```

Приклад 1D масиву одиниць:

```
print(np.ones(40), "\n")
```

Результат:

```
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

1D масив нулів:

```
print(np.zeros(4), "\n")
```

Результат:

```
[0. 0. 0. 0.]
```

1D масив сипадкових чисел нормального розподілу $N(0, 1)$:

```
print(np.random.rand(40), "\n")
```

Результат:

```
[0.18507018 0.46626434 0.9801947  0.0827781  0.57378447 0.56871301
 0.84269999 0.6657426  0.56345543 0.93262208 0.98343751 0.30900493
 0.38287176 0.01312199 0.23539386 0.15844975 0.90519996 0.26924736
 0.85669799 0.94183472 0.011728   0.63333182 0.80378177 0.64861789
 0.02308254 0.69805843 0.52183114 0.61711567 0.95150433 0.2085504
 0.76690385 0.1997735  0.45904879 0.81617043 0.62357921 0.8821446
 0.56465777 0.09903507 0.66269635 0.09785717]
```

Приклад 2D масиву одиниць (3×4, 3 рядки 4 стовпчики):

```
print(np.ones((3,4)), "\n")
```

Результат:

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

2D масив нулів (3×4):

```
print(np.zeros((3,4)), "\n")
```

Результат:

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

2D масив випадкових чисел нормального розподілу:

```
print(np.random.randn(3,4))
```

Результат:

```
[[-0.72206547  0.26200423  1.45715136 -1.25955877]
 [-0.01199544  0.27812632  2.34386974  0.1253467 ]
 [ 1.60420367  1.11023166  1.34434453  0.87136904]]
```

2D масив випадкових чисел нормального розподілу із середнім значенням *mu* та дисперсією *sigma*:

```
sigma, mu = 10, 50
print(sigma * np.random.randn(5, 5) + mu)
```

Результат:

```
[[54.73972213 58.44438959 49.52225911 61.93038812 62.89356473]
 [58.20297378 54.47071892 53.323917   46.94830739 54.82418092]
 [61.0468657  30.48331679 57.33545183 84.93594736 55.92995469]
 [52.76434983 56.28182261 48.24665623 50.28029835 57.08573032]
 [44.30725611 48.51274777 40.88648182 36.13557207 55.81138676]]
```

Ви також можете створити матрицю ідентичності за допомогою команди `np.eye()`, а діагональну матрицю — за допомогою команди `np.diag()`.

Наступний приклад створює масив для одиничної матриці 3×3, та діагональний масив випадкових чисел нормального розподілу:

```
print(np.eye(3),"\n")
print(np.diag(np.random.randn(3)),"\n")
```

Результат:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

[[ 0.50651677  0.          0.        ]
 [ 0.         -0.35063699  0.        ]
 [ 0.          0.         -0.72949183]]
```

4.2.1 Індексація в масивах NumPy.

В NumPy можна вибрати окремий елемент масиву, знаючи його положення в масиві, через індексацію `A[i,j]`. Можна також вибрати цілий ряд `A[i,:]`, або зріз масив (slice) `A[start:end, :]`.

```
A = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
print(A, "\n")
print(A[1,1],"\n")
print(A[1,:],"\n")
print(A[1:3, :], "\n")
```

Результат:

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
```

```
 [10 11 12]]

5

[4 5 6]

[[4 5 6]
 [7 8 9]]
```

Можна також вибрати A[1,1] як одиничний 2D масив:

```
print(A[1:2,1:2], "\n")
```

Результат:

```
[[5]]
```

Можна також вибрати окремі рядки, наприклад 1, 2, та 3:

```
print(A[[1,2,3],:], "\n")
```

Результат:

```
[[ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

Або, для прикладу, 2, 1, та 2 знову:

```
print(A[[2,1,2],:], "\n")
```

Результат:

```
[[7 8 9]
 [4 5 6]
 [7 8 9]]
```

Можна також вибирати окремі рядки масиву, які нас цікавлять, вказуючи в окремому масиві через bool значення True або False, в залежності від того, чи ми хочемо вибрати відповідний рядок:

```
print(A[[False, True, False, True],:], "\n")
```

Результат:

```
[[ 4  5  6]
 [10 11 12]]
```

4.2.2 Основні операції над масивами в NumPy.

Масиви можна додавати/віднімати, множити/ділити та транспонувати, але ці операції не є ідентичними до відповідних операцій у лінійній алгебрі.

Множення та ділення масивів виконуються поелементно, вони не є множенням матриць або чимось, пов'язаним з оберненням матриць.

Для прикладу, визначимо два масиви:

```
A = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
B = np.array([[1, 1, 1], [1,2,1], [3, 1, 3], [1, 4, 1]])
```

Додати A та B поелементно:

```
print(A+B, "\n")
```

Результат:

```
[[ 2  3  4]
 [ 5  7  7]
 [10  9 12]
 [11 15 13]]
```

Відняти B від A поелементно:

```
print(A-B, "\n")
```

Результат:

```
[[ 0  1  2]
 [ 3  3  5]
 [ 4  7  6]
 [ 9  7 11]]
```

Поелементне множення (не множення матриць):

```
print(A*B, "\n")
```

Результат:

```
[[ 1  2  3]
 [ 4 10  6]
 [21  8 27]
 [10 44 12]]
```

Поелементне ділення:

```
print(A/B, "\n")
```

Результат:

```
[[ 1.         2.         3.        ]
 [ 4.         2.5        6.        ]
 [ 2.33333333 8.         3.        ]
 [10.         2.75      12.        ]]
```

Ви можете транспонувати масиви, але зверніть увагу, що це має значення тільки для 2D (або вищих) масивів. Транспонування 1D масиву нічого не дає, оскільки NumPy не розрізняє стовпцеві вектори та рядкові вектори для 1D масивів.

```
print(A, "\n")
print(A.T, "\n")
```

Результат:

```
[[ 1  2  3]
 [ 4  5  6]
```

```
 [ 7  8  9]
 [10 11 12]]

[[ 1  4  7 10]
 [ 2  5  8 11]
 [ 3  6  9 12]]
```

4.2.3 Трансляція NumPy.

Справжня цікавість починається, коли ви додаєте/віднімаєте/множите/ділите масиви різних розмірів. Замість того, щоб видавати помилку, NumPy спробує зрозуміти вашу операцію, використовуючи правила трансляції (broadcasting) NumPy. Це складна тема, яка часто бентежить новачків у NumPy, але з невеликою практикою правила стають досить інтуїтивними.

```
A = np.ones((4,3))        # A - 4x3
x = np.array([[1,2,3]])   # x - 1x3

print(A, '\n')
print(x, '\n')
```

Результат:

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]

[[1 2 3]]
```

Ми визначили масив 4×3 та вектор розмірності 3.

```
print(A*x)
```

Результат:

```
[[1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]]
```

Коли ми викликали перемноження масиву та вектора, NumPy автоматично повторив x 4 рази, та перемножив на масив.

4.2.4 Операції лінійної алгебри в NumPy.

Починаючи з Python 3, між масивами NumPy тепер існує оператор множення матриць `@` (раніше для цього доводилося використовувати більш громіздку функцію `np.dot()`).

```
A = np.random.randn(5,4)
C = np.random.randn(4,3)
x = np.random.randn(4)
y = np.random.randn(5)
z = np.random.randn(4)
```

Приклад множення матриці на матрицю (вертає 2D масив):

```
print(A @ C, "\n")
```

Результат:

```
[[-0.77983164  0.86915701  0.74016041]
 [ 0.9441811  -1.13337787 -2.45126146]
 [ 1.06065113 -3.17950771 -3.50084458]
 [ 1.26732128 -1.3041138  -2.5499486 ]
 [ 1.60891994 -4.68541868 -4.6433185 ]]
```

Приклад множення матриці на вектор (вертає 1D масив):

```
print(A @ x, "\n")
```

Результат:

```
[ 1.53587182 -2.9122003  -2.53401694 -2.88846645 -3.50775768]
```

Внутрішній добуток (скалярний) - зверніть увагу на тип даних, що вертаються - це скаляр, а не масив!

```
print(x @ z)
```

Результат:

```
2.236322572387176
```

☞ За замовчуванням оператор `@` застосовується зліва направо, що може призвести до дуже неефективних порядків множення матриць.

```
A = np.random.randn(1000,1000)
B = np.random.randn(1000,2000)
x = np.random.randn(2000)

%timeit A @ B @ x
```

Результат:

```
59.1 ms +/- 7.45 ms per loop (mean +/- std. dev. of 7 runs, 10 loops each)
```

Це виконує матричні добутки $(AB)x$, які спочатку обчислюють неефективне множення матриць. Якщо ми хочемо обчислити добуток у набагато ефективнішому (в даному прикладі) порядку $A(Bx)$, ми вказуємо порядок виконання операції множення матриць через дужки:

```
%timeit A @ (B @ x)
```

Результат:

```
2.13 ms +/- 226' ns per loop (mean +/- std. dev. of 7 runs, 100 loops each)
```

Нарешті, Numpy включає процедуру `np.linalg.inv()` для обчислення оберненої матриці A^{-1} та `np.linalg.solve()` для обчислення матричного рівняння $A^{-1}b$.

```
b = np.array([-13,9])
A = np.array([[4,-5], [-2,3]])
```

```
print(np.linalg.inv(A), "\n")
print(np.linalg.solve(A,b))
```

Результат:

```
[[1.5 2.5]
 [1.  2. ]]

[3. 5.]
```

4.2.5 Складність операцій.

Припустимо, що A, B $[n \times n]$, x, y - вектори [n].
Добуток матриць AB: $O(n^3)$
Добуток матриці та вектора Ax: $O(n^2)$
Внутрішній добуток векторів $x^T y$: $O(n)$
Обернена матриця/розв'язання: A^{-1}: $O(n^3)$

4.2.6 Розріджені матриці.

Багато матриць є розрідженими - такими, що містять переважно нульові елементи, з лише декількома ненульовими елементами.

Приклади: матриці, утворені реальними графами, матриці підрахунку слів у документах (більше про це пізніше)

Зберігання всіх цих нулів у стандартному форматі матриці може бути великою тратою обчислювальних ресурсів і пам'яті.

Бібліотеки розріджених матриць надають ефективні засоби для обробки цих розріджених матриць, зберігаючи і оперуючи тільки ненульовими елементами

Існує кілька різних способів зберігання розріджених матриць, кожен з яких оптимізований для різних операцій

Формат координат (COO): зберігання кожного елемента як кортежу ($inde_{row}$, $index_{column}$, $value$)

Наприклад, замість матриці:

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 3 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 4 & 0 & 1 & 0 \end{bmatrix}$$

Матимемо:

$$values = [2, 4, 1, 3, 1, 1]$$

$$indices_{row} = [1, 3, 2, 0, 3, 1]$$

$$indecies_{column} = [0, 0, 1, 2, 2, 3]$$

Приклад використання біблотеки `scipy` для роботи з розрідженими матрицями:

```
import scipy.sparse as sp

values = [2, 4, 1, 3, 1, 1]
row_indices = [1, 3, 2, 0, 3, 1]
column_indices = [0, 0, 1, 2, 2, 3]
A = sp.coo_matrix((values, (row_indices, column_indices)), shape=(4,4))
print(A.todense())
```

Результат:

```
[[0 0 3 0]
 [2 0 0 1]
 [0 1 0 0]
 [4 0 1 0]]
```

✎ **Питання для самоконтролю.**

1. Що таке лінійна алгебра і чому вона є важливою для інтелектуального аналізу даних?
2. Які задачі аналізу даних розв'язуються за допомогою методів лінійної алгебри?
3. Що таке скаляр?
4. Що таке вектор?
5. Що таке матриця?
6. Чим відрізняється вектор від матриці?
7. Які основні операції можна виконувати над векторами?
8. Які основні операції можна виконувати над матрицями?
9. Що таке транспонування матриці?
10. Що таке одинична матриця?
11. Що таке нульова матриця?
12. Що таке квадратна матриця?
13. Які умови повинні виконуватися для множення двох матриць?
14. Що таке скалярний добуток векторів?
15. Для чого використовується матричне множення в машинному навчанні?
16. Що таке визначник (детермінант) матриці?
17. Для чого використовується обернена матриця?
18. Що таке власні значення та власні вектори матриці?
19. Чому власні значення важливі для методів аналізу даних?
20. Що таке бібліотека NumPy?
21. Які переваги NumPy порівняно зі звичайними списками Python?
22. Як встановити бібліотеку NumPy?

23. Як імпортувати NumPy у Python-програму?
24. Що таке масив (ndarray) у NumPy?
25. Як створити одновимірний масив NumPy?
26. Як створити двовимірний масив NumPy?
27. Як отримати розмірність масиву?
28. Як визначити форму (shape) масиву?
29. Як створити масив, заповнений нулями?
30. Як створити масив, заповнений одиницями?
31. Для чого використовуються функції arange() та linspace()?
32. Як змінити форму масиву за допомогою reshape()?
33. Як звернутися до окремого елемента масиву?
34. Як виконати зрізи (slicing) масивів?
35. Як працює індексація багатовимірних масивів?
36. Як виконуються арифметичні операції над масивами NumPy?
37. Що таке векторизовані обчислення?
38. Які переваги векторизації над використанням циклів?
39. Як обчислити суму, середнє значення та стандартне відхилення елементів масиву?
40. Як знайти максимальне та мінімальне значення масиву?
41. Як виконати множення матриць у NumPy?
42. Для чого використовується функція dot()?
43. Як знайти транспоновану матрицю в NumPy?
44. Як обчислити обернену матрицю та визначник за допомогою NumPy?
45. Яку роль відіграють NumPy та лінійна алгебра в сучасному машинному навчанні та інтелектуальному аналізі даних?

Розділ 5

Обробка Природної Мови.

Обробка природної мови (Natural Language Processing, NLP) - напрям аналізу даних, що досліджує методи автоматичної обробки, аналізу та розуміння тексту й мовлення, створених людиною. NLP дає змогу комп'ютеру працювати з природною мовою та знаходити в ній корисну інформацію.

Велика кількість даних у багатьох реальних наборах даних представлена у вигляді так званого вільного тексту (free text). Це є, наприклад, коментарі користувачів, телеметрія програмних додатків, будь-які «неструктуровані» поля, та багато чого іншого.

Обчислювальна обробка природної мови полягає в тому, що ми пишемо комп'ютерні програми, які можуть розуміти природну мову. Обробка природної мови займається тим, як перетворити текст — часто нечіткий і непідготовлений — у форму, зручну для аналізу та моделювання. В цьому розділі ми спробуємо отримати деяку значущу інформацію з неструктурованих текстових даних.

До основних задач NLP належать класифікація текстів, аналіз тональності, визначення тематики документів, пошук інформації, автоматичний переклад, розпізнавання іменованих сутностей, створення текстових резюме, відповіді на запитання та генерація тексту.

5.1 Попередня обробка тексту.

Комп'ютер не сприймає текст так само, як людина. Для алгоритму слова є послідовністю символів, тому перед аналізом їх необхідно перетворити на структуроване представлення.

Попередня обробка дозволяє усунути зайві елементи, привести текст до єдиного формату та підготувати його для подальшого перетворення у числові ознаки, з якими можуть працювати алгоритми машинного навчання.

Типовий процес може включати очищення тексту, нормалізацію, токенізацію, вилучення стоп-слів, стемінг або лематизацію та подальше числове представлення тексту.

5.1.1 Очищення тексту.

Першим кроком до роботи з такими даними зазвичай стає підготовка тексту: приведення символів до одного регістру, видалення розділових знаків і зайвих символів, очищення чисел і зведення виробленого шуму у більш стандартизований вигляд.

Очищення тексту призначене для видалення елементів, які не несуть суттєвої інформації для конкретної задачі. Це може зменшити кількість зайвих ознак і спростити подальший аналіз. Наприклад, під час аналізу тематики документа можуть бути видалені HTML-теги, зайві пробіли або технічні символи.

Застосуємо біблотеку nltk для цих цілей.

```
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk
```

Спершу завантажимо ресурси для роботи із найпоширенішими словами в англійській мові ('the', 'is', 'and') та для роботи із пунктуацією: "stopwords"та "punkt":

```
nltk.download('stopwords')
nltk.download('punkt')
```

Стоп-слова (stop words) — це часто вживані слова, які в деяких задачах не містять достатньо інформації для розрізнення документів. До них можуть належати службові слова, прийменники, сполучники та займенники. Наприклад, для англійської мови це можуть бути the, a, and, is. Перелік стоп-слів залежить від мови та конкретної задачі аналізу.

Стоп-слова можуть становити значну частину тексту, але часто мало допомагають визначити його тематику. Їх вилучення дозволяє зменшити кількість токенів та ознак, прискорити обробку й зосередити увагу моделі на більш інформативних словах.

Видалення стоп-слів може бути шкідливим, якщо порядок і значення службових слів впливають на зміст. Наприклад, у задачах аналізу тональності вилучення слова «не» може повністю змінити зміст висловлювання.

Перш за все, варто розпочати роботу з текстом із того, щоб перевести всі літери в нижній регістр:

```
text = text.lower()
```

Та видалити розділові знаки:

```
text = text.translate(str.maketrans('', '', string.punctuation))
```

Також із тексту можна видалити числа:

```
text = re.sub(r'\d+', '', text)
```

Та спеціальні символи:

```
text = re.sub(r'[^a-z\s]', '', text)
```

Щоб видалити найпоширеніші в англійській мові стоп-слова:

```
stop_words = set(stopwords.words('english'))
text = ' '.join(word for word in word_tokenize(text) if word not in stop_words)
```

Повністю очищення тексту виглядає наступним чином:

```
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk
```

```
nltk.download('stopwords')
nltk.download('punkt')

def clean_text(text):
        text = text.lower()

        text = text.translate(str.maketrans('', '', string.punctuation))

        text = re.sub(r'\d+', '', text)

        text = re.sub(r'[^a-z\s]', '', text)

        stop_words = set(stopwords.words('english'))
        text = ' '.join(word for word in word_tokenize(text) if word not in
           stop_words)

        return text

sample_text = "Hello! This is an example text with numbers 123 and punctuation!!!"
cleaned_text = clean_text(sample_text)
print(cleaned_text)
```

Результат:

```
hello example text numbers punctuation
```

Залежно від задачі можуть видалятися зайві пробіли, HTML-теги, службові символи, непотрібні розділові знаки, спеціальні символи та частини тексту, які не містять корисної інформації. Однак універсального правила не існує. Наприклад, символ ! може бути важливим для визначення емоційності тексту, а # може мати значення під час аналізу хештегів.

5.1.2 Токенізація, стемінг, лематизація, нормалізація.

Після базового очищення йде токенізація — розбиття тексту на слова або речення, що дозволяє оперувати окремими «будівельними блоками» змісту. Далі часто застосовують стемінг або лематизацію: стемінг обрізає закінчення й повертає корінь слова, іноді утворюючи несловникові форми, тоді як лематизація намагається відновити словникову форму з урахуванням контексту. Нормалізація може також включати виправлення орфографії, розгортання абревіатур і узгодження синонімів — усе задля того, щоб одна й та сама ідея не розпадалася на багато еквівалентних представлень.

Приклад токенізації:

```
from nltk.tokenize import word_tokenize, sent_tokenize

def tokenize_text(text):
        words = word_tokenize(text)
        sentences = sent_tokenize(text)
        return words, sentences

sample_text = "Natural Language Processing (NLP) is exciting! It enables computers
   to understand human language."
```

```
words, sentences = tokenize_text(sample_text)
print('Words:', words)
print('Sentences:', sentences)
```

Результат:

```
Words: ['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'exciting', '!', 'It', 'enables',
    'computers', 'to', 'understand', 'human', 'language', '.']
Sentences: ['Natural Language Processing (NLP) is exciting!', 'It enables computers to understand human
    language.']
```

Найпростішим є словна токенізація, коли текст розділяється на окремі слова. Також застосовується токенізація за реченнями, коли текст поділяється на речення.

У сучасних NLP-системах широко використовується субсловна токенізація, за якої слова можуть розділятися на частини. Такий підхід використовується, зокрема, у Transformer-моделях і дозволяє ефективніше працювати з рідкісними та новими словами.

Перш ніж перейти до лематизації, варто завантажити в nltk спеціальний словник для цієї потреби:

```
nltk.download('wordnet')
```

Стемінг (Stemming) — це процес скорочення слова до його основи, або стема, шляхом видалення закінчень, суфіксів та інших частин слова за певними правилами. Наприклад, для англійської мови слова connect, connected, connecting можуть бути зведені до форми connect. Важливо, що результат стемінгу не обов'язково є повноцінним словом.

Основна мета стемінгу — об'єднати різні граматичні форми одного слова в одну ознаку. Це зменшує кількість різних токенів у тексті та дозволяє алгоритмам враховувати слова з однаковою основою як пов'язані між собою.

Головною перевагою стемінгу є простота та висока швидкість роботи. Він не потребує глибокого лінгвістичного аналізу тексту та дозволяє суттєво зменшити кількість унікальних слів. Недоліком є те, що алгоритм може надмірно скорочувати слова або об'єднувати слова, які мають різне значення. Крім того, отриманий стем може не бути реальною формою слова. Тому стемінг не завжди забезпечує лінгвістично коректний результат.

Лематизація (Lemmatization) — це процес приведення слова до його словникової форми, яка називається лемою. На відміну від простого скорочення слова, лематизація враховує граматичні характеристики та, залежно від використовуваного інструменту, контекст. Наприклад, англійські форми running, ran та runs можуть бути приведені до леми run. Стемінг механічно скорочує слово за певними правилами, тоді як лематизація намагається визначити його правильну словникову форму.

Лематизація доцільна тоді, коли значення та правильна форма слова мають важливе значення для аналізу. Її можна використовувати під час семантичного аналізу, пошуку інформації, аналізу текстів природною мовою та побудови систем, де важливо коректно інтерпретувати слова.

Стемінг та лематизація:

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize
import nltk

nltk.download('wordnet')

def apply_stemming(text):
```

```
        stemmer = PorterStemmer()
        words = word_tokenize(text)
        stemmed_words = [stemmer.stem(word) for word in words]
        return stemmed_words

def apply_lemmatization(text):
        lemmatizer = WordNetLemmatizer()
        words = word_tokenize(text)
        lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
        return lemmatized_words

sample_text = "The children are playing with their toys."
print('Stemmed:', apply_stemming(sample_text))
print('Lemmatized:', apply_lemmatization(sample_text))
```

Результат:

```
Stemmed: ['the', 'children', 'are', 'play', 'with', 'their', 'toy', '.']
Lemmatized: ['The', 'child', 'are', 'playing', 'with', 'their', 'toy', '.']
```

Нормалізація тексту — це приведення різних варіантів запису тексту до узгодженої форми. Її метою є зменшення непотрібних відмінностей між текстовими даними.

Приклад нормалізації з використанням модуля autocorrect:

```
from nltk.tokenize import word_tokenize
from autocorrect import Speller
import nltk

nltk.download('punkt')

def normalize_text(text):
        spell = Speller(lang='en')
        normalization_dict = {
                "u": "you",
                "ur": "your",
                "r": "are",
                "pls": "please",
                "thx": "thanks"
        }
        words = word_tokenize(text.lower())
        normalized_words = [normalization_dict.get(word, spell(word)) for word in
           words]
        normalized_text = ' '.join(normalized_words)
        normalized_text = normalized_text.replace('!', '').replace('.', '').strip()
        return normalized_text

sample_text = "U r the best! Pls help me with this assignment. Thx!"
print(normalize_text(sample_text))
```

Результат:

```
you are the best  please help me with this assignment  thanks
```

5.2 Чисельне представлення тексту.

Розуміння мови є складним завданням.

Розглянемо такий текст: «Під час полювання в Африці я застрелив слона в піжамі. Як він потрапив у мою піжаму, я не знаю». Без повного бачення всього тексту тяжко зрозуміти, що саме мається на увазі, наприклад, в першому реченні.

Основна думка полягає в наступному: ми використовуємо неймовірну кількість контексту, щоб зрозуміти значення речень природної мови. Саме з розумінням цього контексту допомагають великі мовні моделі та механізм attention в трансформерній архітектурі. Але чи завжди це настільки складно, щоб вимагати застосування передових технологій обробки тексту?

Розглянемо дві рецензії на фільм:

"Barbie is a fun movie that has surprisingly plenty of ***heartwarming*** *moments and* ***good*** *messages to offer. The set designs are* ***eye-catching*** *and on point as well. Both Margot Robbie and Ryan Gosling nailed their parts, but it is Gosling who mostly shines."*

"It didn't grab me; I didn't find it funny anywhere. It went from the most ***annoying*** *opening to as far as one can get annoying in every second of this film. The music was* ***shallow****; the production design teetered on Terry Gilliam's work but without the imagination or talent, and old 1920s/1930s film backgrounds and old Three Stooges routines were very* ***tiring****."*

Який з них є позитивним? Ми можемо уце зрозуміти, просто глянувши на виділені слова. Часто ми можемо дуже легко визначити «загальний зміст» тексту природною мовою, навіть не розуміючи речень.

5.2.1 Хмара слів та мішок слів.

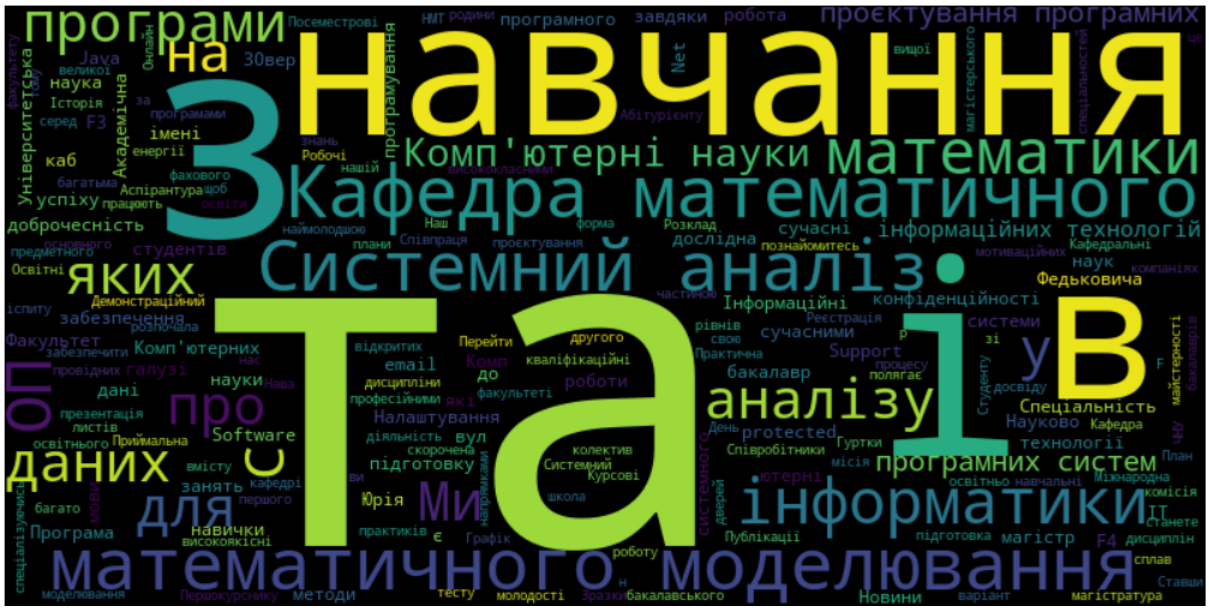
Хмара слів (Word Cloud) — це візуальне представлення тексту, у якому розмір слова залежить від частоти його появи або іншого показника важливості.

Генерація wordcloud (приклад збору тексту зі сторінки та візуалізації):

```
from wordcloud import WordCloud
from bs4 import BeautifulSoup
import requests
import re

response = requests.get("https://mathmod.chnu.edu.ua/")
root = BeautifulSoup(response.content, "lxml")
wc = WordCloud(width=800, height=400).generate(re.sub(r"\s+", " ", root.text))
wc.to_image()
```

Результат:



Хмара слів використовується переважно для первинного дослідження та візуалізації текстових даних. Вона може допомогти швидко визначити основні теми, ключові поняття та найбільш поширені слова. Наприклад, хмару слів можна побудувати для відгуків користувачів, щоб візуально визначити слова, які найчастіше використовуються у відгуках.

Хмара слів показує переважно частоту слів, але практично не враховує їхній контекст, порядок та взаємозв'язки. Часте слово не обов'язково є найважливішим з точки зору змісту.

Після підготовки тексту його потрібно представити чисельно. Простий і часто ефективний підхід — модель "мішка слів при якій кожен документ кодується вектором частот слів.

Основним недоліком є втрата порядку слів і контексту. Модель не розрізняє речення, у яких однакові слова розташовані по-різному.

5.2.2 TFIDF та косинусна подібність тексту.

"Частота терміна"(Term Frequency) просто означає кількість кожного слова в документі. Позначається $tf_{i,j}$ частотою слова i в документі (тексті) j (іноді індекси обертаються, ми використовуємо їх для узгодженості з матрицею).

```
documents = [
    "In Inception, the plot unfolds with thrilling action, keeping audiences on
       edge through a series of intense and unexpected twists.", # t_i_0 = 1
    "The story of Inception keeps audiences hooked with its thrilling action and
       surprising twists, making every moment unforgettable.", # t_i_1 = 1
    "The Matrix transformation preserves the vector's direction, illustrating how
       linear combinations affect the overall system." # t_i_2 = 0
]

document_words = [doc.lower().split() for doc in documents]
vocab = sorted(set(sum(document_words, [])))
vocab_dict = {k:i for i,k in enumerate(vocab)}
print(vocab, "\n")
print(vocab_dict, "\n")
```

Результат:

```
['a', 'action', 'action,', 'affect', 'and', 'audiences', 'combinations', 'direction,', 'edge', 'every',
    'hooked', 'how', 'illustrating', 'in', 'inception', 'inception,', 'intense', 'its', 'keeping', 'keeps',
    'linear', 'making', 'matrix', 'moment', 'of', 'on', 'overall', 'plot', 'preserves', 'series', 'story',
    'surprising', 'system.', 'the', 'thrilling', 'through', 'transformation', 'twists,', 'twists.',
    'unexpected', 'unfolds', 'unforgettable.', "vector's", 'with']

{'a': 0, 'action': 1, 'action,': 2, 'affect': 3, 'and': 4, 'audiences': 5, 'combinations': 6, 'direction,':
    7, 'edge': 8, 'every': 9, 'hooked': 10, 'how': 11, 'illustrating': 12, 'in': 13, 'inception': 14,
    'inception,': 15, 'intense': 16, 'its': 17, 'keeping': 18, 'keeps': 19, 'linear': 20, 'making': 21,
    'matrix': 22, 'moment': 23, 'of': 24, 'on': 25, 'overall': 26, 'plot': 27, 'preserves': 28, 'series':
    29, 'story': 30, 'surprising': 31, 'system.': 32, 'the': 33, 'thrilling': 34, 'through': 35,
    'transformation': 36, 'twists,': 37, 'twists.': 38, 'unexpected': 39, 'unfolds': 40, 'unforgettable.':
    41, "vector's": 42, 'with': 43}
```

Тепер побудуємо матрицю, яка містить кількість слів (частоту вживання термінів) для всіх документів.

```
import numpy as np

X_tf = np.zeros((len(documents), len(vocab)), dtype=int)
for i, doc in enumerate(document_words):
    for word in doc:
        X_tf[i, vocab_dict[word]] +=1

print(X_tf)
```

Результат:

```
[[1 0 1 0 1 1 0 0 1 0 0 0 0 1 0 1 1 0 1 0 0 0 0 0 1 1 0 1 0 1 0 0 0 1 1 1
  0 0 1 1 1 0 0 1]
 [0 1 0 0 1 1 0 0 0 1 1 0 0 0 1 0 0 1 0 1 0 1 0 1 1 0 0 0 0 0 0 1 1 0 1 1 0
  0 1 0 0 0 1 0 1]
 [0 0 0 1 0 0 1 1 0 0 0 1 1 0 0 0 0 0 0 0 1 0 1 0 0 0 1 0 1 0 0 0 1 3 0 0
  1 0 0 0 0 0 1 0]]
```

Частота вживання термінів, як правило, «перевантажена» дуже поширеними словами («the», «is», «of» тощо).

☞ Примітка: дуже поширені слова, які називаються «стоп-словами», можуть бути навіть видалені з використанням засобів nltk, як ми це обговорювали на початку цього розділа.

Ідея зворотної частоти документа (Inverse Document Frequency) полягає в тому, щоб надавати словам негативну вагу пропорційно до частоти їх вживання в усьому наборі документів.

$$idf_i = \log(\frac{N_{documents}}{N_{documents-with-word-i}})$$

Як і у випадку з частотою терміна, існують також інші версії з різними масштабами, але найпоширенішим є вищезазначений логарифмічний масштаб.

Зверніть увагу, що обернена частота документа визначається лише для слів, а не для пар «слово-документ», як у випадку з частотою терміна.

```
idf = np.log(X_tf.shape[0]/X_tf.astype(bool).sum(axis=0))
print(idf)
```

Результат:

```
[1.09861229 1.09861229 1.09861229 1.09861229 0.40546511 0.40546511
 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
```

```
 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
 0.40546511 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
 1.09861229 1.09861229 1.09861229 0.         0.40546511 1.09861229
 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
 1.09861229 0.40546511]
```

Ідея IDF полягає в тому, що якщо слово зустрічається майже в усіх документах, воно мало допомагає відрізняти один документ від іншого. Тому його IDF буде низьким. Якщо ж слово зустрічається лише в невеликій кількості документів, воно може бути хорошим індикатором їхньої специфічної тематики. Тому таке слово отримує вищу вагу IDF.

Зрештою, поєднання частоти терміна та зворотної частоти документа дає відому міру TF–IDF (term frequency inverse document frequency):

$$tfidf_{i,j} = tf_{i,j} \times idf_i,$$

яка підсилює рідкісні й інформативні слова й послаблює надто загальні. Це метод оцінювання важливості слова для конкретного документа з урахуванням того, наскільки часто це слово зустрічається в документі та наскільки рідкісним воно є в усій колекції.

```
X_tfidf = X_tf * idf
print(X_tfidf)
```

Результат:

```
[[1.09861229 0.         1.09861229 0.         0.40546511 0.40546511
  0.         0.         1.09861229 0.         0.         0.
  0.         1.09861229 0.         1.09861229 1.09861229 0.
  1.09861229 0.         0.         0.         0.         0.
  0.40546511 1.09861229 0.         1.09861229 0.         1.09861229
  0.         0.         0.         0.         0.40546511 1.09861229
  0.         0.         1.09861229 1.09861229 1.09861229 0.
  0.         0.40546511]
 [0.         1.09861229 0.         0.         0.40546511 0.40546511
  0.         0.         0.         1.09861229 1.09861229 0.
  0.         0.         1.09861229 0.         0.         1.09861229
  0.         1.09861229 0.         1.09861229 0.         1.09861229
  0.40546511 0.         0.         0.         0.         0.
  1.09861229 1.09861229 0.         0.         0.40546511 0.
  0.         1.09861229 0.         0.         0.         1.09861229
  0.         0.40546511]
 [0.         0.         0.         1.09861229 0.         0.
  1.09861229 1.09861229 0.         0.         0.         1.09861229
  1.09861229 0.         0.         0.         0.         0.
  0.         0.         1.09861229 0.         1.09861229 0.
  0.         0.         1.09861229 0.         1.09861229 0.
  0.         0.         1.09861229 0.         0.         0.
  1.09861229 0.         0.         0.         0.         0.
  1.09861229 0.        ]]
```

TF-IDF широко застосовується для пошуку інформації, класифікації документів, визначення схожості текстів, пошуку ключових слів та побудови рекомендаційних систем.

Косинусна подібність — простий спосіб виміряти схожість між двома векторними представленнями тексту:

$$CosineSim(x, y) = \frac{x \cdot y}{\|x\|_2 \|y\|_2}.$$

Більші числа означають, що вектори tfidf є ближчими в n-вимірному просторі, а отже, тексти подібніші між собою.

```
X_tfidf_norm = X_tfidf / np.linalg.norm(X_tfidf, axis=1)[:,None]
M = X_tfidf_norm @ X_tfidf_norm.T
print(M)
```

Результат:

```
[[1.         0.04991504 0.        ]
 [0.04991504 1.         0.        ]
 [0.         0.         1.        ]]
```

Як бачимо, тексти, що описували одні і ті ж речі, отримали більшу косинусну подібність, а третій текст з обома іншими має косинусну подібність 0.

Використання tfidf та косинусної подібності повністю:

```
import numpy as np

documents = [
        "In Inception, the plot unfolds with thrilling action, keeping audiences on
           edge through a series of intense and unexpected twists.",
        "The story of Inception keeps audiences hooked with its thrilling action
           and surprising twists, making every moment unforgettable.",
        "The Matrix transformation preserves the vector's direction, illustrating
           how linear combinations affect the overall system."
]

document_words = [doc.lower().split() for doc in documents]
vocab = sorted(set(sum(document_words, [])))
vocab_dict = {k: i for i, k in enumerate(vocab)}

X_tf = np.zeros((len(documents), len(vocab)), dtype=int)
for i, doc in enumerate(document_words):
        for word in doc:
                X_tf[i, vocab_dict[word]] += 1


idf = np.log(X_tf.shape[0] / X_tf.astype(bool).sum(axis=0))
X_tfidf = X_tf * idf
X_tfidf_norm = X_tfidf / np.linalg.norm(X_tfidf, axis=1)[:, None]
M = X_tfidf_norm @ X_tfidf_norm.T
print('Cosine similarity matrix:\n', M)
```

Результат:

```
Cosine similarity matrix:
 [[1.         0.04991504 0.        ]
 [0.04991504 1.         0.        ]
 [0.         0.         1.        ]]
```

Косинусна подібність популярна тому, що вона порівнює напрямок векторів, а не їхню абсолютну довжину. Це особливо корисно для текстів, оскільки два документи можуть мати різну

кількість слів, але бути схожими за змістом.

☞ Косинусна подібність добре працює для порівняння документів у просторі TF–IDF, але має й обмеження: в такому підході немає розуміння семантики слів, і різні слова, які мають близьке значення, залишаються віддаленими в просторі one-hot або частотних векторів.

Наприклад, наступні окремі слова мають між собою відстань 1, хоча деякі із них є більш спорідненими між собою:

$$||kyiv - zhytomyr|| = ||zhytomyr - uncle|| = 1$$

5.2.3 Векторне подання слів.

Щоб виправити недоліки косинусної подібності, використовують вбудовування слів (word embeddings) — щільні вектори фіксованого розміру, у яких семантично близькі слова займають близькі позиції у просторі. Моделі на кшталт word2vec або переднавчені матриці типу GloVe навчають векторні представлення таким чином, що арифметичні операції над векторами можуть відображати мовні відношення: наприклад, операція king − man + woman зазвичай наближається до вектора, близького до queen. Це відкриває шлях до більш змістовних порівнянь і до знаходження синонімів, антонімів і тематичних зв'язків.

Щоб завантажити векторне подання, варто спочатку встановити модуль sentence-transformers, та завантажити претреновану трансоффмер модель:

```
import numpy as np
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer('all-MiniLM-L6-v2')
```

Давайте глянемо, як виглядає векторне подання слова "computer"(або, хоча б, перші 10 чисел):

```
vector = model.encode(['computer'])[0]
print('vector sample:', vector[:10])
```

Результат:

```
vector sample: [-0.05127361  0.04783836 -0.00333908 -0.0388703  -0.03269714 -0.0328583
  0.09360427  0.03932324  0.08196709  0.01824838]
```

Тепер ми можемо обчислювати подібності між окремими словами:

```
embeddings = model.encode(['woman', 'man', 'kyiv'])

similarity = util.cos_sim(embeddings[0], embeddings[1]).item()
print('similarity woman/man:', similarity)

similarity = util.cos_sim(embeddings[1], embeddings[2]).item()
print('similarity man/kyiv:', similarity)
```

Результат:

```
similarity woman/man: 0.32567834854125977
similarity man/kyiv: 0.17196238040924072
```

Тепер, для прикладу, обчислимо вектор 'King - Man + Woman' та знайдемо схожі до нього.

```
words_to_encode = ['king', 'man', 'woman']
word_vectors = model.encode(words_to_encode)

king = word_vectors[0]
man = word_vectors[1]
woman = word_vectors[2]

result_vector = king - man + woman

vocabulary = ['queen', 'princess', 'monarch', 'empress', 'man', 'computer',
    'throne', 'kyiv']
vocab_embeddings = model.encode(vocabulary)

similarities = util.cos_sim(result_vector, vocab_embeddings)[0].numpy()

top_indices = np.argsort(similarities)[::-1][:5]

print('\nAnalogy results:')
for idx in top_indices:
    print(f"{vocabulary[idx]}: {similarities[idx]:.4f}")
```

Результат:

```
Analogy results:
queen: 0.5795
monarch: 0.5469
throne: 0.4804
princess: 0.4418
empress: 0.4368
```

Як бачимо, вектор 'King - Man + Woman' доволі таки близький до вектора 'Queen'.

Проведемо тепер tfidf-аналіз та знайдемо косинусну подібність між текстами, використовуючи тепер векторне представлення кожного окремого слова.

```
from sentence_transformers import SentenceTransformer, util

documents = [
    "kyiv has some excellent new restaurants",
    "kharkiv is a city with great cuisine",
    "postgresql is a relational database management system"
]

model = SentenceTransformer('all-MiniLM-L6-v2')

X_embeddings = model.encode(documents)
print("Embedding matrix:")
print(X_embeddings)
similarity_matrix = util.cos_sim(X_embeddings, X_embeddings)

print("\nCosinus similarity matrix:")
print(similarity_matrix.numpy())
```

Результат:

```
Embedding matrix:
[[-0.00245756 -0.05211234  0.05479724 ...  0.05498891 -0.00664046
  -0.02283956]
 [ 0.04663361  0.06056985 -0.02163695 ...  0.04387321 -0.0416828
   0.0089187 ]
 [ 0.01276393 -0.00399948 -0.08929092 ... -0.02673415  0.04939254
   0.0399594 ]]

Cosinus similarity matrix:
[[ 1.          0.53684413 -0.01055246]
 [ 0.53684413  1.          0.03759826]
 [-0.01055246  0.03759826  1.        ]]
```

Далі стоять мовні моделі, які намагаються оцінити ймовірність слова, враховуючи контекст: $P(word_i \mid word_1, \ldots, word_{i-1})$. Простий, але корисний клас таких моделей — n-грами, що враховують обмежений контекст попередніх $n-1$ слів; їх можна вважати аппроксимацією повної ймовірнісної моделі мови. Сучасні нейронні та трансформерні моделі значно перевершують n-грами за здатністю захоплювати довготривалі залежності та контекст.

У практиці науковця з даних частина роботи полягає в доборі правильного представлення залежно від задачі: для швидкої класифікації рецензій часто достатньо TF–IDF і лінійних моделей; для складних завдань семантичного пошуку або витягнення сутностей корисні вбудовування слів і контекстні мовні моделі. Важливо пам'ятати, що підготовка даних і прості методи іноді дають вражаючі результати, і вибір інструменту має базуватись на доступності даних, вимогах до точності й обмеженнях по часу на навчання.

✎ **Питання для самоконтролю.**

1. Що таке обробка природної мови (Natural Language Processing, NLP)?
2. Які задачі належать до сфери обробки природної мови?
3. Чому текстові дані потребують попередньої обробки?
4. Які основні етапи попередньої обробки тексту?
5. Що таке токенізація?
6. Які існують способи токенізації тексту?
7. Для чого виконується очищення тексту?
8. Які символи зазвичай видаляють під час очищення тексту?
9. Що таке стоп-слова?
10. Чому стоп-слова часто вилучають із тексту?
11. У яких випадках видаляти стоп-слова недоцільно?
12. Що таке нормалізація тексту?
13. Для чого текст переводять до нижнього регістру?
14. Чим нормалізація відрізняється від токенізації?
15. Що таке стемінг?
16. Які задачі вирішує стемінг?

17. Які переваги та недоліки стемінгу?
18. Що таке лематизація?
19. Чим лематизація відрізняється від стемінгу?
20. У яких випадках доцільніше використовувати лематизацію?
21. Що таке хмара слів (Word Cloud)?
22. Для чого використовується хмара слів?
23. Які обмеження має хмара слів як інструмент аналізу?
24. Що таке модель "мішок слів"(Bag of Words)?
25. Як формується матриця Bag of Words?
26. Які недоліки моделі Bag of Words?
27. Що таке TF (Term Frequency)?
28. Як обчислюється TF?
29. Що таке IDF (Inverse Document Frequency)?
30. Чому рідкісні слова мають вищу вагу в IDF?
31. Що таке TF-IDF?
32. Які переваги TF-IDF порівняно з Bag of Words?
33. У яких задачах застосовується TF-IDF?
34. Що таке векторне подання тексту?
35. Як текст перетворюється на числовий вектор?
36. Що таке косинусна подібність?
37. Як інтерпретується значення косинусної подібності?
38. Чому косинусна подібність є популярною мірою схожості документів?
39. Що таке векторне подання слів (Word Embeddings)?
40. Чим векторні подання слів відрізняються від TF-IDF?
41. Які переваги мають моделі Word2Vec та GloVe?
42. Які сучасні моделі контекстних векторних подань слів існують?
43. У яких задачах використовуються векторні подання слів?
44. Які бібліотеки Python найчастіше застосовують для обробки природної мови?
45. Які сучасні напрями розвитку технологій обробки природної мови пов'язані з великими мовними моделями (LLM)?

Розділ 6

Теорія ймовірності в контексті аналізу даних.

6.1 Основи теорії ймовірності.

Теорія ймовірності це розділ математики, що вивчає закономірності випадкових явищ і дає змогу оцінити, наскільки ймовірною є та чи інша подія.

Ніколи не можна точно знати, якою, наприклад, буде завтрашня погода. Проте маючи дані спостереження за регіоном на протязі певного періоду часу в даний сезон, можна оцінити, з якою ймовірністю завтра буде дощ. Спеціаліст з аналізу даних повинен вміти працювати з ймовірностями для того, щоб працювати з невизначеністю, прогнозувати майбутні події, оцінювати ризики та будувати моделі машинного навчання.

Теорія ймовірності лежить в основі багатьох алгоритмів машинного навчання, статистики, прогнозування часових рядів, аналізу ризиків і систем підтримки прийняття рішень.

Теорія ймовірності вивчає випадкові явища та закономірності їх виникнення. Центральними поняттями цієї теорії є випадковий експеримент, подія та ймовірність. Саме вони лежать в основі статистичного аналізу, машинного навчання, прогнозування, аналізу ризиків та багатьох інших методів інтелектуального аналізу даних.

```
import random
import math
import numpy as np
from collections import Counter

pX = {1: 0.1, 2: 0.5, 3:0.4}
pWeather = {"sunny": 0.3, "rainy":0.2, "cloudy":0.4, "snowy":0.1}
print(pWeather["sunny"])
```

Результат:

```
0.3
```

Випадковий експеримент — це дія або процес, результат якого наперед точно невідомий, хоча всі можливі результати можна перелічити. Прикладами випадкових експериментів є підкидання монети, кидання грального кубика, витягування карти з колоди, випадковий вибір клієнта із бази даних або визначення того, чи відкриє користувач рекламне повідомлення.

Елементарною подією називають окремий можливий результат випадкового експерименту. Наприклад, при киданні шестигранного кубика можливими елементарними подіями є випадіння

чисел 1, 2, 3, 4, 5 або 6. Під час підкидання монети існує лише дві елементарні події: випадіння герба або цифри.

Приклад: Кидання монети. Симулюємо 10 підкидувань, обчислюємо емпіричну ймовірність випадання «решки».

```
tosses = [random.choice(["H", "T"]) for _ in range(10)]
print(tosses)

p_heads = tosses.count("H") / len(tosses)
print(f"P(Heads) approx. {p_heads:.2f}")
```

Результат:

```
['H', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'T', 'T']
P(Heads) approx. 0.50
```

Множина всіх можливих елементарних подій називається простором елементарних подій, і позначається символом (Ω).

Для кидання кубика $\Omega = 1, 2, 3, 4, 5, 6$, для підкидання монети $\Omega =$ герб, цифра. Будь-яка випадкова подія є підмножиною простору елементарних подій.

Ми називаємо дві події "незалежними якщо однієї з них не впливає на ймовірність настання іншої. Наприклад, якщо двічі підкинути монету, результат другого підкидання не залежить від результату першого - якщо ми знаємо, що перший раз випала "решка це нічого не каже нам про те, яким буде результат наступного підкидання.

Для двох незалежних подій справедливою є наступна формула:

$$P(A \cap B) = P(A) \cdot P(B).$$

Наприклад, імовірність того, що при двох підкиданнях монети двічі випаде герб, дорівнює $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$.

6.1.1 Закон великих чисел.

Одним із фундаментальних математичних законів теорії ймовірності є закон великих чисел.

Давайте подивимося, як емпірична ймовірність наближається до теоретичної ймовірності для різної кількості експериментів. Симулюємо підкидання монети 10, 100, 1000 та 10000 разів:

```
def simulate_coin(n_tosses=1000):
    tosses = [random.choice(["H", "T"]) for _ in range(n_tosses)]
    return tosses.count("H") / n_tosses

for n in [10, 100, 1000, 10000]:
    print(f"{n} tosses -> P(Heads) approx. {simulate_coin(n):.3f}")
```

Результат:

```
10 tosses -> P(Heads) approx. 0.200
100 tosses -> P(Heads) approx. 0.580
1000 tosses -> P(Heads) approx. 0.496
10000 tosses -> P(Heads) approx. 0.500
```

По мірі того, як число підкидувань зростає, ймовірність стабілізується около 0.5. Це є наглядним прикладом виконання закону великих чисел.

Закон великих чисел стверджує, що при багаторазовому повторенні однакового випадкового експерименту за однакових умов відносна частота появи певної події наближається до її теоретичної ймовірності. Для вищенаведеного прикладу із монетами, ця теоретична ймовірність рівна якраз 0.5.

Закон великих чисел пояснює, чому аналіз великих обсягів даних дозволяє отримувати більш надійні висновки.

Якщо вибірка містить дуже мало спостережень, випадкові коливання можуть суттєво спотворювати результати. Зі збільшенням кількості даних вплив випадковості зменшується, а оцінки характеристик генеральної сукупності стають точнішими. Саме тому в дослідженні даних використовуються якомога більші вибірки.

У машинному навчанні закон великих чисел є однією з причин, чому якість моделей зазвичай покращується зі збільшенням обсягу навчальних даних.

6.2 Використання NumPy для роботи з розподілами ймовірностей.

6.2.1 Рівномірний розподіл.

В наступному прикладі генеруємо випадкові числа між 0 та 1, та виводимо ймовірність того, що значення із цього розподілу менше за 0.5.

```
import numpy as np

samples = np.random.rand(5)
print(samples)

p = np.mean(samples < 0.5)
print("P(X < 0.5) approx.", p)
```

Результат:

```
[0.75955599 0.02132349 0.45415367 0.32714836 0.93499247]
P(X < 0.5) approx. 0.6
```

6.2.2 Гаусівський розподіл.

Гаусівський розподіл (Normal, Gaussian Distribution) є розподілом по дійсних числах і, ймовірно, є найпоширенішим розподілом у науці про дані (я не кажу, що це найпоширеніший розподіл у самих даних, а в науці про дані як галузі). Це пояснюється багатьма приємними властивостями розподілу: він легко інтегрується, має природний багатовимірний аналог, а також тим фактом, про який ми поговоримо в декількох лекціях, що певні розподіли даних середніх значень вибірки мають тенденцію до гаусівського розподілу (так звана центральна гранична теорема). Гаусівський розподіл має "дзвоноподібну"функцію щільності, яка, ймовірно, знайома багатьом людям.

Згенеруємо гаусівський розподіл із середнім 0 та відхиленням 1 засобами NumPy:

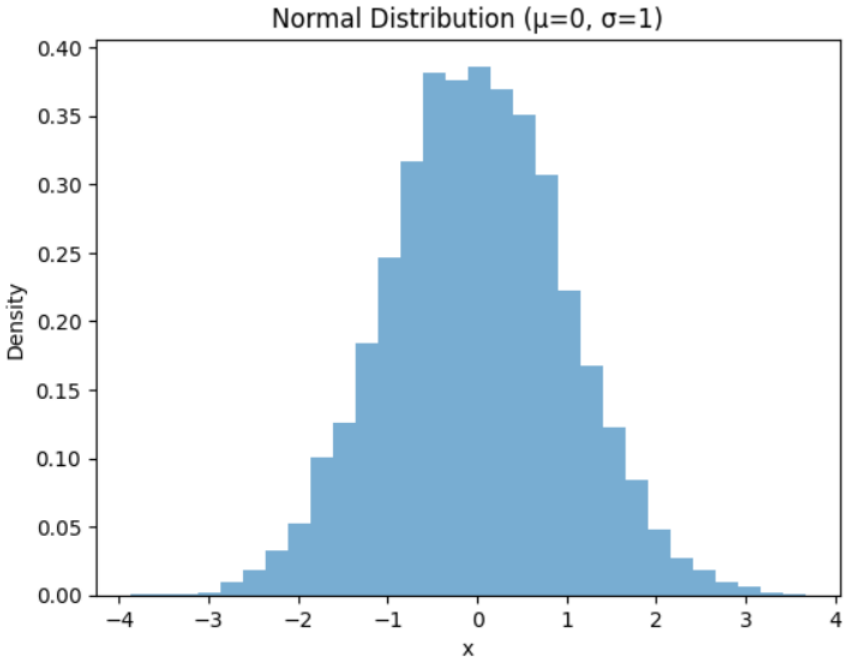
```
samples = np.random.normal(loc=0, scale=1, size=10000)
print("Mean:", np.mean(samples))
print("Std Dev:", np.std(samples))

import matplotlib.pyplot as plt
```

```
plt.hist(samples, bins=30, density=True, alpha=0.6)
plt.title("Normal Distribution (nu=0, sigma=1)")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()
```

Результат:

```
Mean: -0.002846126267790618
Std Dev: 0.9992466721005764
```



Можна навести багато прикладів, де зустрічається нормальний розподіл: зріст людей (запишіть ріст ваших одногруників - ви побачите, що він підпадає саме під нормальний розподіл), результати іспитів, середній час виконання тієї чи іншої дії. Багато математичних моделей було розроблено із припущенням, що дані мають саме нормальний розподіл.

6.2.3 Дискретні розподіли.

Задамо параметри розподілу: кількість випробувань 10 та ймовірність успіху 0.5. Обчислимо ймовірність випадання рівно 3 "решок"за 10 підкидань, та кумулятивну ймовірність $P(X \leq 3)$.

```
from scipy.stats import binom

n = 10
p = 0.5

prob = binom.pmf(k=3, n=n, p=p)
print(f"P(X=3) = {prob:.4f}")

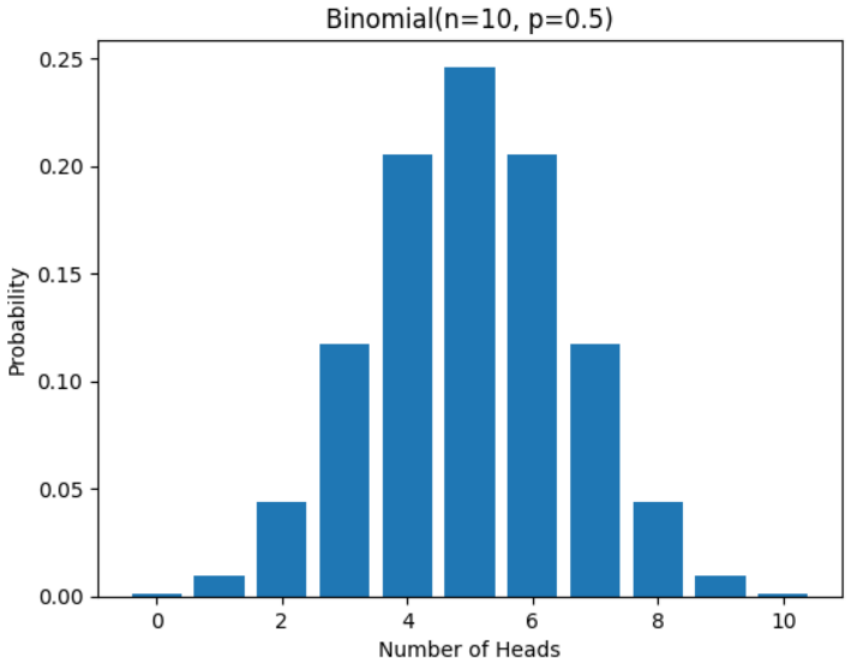
cum_prob = binom.cdf(k=3, n=n, p=p)
print(f"P(X <= 3) = {cum_prob:.4f}")
```

```
x = np.arange(0, 11)
y = binom.pmf(x, n, p)

plt.bar(x, y)
plt.xlabel("Number of Heads")
plt.ylabel("Probability")
plt.title("Binomial(n=10, p=0.5)")
plt.show()
```

Результат:

```
P(X=3) = 0.1172
P(X <= 3) = 0.1719
```



6.3 Теорема Байєса.

Обчислимо ймовірність захворювання за умови позитивного результату тесту. Якщо 1 % населення хворіє, то P(Захворювання) = 0,01. Точність тесту становить 99 %, то P(Позитивний результат | Захворювання) = 0,99; P(Позитивний результат | Відсутність захворювання) = 0,01. Обчислимо P(Захворювання | Позитивний результат):

```
P_D = 0.01
P_Pos_given_D = 0.99
P_Pos_given_notD = 0.01

P_Pos = P_Pos_given_D * P_D + P_Pos_given_notD * (1 - P_D)

P_D_given_Pos = (P_Pos_given_D * P_D) / P_Pos
print(f"P(Disease | Positive Test) = {P_D_given_Pos:.3f}")
```

Результат:

```
P(Disease | Positive Test) = 0.500
```

✎ Питання для самоконтролю.

1. Що вивчає теорія ймовірності?
2. Чому теорія ймовірності є важливою для інтелектуального аналізу даних?
3. Що таке випадковий експеримент?
4. Що таке елементарна подія?
5. Що називають простором елементарних подій?
6. Що таке випадкова подія?
7. Які види подій розрізняють у теорії ймовірності?
8. Що таке достовірна подія?
9. Що таке неможлива подія?
10. Які події називають несумісними?
11. Які події називають незалежними?
12. Як визначається ймовірність події?
13. Які властивості має ймовірність?
14. Як обчислюється ймовірність суми несумісних подій?
15. Як обчислюється ймовірність суми довільних подій?
16. Як обчислюється ймовірність перетину незалежних подій?
17. Що таке умовна ймовірність?
18. Як обчислюється умовна ймовірність?
19. У чому полягає правило множення ймовірностей?
20. Що таке повна група подій?
21. У чому полягає формула повної ймовірності?
22. Що таке випадкова величина?
23. Чим дискретна випадкова величина відрізняється від неперервної?
24. Що таке закон розподілу випадкової величини?
25. Що таке функція розподілу?
26. Що таке математичне сподівання?
27. Як обчислюється математичне сподівання дискретної випадкової величини?
28. Який практичний зміст має математичне сподівання?
29. Що таке дисперсія випадкової величини?
30. Для чого використовується стандартне відхилення?

31. Що показує дисперсія в аналізі даних?
32. У чому полягає закон великих чисел?
33. Яке практичне значення закону великих чисел?
34. Як закон великих чисел застосовується в машинному навчанні?
35. Що таке біноміальний розподіл?
36. У яких задачах застосовується біноміальний розподіл?
37. Що таке розподіл Бернуллі?
38. Чим розподіл Бернуллі відрізняється від біноміального?
39. Що таке нормальний розподіл?
40. Якими властивостями характеризується нормальний розподіл?
41. Чому нормальний розподіл часто зустрічається в реальних даних?
42. Що таке розподіл Пуассона?
43. У яких практичних задачах застосовується розподіл Пуассона?
44. У чому полягає теорема Байеса?
45. Як обчислюється апостеріорна ймовірність за теоремою Байеса?
46. Яке практичне значення має теорема Байеса для аналізу даних?
47. Як теорема Байеса використовується в задачах класифікації?
48. Що таке наївний байєсівський класифікатор і на якому припущенні він ґрунтується?
49. Яку роль відіграють ймовірнісні моделі в сучасному інтелектуальному аналізі даних?
50. Чому розуміння основ теорії ймовірності є необхідним для побудови моделей машинного навчання та прийняття рішень на основі даних?

Розділ 7

Вступ в машинне навчання.

До цього часу ми дивилися до того, як спеціаліст сам може описувати всі правила, за якими працює обробка даних. Але задачі, які постають перед сучасним дослідником, є занадто громіздкими для рутинної обробки. Саме тому постала необхідність у використання алгоритмів , які самі знаходять закономірності у навчальних даних, та надають правильні відповіді.

Наприклад, щоб навчити комп'ютер розпізнавати електронні листи зі спамом, достатньо показати йому тисячі прикладів звичайних листів і спаму. Після навчання модель зможе класифікувати нові повідомлення.

Машинне навчання є на даний момент швидкозростаючою галуззю, хоча слід підкреслити, що початкові методи, які ми тут розглянемо (а саме лінійна регресія), з'явилися приблизно на 200 років раніше, ніж сам термін (зазвичай вважається, що Гаусс розробив метод найменших квадратів близько 1800 року). Більшість пізніших методів, про які ми говоримо, були добре вивчені в статистиці на початку століття. Ми все одно будемо використовувати загальний термін "машинне навчання"для позначення всіх цих методів, але важливо знати, що ці ідеї не виникли в спільноті ML. Натомість тема машинного навчання розвинулася завдяки поєднанню трьох різних елементів:

- зростання обчислювальної потужності (машинне навчання, яке виникло з інформатики, завжди було фундаментально пов'язане з обчислювальними алгоритмами),
- величезні обсяги доступних даних (сировина для методів машинного навчання) та
- деякі значні досягнення в галузі алгоритмів, що відбулися за останні 30 років. Однак для початку ми розглянемо деякі з найпростіших алгоритмів, щоб систематизувати основні принципи.

У традиційному програмуванні розробник задає правила та алгоритм, а програма разом із вхідними даними формує результат. У машинному навчанні розробник надає дані та бажані результати, а алгоритм намагається автоматично визначити закономірності, які пов'язують вхідні дані з результатом. Саме в цьому і полягає основна відмінність між традиційним програмуванням та машинним навчанням.

Машинне навчання використовується для прогнозування, класифікації, кластеризації, виявлення аномалій та пошуку закономірностей у даних. Наприклад, за його допомогою можна прогнозувати ціни, визначати ризик захворювання, виявляти шахрайські транзакції, сегментувати клієнтів, рекомендувати товари або прогнозувати попит.

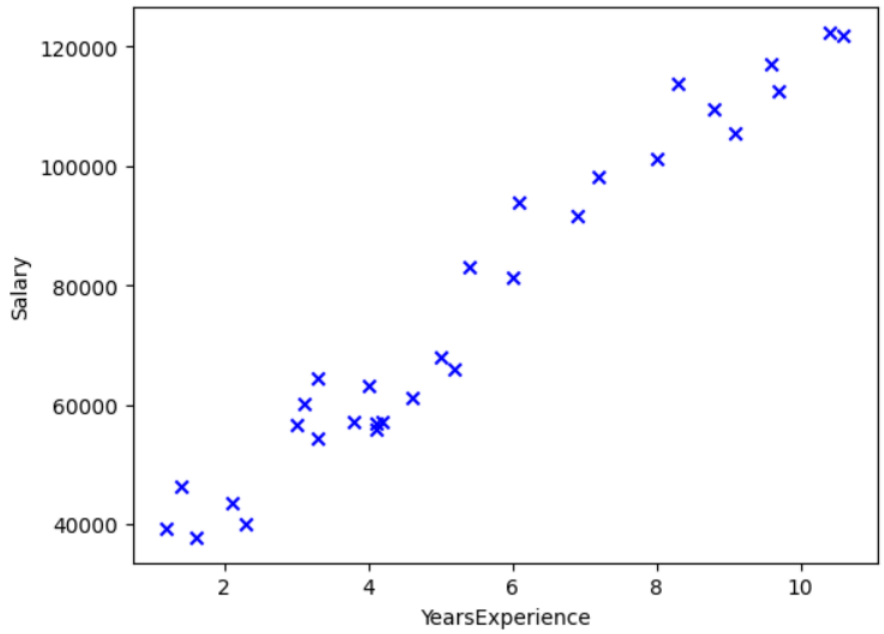
```
import matplotlib.pyplot as plt
import pandas as pd

def build_plot(file_path, x, y):
```

```
    data = pd.read_csv(file_path)
    plt.scatter(data[x], data[y], c='blue', marker='x')
    plt.xlabel(x)
    plt.ylabel(y)
    return plt

build_plot('../resources/Salary_dataset.csv', 'YearsExperience', 'Salary').show()
```

Результат:



Ось простий приклад залежності між стажем роботи співробітників та їхньою заробітною платою. Чи можна передбачити другий параметр на основі значення першого?

Можна припустити, що залежність відповідає лінійній моделі.

$$y \approx \theta_1 \cdot x + \theta_2$$

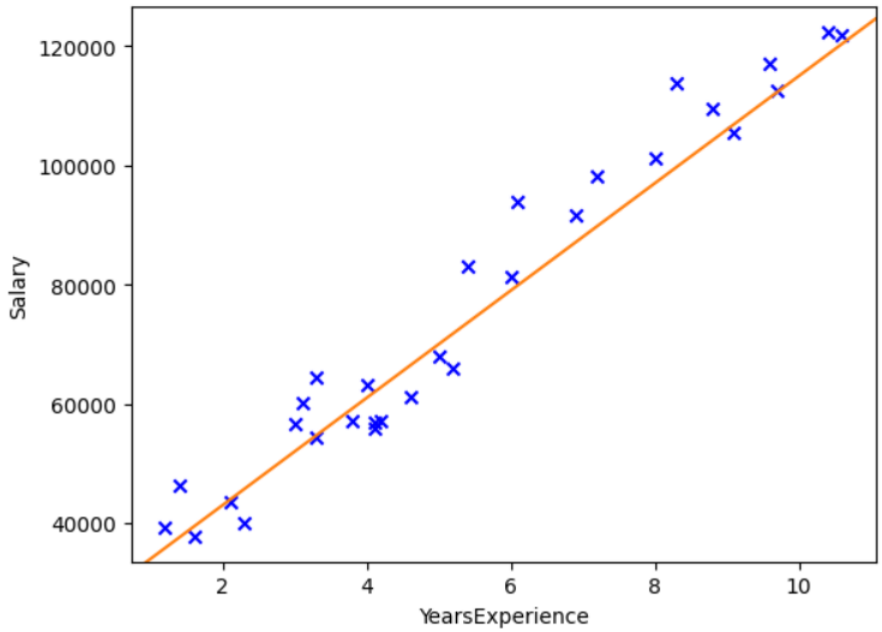
Тут - θ_1 це "нахил" лінії, та (slope) - θ_2 зсув. (intercept)

Чи можемо ми робити передбачення даних?

```
import numpy as np

def build_line(plt, theta):
    xlim, ylim =(plt.gca().get_xlim(), plt.gca().get_ylim())
    plt.plot(xlim, [theta[0]*xlim[0]+theta[1], theta[0]*xlim[1]+theta[1]], 'C1')
    plt.xlim(xlim)
    plt.ylim(ylim)

plt = build_plot('../resources/Salary_dataset.csv', 'YearsExperience', 'Salary')
build_line(plt, np.array([9000, 25000]))
```



Ознака (feature) — це характеристика об'єкта, яка використовується моделлю для прийняття рішення. Цільова змінна (target) - це значення, яке модель повинна навчитися прогнозувати. У нашому прикладі, стаж працівника є ознакою (зазвичай у задачах машинного навчання їх значно більше, ніж одна), а заробітна плата - це цільова змінна.

7.1 Градієнтний спуск.

Припустимо, що прогнозована зарплата відповідає лінійній моделі. Як ми можемо знайти θ?

Є багато можливостей, але природною метою є мінімізація деякої різниці між цією лінією і спостережуваними даними, наприклад, квадратична втрата:

$$E(\theta) = \sum_i (\theta_1 * x_i + \theta_2 - y_i)^2$$

Тепер нам потрібно знайти параметри, які мінімізують значення функції $E(\theta)$.

$$minimize_\theta E(\theta)$$

Найкращий спосіб знайти значення функції $f(x)$, в якій її значення є мінімальним, — це знайти похідну. Ми починаємо з деякої точки і змінюємо вхідне значення в напрямку від'ємної похідної. У випадку багатозмінних функцій ми будемо використовувати часткові похідні.

Щоб знайти хороше значення θ, ми можемо повторно робити кроки в напрямку негативних похідних для кожного значення. Ітерація:

$$\theta_1 := \theta_1 - \alpha \frac{\delta}{\delta \theta_1} E(\theta_1, \theta_2)$$

$$\theta_2 := \theta_2 - \alpha \frac{\delta}{\delta \theta_2} E(\theta_1, \theta_2)$$

де α — це невелике додатне число, яке називається розміром кроку, або швидкістю навчання (Learning Rate).

Якщо learning rate занадто малий, модель навчається дуже повільно, оскільки робить маленькі кроки. Якщо learning rate занадто великий, алгоритм може перестрибувати через мінімум, коливатися або навіть не збігатися.

Це алгоритм градієнтного спуску, основа сучасного машинного навчання.
Тут:

$$\frac{\delta}{\delta\theta_1}E(\theta_1,\theta_2) = \sum_i 2(\theta_1 * x_i + \theta_2 - y_i) * x_i$$

$$\frac{\delta}{\delta\theta_2}E(\theta_1,\theta_2) = \sum_i 2(\theta_1 * x_i + \theta_2 - y_i)$$

Застосуємо градієнтний спуск (Gradient descent) до наших даних, спочатку нормалізувавши їх.

```
def build_plot_normalized(file_path, x, y):
    data = pd.read_csv(file_path)
    x_nor = (data[x] - min(data[x])) / (max(data[x]) - min(data[x]))
    y_nor = (data[y] - min(data[y])) / (max(data[y]) - min(data[y]))
    plt.scatter(x_nor, y_nor, c='blue', marker='x')
    plt.xlabel(x)
    plt.ylabel(y)
    return plt

build_plot_normalized('../resources/Salary_dataset.csv', 'YearsExperience',
    'Salary').show()
```

Візуалізуємо процес градієнтного спуску. Поки що, проведемо лише одну ітерацію.

```
# Visualizing the gradient descent:

def get_data(file_path, x_name, y_name):
    data = pd.read_csv(file_path)
    return data[x_name], data[y_name]

def normalize_data(x):
    x_normalized = (x - min(x)) / (max(x) - min(x))
    return x_normalized

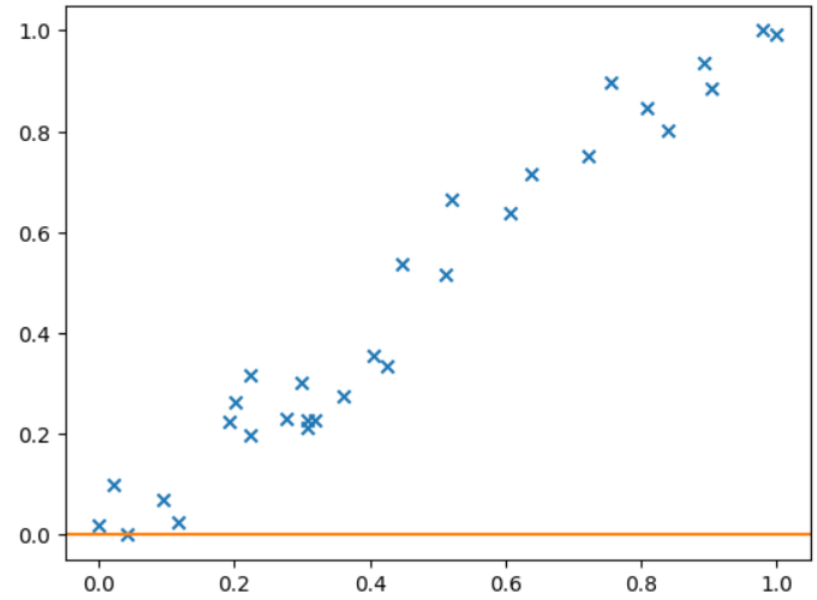
def gradient_descent(x, y, iters, alpha):
    theta = np.array([0., 0.])
    for t in range(iters):
        theta[0] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y)*x)
        theta[1] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y) )
    return theta

def plot_fit(x, y, theta):
    plt.scatter(x, y, marker = 'x')
    xlim, ylim =(plt.gca().get_xlim(), plt.gca().get_ylim())
    plt.plot(xlim, [theta[0]*xlim[0]+theta[1], theta[0]*xlim[1]+theta[1]], 'C1')
    plt.xlim(xlim)
    plt.ylim(ylim)

x, y = get_data('../resources/Salary_dataset.csv', 'YearsExperience', 'Salary')
```

```
x_normalized, y_normalized = normalize_data(x), normalize_data(y)

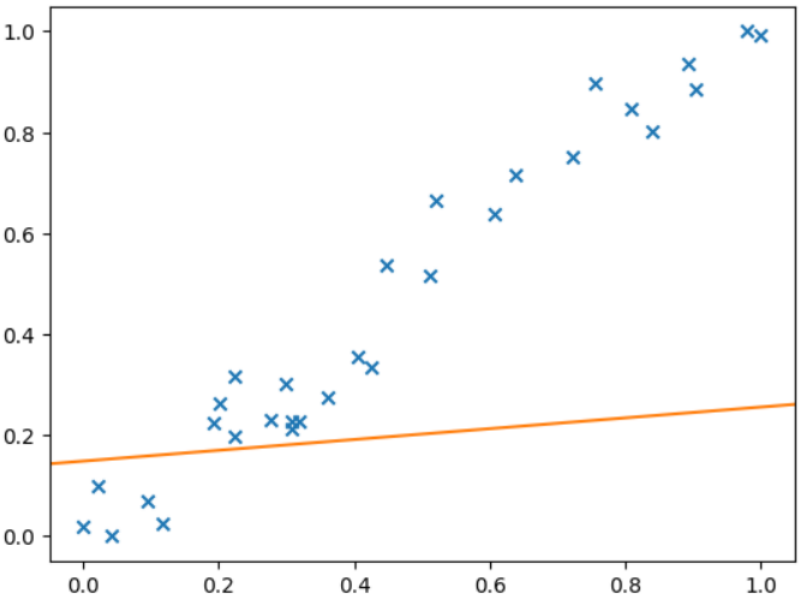
#first iteration
theta = gradient_descent(x_normalized, y_normalized, 0, 0.1)
plot_fit(x_normalized, y_normalized, theta)
```



Як бачимо, на першій ітерації значення параметрів θ віддалені від бажаних. Проведемо іще декілька ітерацій.

```
theta = gradient_descent(x_normalized, y_normalized, 2, 0.1) # change third
   parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```

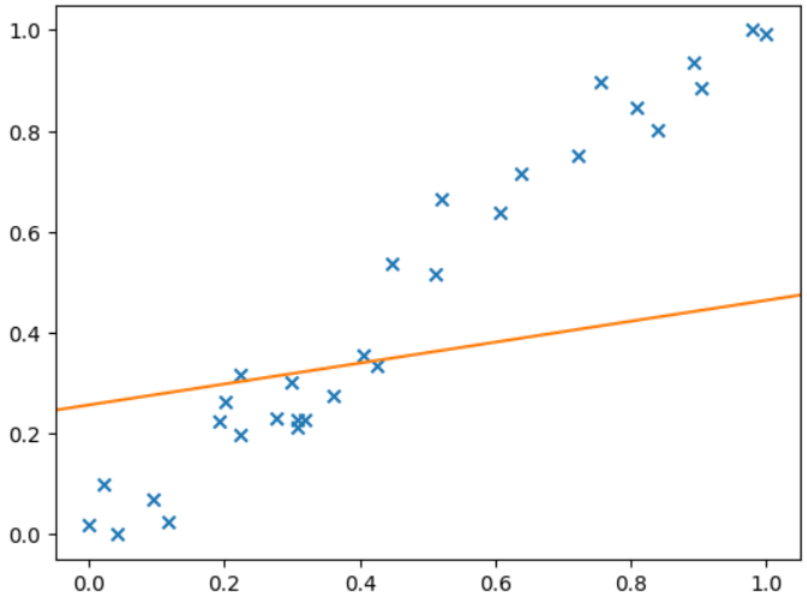
Результат:




```
theta = gradient_descent(x_normalized, y_normalized, 5, 0.1) # change third
```

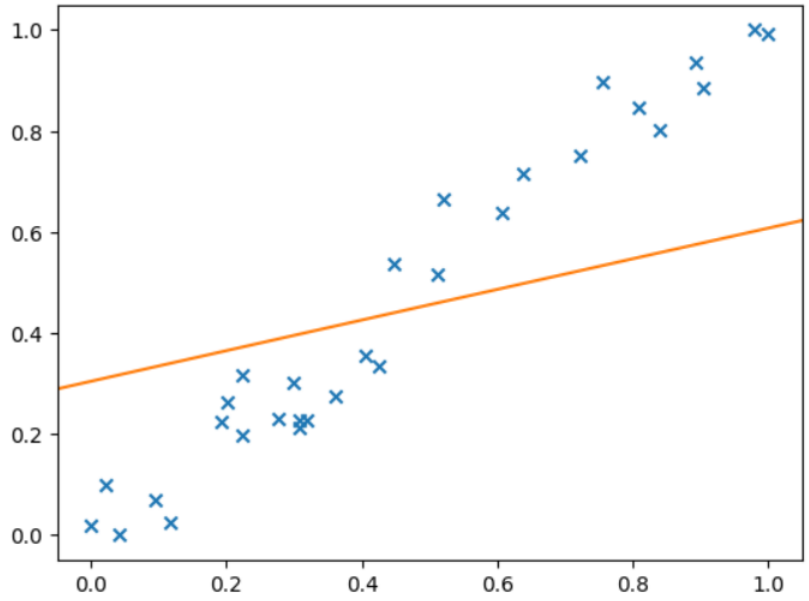
```
    parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```

Результат:



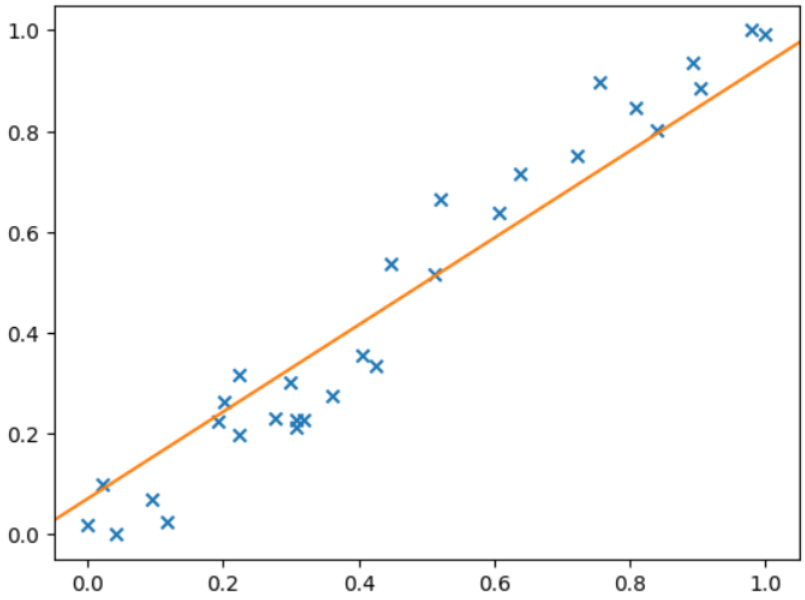

```
theta = gradient_descent(x_normalized, y_normalized, 10, 0.1) # change third
    parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```

Результат:




```
theta = gradient_descent(x_normalized, y_normalized, 100, 0.1) # change third
    parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```

Результат:



Після 100-ї ітерації ми наблизилися із нашою лінійною моделлю до реального розподілу.

☞ Мета моделі машинного навчання полягає не в тому, щоб обов'язково пройти через кожну точку, а в тому, щоб знайти закономірність, яка найкраще описує дані. У реальних даних присутні шум, похибки вимірювання та інші випадкові фактори. Тому точки можуть розташовуватися не точно на одній прямій. Якщо змусити модель пройти через усі точки, вона може запам'ятати випадкові особливості навчальних даних і гірше працювати з новими даними.

$E(\theta) = \sum_i (\theta_1 * x_i + \theta_2 - y_i)^2$ у нашому випадку є функцією втрат - математичної функцією, яка оцінює, наскільки прогнози моделі відрізняються від правильних значень. Чим менше значення функції втрат, тим краще модель узгоджується з навчальними даними.

Зверніть увагу піднесення до квадрату у формулі обчислення помилки. Це - середньоквадратична помилка (Mean Squared Error, MSE) - вона використовується для оцінювання якості моделей регресії. MSE показує середній квадрат різниці між фактичними та прогнозованими значеннями. Особливістю MSE є те, що великі помилки отримують значно більшу вагу через піднесення до квадрата. Тому MSE особливо чутлива до великих відхилень прогнозів. Функція втрат показує, наскільки погано модель прогнозує результати. Тому під час навчання потрібно знайти такі параметри моделі, за яких значення функції втрат буде якомога меншим - тобто, оптимізувати модель.

Для оптимізації моделі ми застосували алгоритм градієнтного спуску (Gradient Descent). Як ми щойно побачили на прикладі, цей алгоритм поступово змінює параметри моделі, рухаючись у напрямку зменшення значення функції втрат.

Уявімо функцію втрат як поверхню з пагорбами та западинами. Мета алгоритму — знайти найнижчу точку цієї поверхні. На кожній ітерації алгоритм визначає, у якому напрямку функція втрат зростає найшвидше, а потім рухається у протилежному напрямку, щоб знайти мінімальне значення цієї функції.

Градієнт — це вектор часткових похідних функції за всіма її параметрами. Для функції втрат він показує, у якому напрямку і наскільки швидко змінюється втрата при зміні параметрів моделі.

Градієнтний спуск завершує роботу, коли подальше оновлення параметрів майже не покращує модель. На практиці використовують одну або кілька умов зупинки: зміна функції втрат стає меншою за заданий поріг, величина градієнта стає дуже малою або досягнуто максимальної кількості ітерацій.

Ми також можемо візуалізувати процес зміни параметрів θ:

```
# Visualizing parameter updates

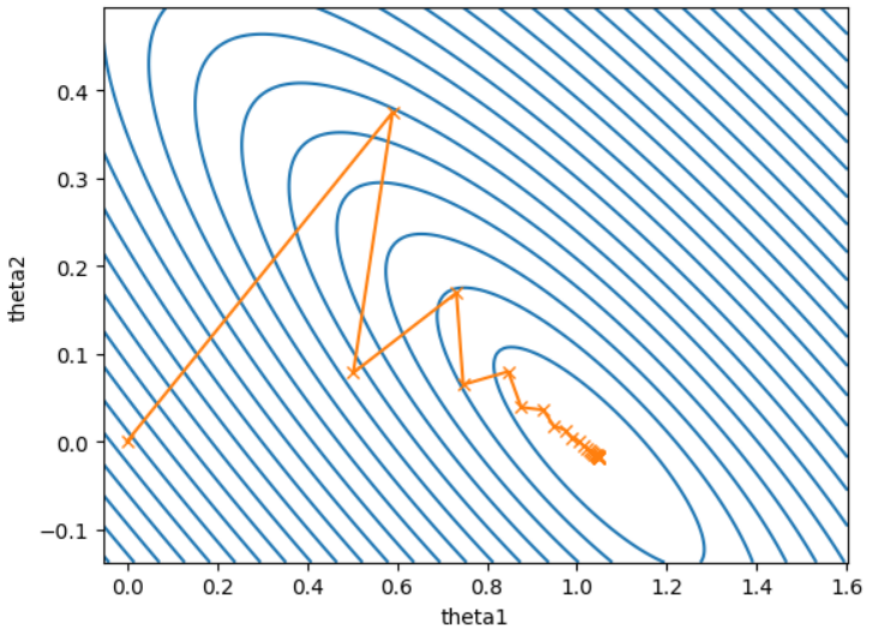
def gradient_descent_params(x, y, iters):
    thetas = []
    theta = np.array([0., 0.])
    alpha = 1.0
    for t in range(iters):
        thetas.append(theta.copy())
        theta[0] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y)*x)
        theta[1] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y) )
    return np.array(thetas)

def err(x, y, theta):
    return np.mean((np.outer(x, theta[:,0]) + theta[:,1] - y[:,None])**2,axis=0)

thetas = gradient_descent_params(x_normalized, y_normalized, 100)
plt.plot(thetas[:,0], thetas[:,1])
xlim, ylim =(np.array(plt.gca().get_xlim()), np.array(plt.gca().get_ylim()))
xlim += np.array([0,0.5])
ylim += np.array([-0.1, 0.1])

XX,YY = np.meshgrid(np.linspace(xlim[0],xlim[1],200), np.linspace(ylim[0],
    ylim[1],200))
ZZ = err(x_normalized.to_numpy(), y_normalized.to_numpy(),
    np.hstack([np.ravel(XX)[:,None], np.ravel(YY)[:,None]])).reshape(XX.shape)
#V = np.logspace(np.log(np.min(ZZ)), np.log(np.max(ZZ)), 30)
V = np.linspace(np.sqrt(np.min(ZZ)), np.sqrt(np.max(ZZ)), 25)**2
plt.clf()
plt.contour(XX,YY,ZZ, V, colors=('C0',))
plt.plot(thetas[:,0], thetas[:,1], 'C1-x')
plt.xlabel("theta1")
plt.ylabel("theta2")
```

Результат:



На практиці градієнтний спуск не завжди використовує всю навчальну вибірку.

Batch Gradient Descent обчислює градієнт, використовуючи всю навчальну вибірку на кожній ітерації. Це забезпечує стабільний напрямок оновлення, але для великих наборів даних може вимагати значних обчислювальних ресурсів.

Stochastic Gradient Descent (SGD) використовує лише один випадково вибраний об'єкт для кожного оновлення параметрів. Тому він працює швидше на великих наборах даних, але напрямок руху є більш шумним.

Mini-Batch Gradient Descent є компромісом між Batch Gradient Descent та SGD. На кожній ітерації для обчислення градієнта використовується не вся вибірка і не один об'єкт, а невелика група об'єктів — mini-batch. Наприклад, навчальна вибірка може містити 10 000 об'єктів, а розмір mini-batch становити 32 або 64.

7.1.1 Лінійна Регресія.

Лінійна регресія — це алгоритм машинного навчання, який моделює залежність між ознаками та числовою цільовою змінною за допомогою лінійної функції. Саме вона описується рівнянням:

$$\hat{y} = \theta_0 + \theta_1 x$$

І саме цю модель ми застосували вище для прогнозування заробітньої плати працівника.

Лінійна регресія використовується для задач регресії, тобто коли потрібно передбачити числове значення. Наприклад, вона може застосовуватися для прогнозування зарплати працівника за досвідом роботи, ціни нерухомості за її характеристиками, обсягу продажів за рекламними витратами або споживання електроенергії за характеристиками об'єкта.

Основним обмеженням лінійної регресії є припущення про лінійну залежність між ознаками та цільовою змінною. Якщо реальна залежність є складною, має нелінійний характер або містить значну кількість взаємодій між ознаками, проста лінійна модель може давати неточні прогнози. У таких випадках можна використовувати поліноміальну регресію, дерева рішень, Random Forest, Gradient Boosting, XGBoost, нейронні мережі та інші нелінійні моделі, про які ми поговоримо у наступних розділах.

Водночас складніша модель не завжди є кращою. Якщо залежність у даних приблизно лінійна, лінійна регресія часто є кращим вибором завдяки простоті, швидкості, інтерпретованості та низькій обчислювальній складності.

Якість регресійної моделі можна оцінювати за допомогою різних метрик регресії. Найпоширенішими є MAE (Mean Absolute Error), MSE (Mean Squared Error), RMSE (Root Mean Squared Error) та R^2. Ми поговоримо про ці метрики детальніше пізніше.

7.1.2 Бібліотека scikit-learn.

Scikit-learn — це популярна Python-бібліотека, яка надає готові реалізації великої кількості алгоритмів машинного навчання. Вона дозволяє використовувати єдиний і зрозумілий інтерфейс для навчання моделей, прогнозування, оцінювання, масштабування даних, вибору ознак і підбору гіперпараметрів. Наприклад, лінійну регресію можна побудувати за допомогою класу LinearRegression, не реалізуючи математичний алгоритм вручну. Це значно скорочує програмний код і дозволяє зосередитися на аналізі даних та якості моделі.

Зробимо те саме лінійне прогнозування, використовуючи бібліотеку scikit-learn. Для того, щоб використати модель лінійної регресії, її спочатку потрібно імпортувати:

```
from sklearn.linear_model import LinearRegression
```

Перетворимо наші дані в масиви numpy (саме з цими масивами і працює scikit-learn) та перетворимо вектор даних у матрицю за допомогою методу `.reshape()`

```
data = pd.read_csv('../resources/Salary_dataset.csv')
X = np.array(data['YearsExperience']).reshape(-1, 1)
y = np.array(data['Salary'])
```

☞ Більшість алгоритмів бібліотеки scikit-learn очікують, що вхідні дані матимуть вигляд `(кількість_об'єктів, кількість_ознак)`. Тобто, навіть якщо ознака лише одна, модель все одно хоче отримати матрицю, а не звичайний вектор. Саме тому і варто застосовувати метод `.reshape()`.

Етап тренування моделі:

```
model = LinearRegression()
model.fit(X, y)
```

Ми також можемо отримати параметри theta:

```
slope = model.coef_[0]    # theta_1
intercept = model.intercept_ # theta_2
print("slope:", slope, "intercept:", intercept)
```

Та, зрештою, побудуємо лінію:

```
X_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_line = model.predict(X_line)

plt.scatter(X, y, c='blue', marker='x')
plt.plot(X_line, y_line) # line
plt.xlabel("X")
plt.ylabel("y")
plt.title("Linear Regression Fit")
```

```
plt.show()
```

Весь код разом:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

data = pd.read_csv('../resources/Salary_dataset.csv')
X = np.array(data['YearsExperience']).reshape(-1, 1)
y = np.array(data['Salary'])

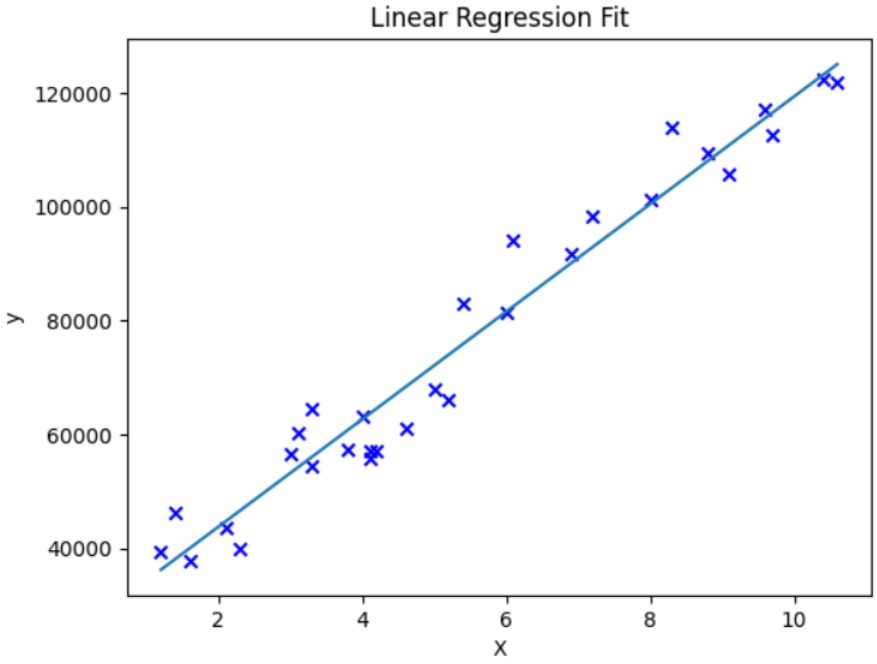
model = LinearRegression()
model.fit(X, y)

slope = model.coef_[0]    # theta_1
intercept = model.intercept_ # theta_2
print("slope:", slope, "intercept:", intercept)

X_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_line = model.predict(X_line)

# plot
plt.scatter(X, y, c='blue', marker='x')
plt.plot(X_line, y_line) # line
plt.xlabel("X")
plt.ylabel("y")
plt.title("Linear Regression Fit")
plt.show()
```

Результат:



7.2 Трохи більше про машинне навчання.

Модель машинного навчання — це математичне представлення закономірностей, які алгоритм знайшов у навчальних даних. Наприклад, після навчання моделі на даних про зарплату та досвід роботи вона може встановити залежність між цими величинами й використовувати її для прогнозування зарплати нового працівника.

Машинне навчання надає можливість автоматично встановлювати прогнозну модель на основі даних.

Термінологія. Функція втрат: $l : y \times y \to \mathrm{R}$, вимірює різницю між прогнозом і фактичним результатом

$$l(y', y) = (y' - y)^2$$

Канонічна задача оптимізації машинного навчання:

$$minimize_\theta \sum_i l(h_\theta(x^i), y^i)$$

Практично кожен алгоритм машинного навчання має таку форму. Для роботи з ним достатньо просто вказати:

- Що таке функція гіпотези?
- Що таке функція втрат?
- Як вирішити задачу оптимізації?

При застосування алгоритмів машинного навчання дані слід розбити на навчальні та тестову. Навчальна вибірка (Training Set) - це набір даних, який використовується для навчання моделі. Тестова вибірка (Test Set) використовується для перевірки якості вже навченої моделі. Під час тестування модель отримує дані, яких вона раніше не бачила. Це дозволяє оцінити, наскільки добре вона здатна узагальнювати знання та працювати з новими прикладами. Ми покажемо як розбивати дані в наступних розділах.

Можна виділити наступні основні етапи виконання алгоритму машинного навчання:

1. Збір даних. Отримання інформації з баз даних, файлів, вебресурсів, датчиків або інших джерел.
2. Попередня обробка даних. Очищення даних від помилок, пропущених значень, дублікатів та аномалій.
3. Виділення ознак. Вибір або створення характеристик, які використовуватимуться моделлю для навчання.
4. Поділ даних. Розбиття даних на навчальну та тестову вибірки.
5. Навчання моделі. Побудова математичної моделі на навчальних даних. Знаходження таких параметрів моделі, за яких вона якомога точніше описує залежність між вхідними ознаками та цільовою змінною (у попередньому прикладі, параметри $\theta_0\theta_1$, які найкраще передбачають заробітню плату працівника).
6. Оцінювання якості. Перевірка точності моделі на тестовій вибірці за допомогою відповідних метрик.
7. Використання моделі. Застосування навченої моделі для прогнозування нових даних - отримання моделлю передбаченого значення для нового об'єкта на основі його вхідних ознак.

7.2.1 Приклади алгоритмів машинного навчання.

Існує велика кількість алгоритмів машинного навчання, які відрізняються типом моделі, способом навчання та припущеннями щодо структури даних. Розглянемо декілька поширених прикладів.

- **Лінійна регресія (Linear Regression).** Модель описує залежність між ознаками та цільовою змінною за допомогою лінійної функції. Параметри моделі зазвичай визначаються шляхом мінімізації середньоквадратичної помилки (MSE). Для задач із невеликою кількістю ознак можливе отримання аналітичного розв'язку.
- **Метод найменших квадратів (Ordinary Least Squares).** Метод найменших квадратів використовується для оцінювання параметрів лінійної регресійної моделі шляхом мінімізації суми квадратів помилок між фактичними та прогнозованими значеннями. Таким чином, це передусім метод оцінювання параметрів, а не окрема модель машинного навчання.
- **Машина опорних векторів (Support Vector Machine, SVM).** SVM шукає гіперплощину, яка розділяє класи з максимально можливим відступом (margin). Для лінійно роздільних даних використовується лінійна модель, а за допомогою ядерних функцій можна будувати нелінійні межі класифікації. У задачі класифікації традиційно використовується шарнірна функція втрат (hinge loss).
- **Нейронна мережа (Neural Network).** Нейронна мережа складається з послідовності шарів, кожен з яких виконує перетворення вхідних даних із використанням вагових коефіцієнтів, зміщень та функцій активації. Завдяки нелінійним функціям активації мережа може моделювати складні нелінійні залежності. Навчання зазвичай виконується за допомогою зворотного поширення помилки та методів градієнтної оптимізації.
- **Дерево рішень (Decision Tree).** Дерево рішень послідовно розділяє простір ознак на області за допомогою умов над окремими ознаками. У результаті формується ієрархічна структура прийняття рішень. Побудова дерева зазвичай ґрунтується на жадібному виборі найкращого розділення на кожному кроці.
- **Наївний Байєсівський класифікатор (Naive Bayes).** Наївний Байєсівський класифікатор використовує теорему Байеса для обчислення ймовірності належності об'єкта до певного класу. Основним припущенням є умовна незалежність ознак за фіксованого класу. Параметри моделі можуть бути оцінені безпосередньо з навчальних даних, що робить алгоритм простим та обчислювально ефективним.

✎ **Питання для самоконтролю.**

1. Що таке машинне навчання?
2. Чим машинне навчання відрізняється від традиційного програмування?
3. Які основні етапи побудови моделі машинного навчання?
4. Що таке навчальна вибірка?
5. Для чого використовується тестова вибірка?
6. Що таке ознака (feature)?
7. Що таке цільова змінна (target)?
8. Які практичні задачі можна розв'язувати за допомогою машинного навчання?

9. Що таке модель машинного навчання?
10. Що означає навчити модель?
11. Що таке прогнозування (prediction)?
12. Що таке лінійна регресія?
13. Для яких задач використовується лінійна регресія?
14. Який вигляд має рівняння простої лінійної регресії?
15. Що означає коефіцієнт нахилу (slope) у рівнянні лінійної регресії?
16. Що означає вільний член (intercept)?
17. Як інтерпретувати коефіцієнти лінійної регресії?
18. Що таке прогнозоване значення моделі?
19. Що таке похибка прогнозування?
20. Чому модель не завжди проходить через усі точки навчальної вибірки?
21. Що таке функція втрат (Loss Function)?
22. Для чого використовується середньоквадратична помилка (MSE)?
23. Як обчислюється середньоквадратична помилка?
24. Чому необхідно мінімізувати функцію втрат?
25. Що таке оптимізація моделі?
26. Що таке градієнтний спуск?
27. Яка основна ідея алгоритму градієнтного спуску?
28. Що таке градієнт функції?
29. Чому під час оптимізації рухаються у напрямку, протилежному градієнту?
30. Що таке швидкість навчання (learning rate)?
31. Як вибір швидкості навчання впливає на процес навчання моделі?
32. Які проблеми можуть виникнути при надто великому значенні learning rate?
33. Які проблеми можуть виникнути при надто малому значенні learning rate?
34. Що таке ітерація градієнтного спуску?
35. Коли алгоритм градієнтного спуску завершує свою роботу?
36. Чим Batch Gradient Descent відрізняється від Stochastic Gradient Descent?
37. Що таке Mini-Batch Gradient Descent?
38. Які переваги використання бібліотеки Scikit-learn для побудови моделей машинного навчання?
39. Яку роль відіграє бібліотека NumPy у задачах машинного навчання?
40. Як оцінити якість побудованої моделі лінійної регресії?

41. Чому важливо розділяти дані на навчальну та тестову вибірки?

42. Які обмеження має лінійна регресія та в яких випадках доцільно використовувати складніші моделі машинного навчання?

Розділ 8

Лінійна Класифікація та Логістична Регресія.

8.1 Лінійна Класифікація.

Доволі поширеною задачею в машинному навчанні є задача класифікації даних. Чи є електронний лист, який ви щойно отримали, спамом? Чи схильний клієнт вашого бізнесу залишити компанію? Чи на фотографії зображена кішка, чи собака? Все це задачі класифікації даних.

Основною відмінністю задачі класифікації від задач регресії, які ми розглядали в попередньому розділі, є тип даних, які прогнозує модель. Якщо в задачі регресії модель прогнозує неперервне числове значення (згадаємо задачу про прогноз заробітньої плати працівника), то модель класифікації прогнозує категоріальний тип даних (спам чи ні, кіт чи собака, та ін.)

- Завдання регресії: прогнозування реальної величини $y \in R$
- Завдання класифікації: прогнозування дискретної величини y
- Бінарна класифікація: $y \in \{-1, 1\}$
- Багатокласова класифікація: $y \in \{1, 2, , ..., k\}$

8.1.1 Приклад лінійної класифікації: класифікація раку молочної залози

Відомий приклад класифікації: використання машинного навчання для діагностики того, чи є пухлина молочної залози доброякісною чи злоякісною.

Умови: лікар бере зразок рідини з пухлини, фарбує клітини, а потім окреслює декілька клітин (обробка зображення уточнює контури)

Система обчислює характеристики кожної клітини, такі як площа, периметр, увігнутість, текстура (всього 10); обчислює середнє/стандартне/максимальне значення для всіх характеристик.

```
import numpy as np
import sklearn.datasets
import matplotlib.pyplot as plt
dat = sklearn.datasets.load_breast_cancer()

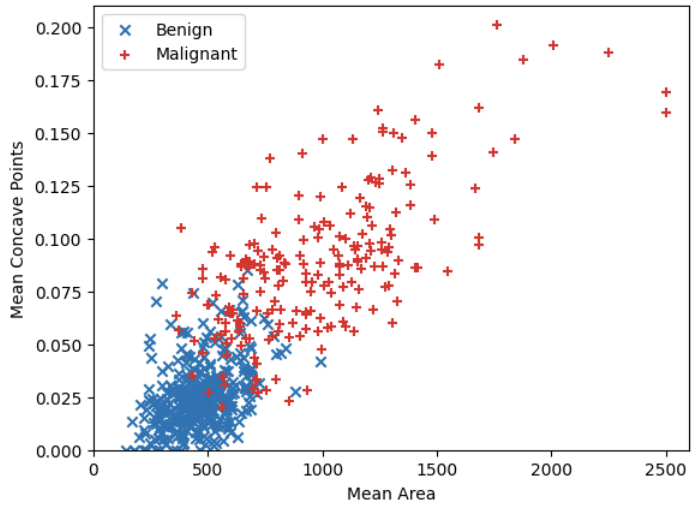
ix = np.where(dat["feature_names"] == "mean area")[0][0]
iy = np.where(dat["feature_names"] == "mean concave points")[0][0]

def plot_cells():
```

```
    plt.scatter(dat["data"][:,ix][dat["target"]==1],
       dat["data"][:,iy][dat["target"]==1], marker='x', color='C0')
    plt.scatter(dat["data"][:,ix][dat["target"]==0],
       dat["data"][:,iy][dat["target"]==0], marker='+', color='C3')
    plt.xlim([0,2600])
    plt.ylim([0,0.21])
    plt.xlabel("Mean Area")
    plt.ylabel("Mean Concave Points")
    plt.legend(['Benign', 'Malignant'])

plot_cells()
```

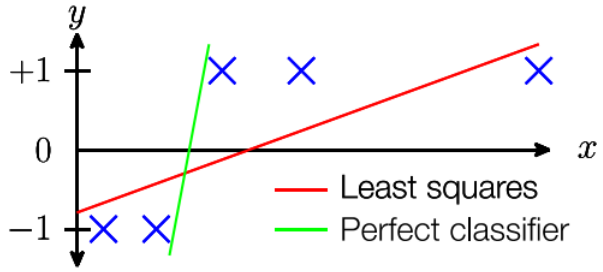
Результат:



Задача лінійної класифікації = "Провести лінію, що розділить класи"

8.1.2 Функція втрат.

Як визначити функцію втрат (Loss function) $l : R \times \{-1, 1\} \to R$? Квадратична втрата є поганою ідеєю:



Ідеальна функція втрат виглядала б наступним чином:

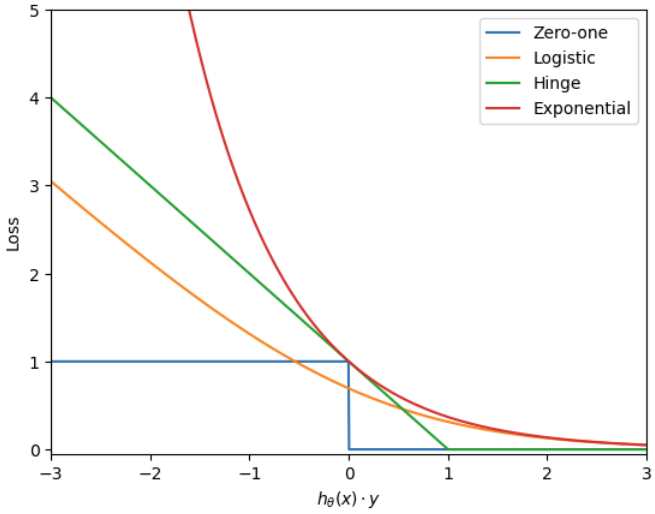
$$l(h_\theta(x), y) = \begin{cases} 0, & \text{if } sign(h_\theta(x)) = y \\ 1, & \text{otherwise} \end{cases}$$

Приклади функцій втрат: logistic, hinge, exponential.

```
hy = np.linspace(-3,3,1000)
plt.plot(hy,(hy<=0))
```

```
plt.plot(hy, np.log(1+np.exp(-hy)))
plt.plot(hy, np.maximum(1 - hy, 0))
plt.plot(hy, np.exp(-hy))
plt.xlim([-3,3])
plt.ylim([-0.05, 5])
plt.ylabel("Loss")
plt.xlabel("$h_theta(x) \cdot y$")
plt.legend(['Zero-one', 'Logistic', 'Hinge', 'Exponential'])
```

Результат:



8.2 Логістична регресія.

Логістична регресія (Logistic regression) характерна вибором наступної функції втрат:

$$l_{logistic}(h_\theta(x), y) = log(1 + exp(-h_\theta(x) \cdot y))$$

Логістична регресія також має гарну ймовірнісну інтерпретацію: певні величини дають ймовірність, за певною моделлю, того, що приклад буде позитивним або негативним.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
   precision_score, recall_score, f1_score, classification_report

# load data
data = load_breast_cancer()
X = data.data[:, [3, 25]] # mean area, mean concave points
y = data.target # 0 means malignant tumor, 1 mean benign tumor
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# fit
clf = LogisticRegression()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

# decision boundary grid
xx, yy = np.meshgrid(
    np.linspace(X_test[:,0].min()-1, X_test[:,0].max()+1, 300),
    np.linspace(X_test[:,1].min()-1, X_test[:,1].max()+1, 300)
)
grid = np.c_[xx.ravel(), yy.ravel()]
Z = clf.predict(grid).reshape(xx.shape)

# true labels + decision boundary
plt.contourf(xx, yy, Z, alpha=0.2, levels=[-1,0,1], colors=['C3','C0'])
# plt.scatter(X_test[:,0], X_test[:,1], c=y_test)
for cls, marker, color in [(0, '+', 'red'), (1, 'x', 'blue')]:
    idx = (y_test == cls)
    plt.scatter(
        X_test[idx, 0],
        X_test[idx, 1],
        marker=marker,
        color=color,
        label=f"Class {cls}"
    )
plt.title("True labels + decision boundary")
plt.xlabel("mean area (scaled)")
plt.ylabel("mean concave points (scaled)")
plt.show()

# confusion matrix + metrics
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()

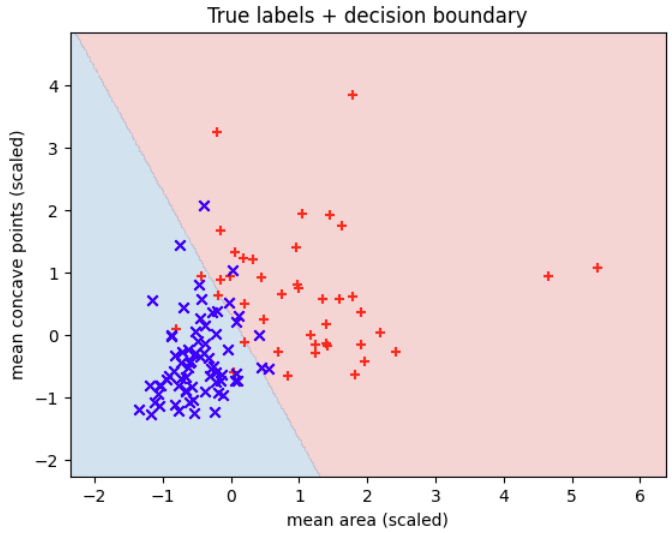
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

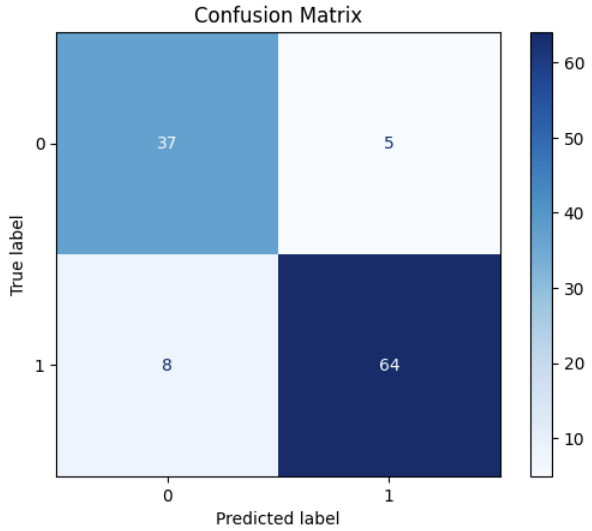
print("Precision:", precision)
print("Recall:", recall)
```

```
print("F1:", f1)

print()
print(classification_report(y_test, y_pred))
```

Результат:






```
Precision: 0.927536231884058
Recall: 0.8888888888888888
F1: 0.9078014184397163

              precision    recall  f1-score   support

           0       0.82      0.88      0.85        42
           1       0.93      0.89      0.91        72

    accuracy                           0.89       114
   macro avg       0.87      0.88      0.88       114
weighted avg       0.89      0.89      0.89       114
```

8.2.1 Метрики моделей машинного навчання.

При роботі з моделяма машинного навчання варто вміти оцінити, наскільки якісно та чи інша модель справляється з задачами, які перед нею ставляться. Як це зробити? Логічно подивитися на відношення даних, які вона класифікує правильно, до кількості неправильно класифікованих даних.

Матриця невідповідностей (Confusion matrix): Це матриця розміром 2×2 для бінарної класифікації з фактичними значеннями на одній осі та прогнозованими на іншій.

Істинно позитивні (True Positive, TP) – модель правильно прогнозує позитивний клас (прогноз і фактичне значення є позитивними).

Істинно негативні (True Negative, TN) – модель правильно прогнозує негативний клас (прогноз і фактичний результат є негативними).

Хибно позитивні (False Positive, FP) – модель дає неправильний прогноз негативного класу (прогноз – позитивний, фактичний результат – негативний). (Type 1 error)

Хибно негативні (False Negative, FN) – модель дає неправильний прогноз щодо позитивного класу (прогноз – негативний, фактичний результат – позитивний). (Type 2 error)

Табл. 8.1: Матриця помилок (Confusion Matrix)

	Передбачувані позитивні	**Передбачувані Негативні**
Справді Позитивні	True Positive (TP)	False Negative (FN)
Справді Негативні	False Positive (FP)	True Negative (TN)

Влучність (Precision): З усіх позитивних прогнозів, який відсоток є дійсно позитивним?

$$P = \frac{TP}{TP + FP}$$

Повнота (Recall): З загальної кількості позитивних результатів, який відсоток є прогнозованими позитивними.

$$R = \frac{TP}{TP + FN}$$

F1-міра (F1 Score): Це гармонійне середнє значення влучності та повноти. Воно враховує як помилкові позитивні, так і помилкові негативні результати. Тому воно добре працює з незбалансованими наборами даних.

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}}$$

✎ **Питання для самоконтролю.**

1. Що таке класифікація в машинному навчанні?
2. Чим задачі класифікації відрізняються від задач регресії?
3. Які приклади задач класифікації можна зустріти в інтелектуальному аналізі даних?
4. Що таке лінійна класифікація?
5. Яка основна ідея лінійного класифікатора?
6. Що таке гіперплощина класифікації?
7. Як рівняння прямої використовується для поділу двох класів?

8. Що таке вагові коефіцієнти моделі класифікації?
9. Яку роль відіграє зміщення (bias) у лінійній класифікації?
10. Як інтерпретувати знак лінійної функції класифікації?
11. Які переваги має лінійна класифікація?
12. Які обмеження мають лінійні класифікатори?
13. Чому не всі задачі можна розв'язати за допомогою лінійної класифікації?
14. Як можна візуалізувати межу прийняття рішення (decision boundary)?
15. Які ознаки використовуються для класифікації раку молочної залози?
16. Чому задача класифікації раку молочної залози є бінарною?
17. Що означає правильна класифікація пацієнта?
18. Чому точність класифікації є критично важливою в медичних задачах?
19. Що таке функція втрат (Loss Function)?
20. Для чого використовується функція втрат під час навчання моделі?
21. Чому модель прагне мінімізувати значення функції втрат?
22. Чим функція втрат відрізняється від метрики якості моделі?
23. Що таке логістична регресія?
24. Чому логістична регресія, незважаючи на назву, використовується для задач класифікації?
25. Яку роль відіграє сигмоїдна (логістична) функція?
26. Які значення може приймати сигмоїдна функція?
27. Як інтерпретувати результат логістичної регресії як ймовірність належності до класу?
28. Як виконується остаточна класифікація на основі отриманої ймовірності?
29. Що таке поріг класифікації (classification threshold)?
30. Як зміна порога класифікації впливає на результати моделі?
31. Що таке матриця невідповідностей (Confusion Matrix)?
32. Які чотири типи прогнозів містить матриця невідповідностей?
33. Що означають True Positive (TP), True Negative (TN), False Positive (FP) та False Negative (FN)?
34. Що таке влучність (Precision) і як вона обчислюється?
35. Що таке повнота (Recall) і як вона обчислюється?
36. У яких випадках важливіше максимізувати Precision, а в яких — Recall?
37. Що таке F1-міра та чому вона використовується?
38. Чому однієї лише точності (Accuracy) часто недостатньо для оцінювання моделі?

39. Як інтерпретувати результати моделі за допомогою матриці невідповідностей і метрик Precision, Recall та F1?

40. Які практичні рекомендації можна дати щодо вибору метрик оцінювання для різних задач класифікації?

Розділ 9

Нелінійні Алгоритми Машинного Навчання.

До цього моменту ми ілюстрували лінійну регресію як «проведення лінії через дані», але насправді це було функцією наших вхідних характеристик.

Хоча це може здаватися обмеженим, алгоритми лінійної регресії є досить потужними, коли застосовуються до нелінійних характеристик вхідних даних.

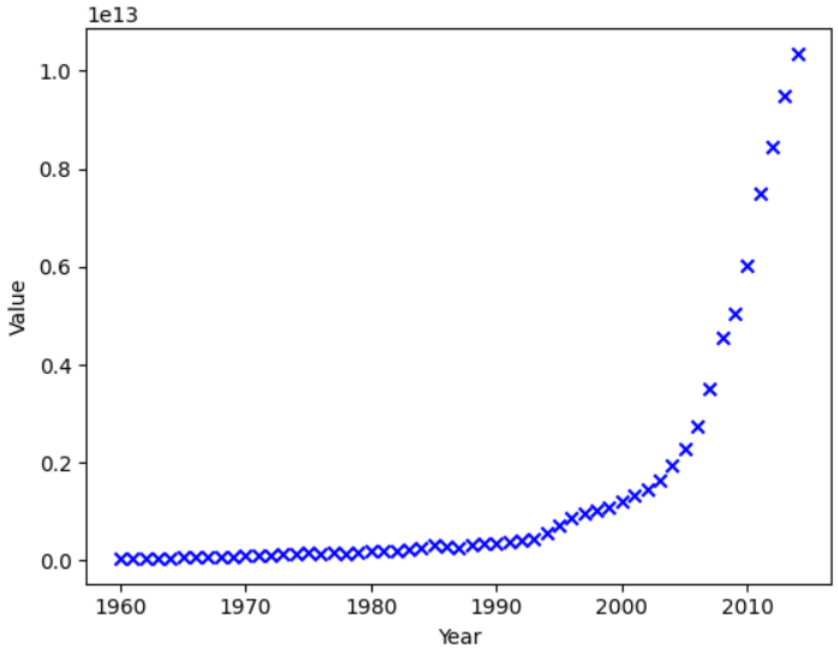
Розглянемо, для прикладу, дані про ріст ВВП Китаю за останні 60 років, і спробуємо побудувати модель, що дає найкраще передбачення.

```
import matplotlib.pyplot as plt
import pandas as pd

def build_plot(file_path, x, y):
    data = pd.read_csv(file_path)
    plt.scatter(data[x], data[y], c='blue', marker='x')
    plt.xlabel(x)
    plt.ylabel(y)
    return plt
    # plt.show()

build_plot('../resources/china_gdp.csv', 'Year', 'Value').show()
```

Результат:



Якщо спробувати провести лінію через цей графік, отримаємо наступне:

```
plt = build_plot('../resources/china_gdp.csv', 'Year', 'Value')
data = pd.read_csv('../resources/china_gdp.csv')
x, y = data['Year'], data['Value']
x_reshaped, y_reshaped = x.values.reshape(-1, 1), y.values.reshape(-1, 1)

from sklearn.linear_model import LinearRegression
import numpy as np

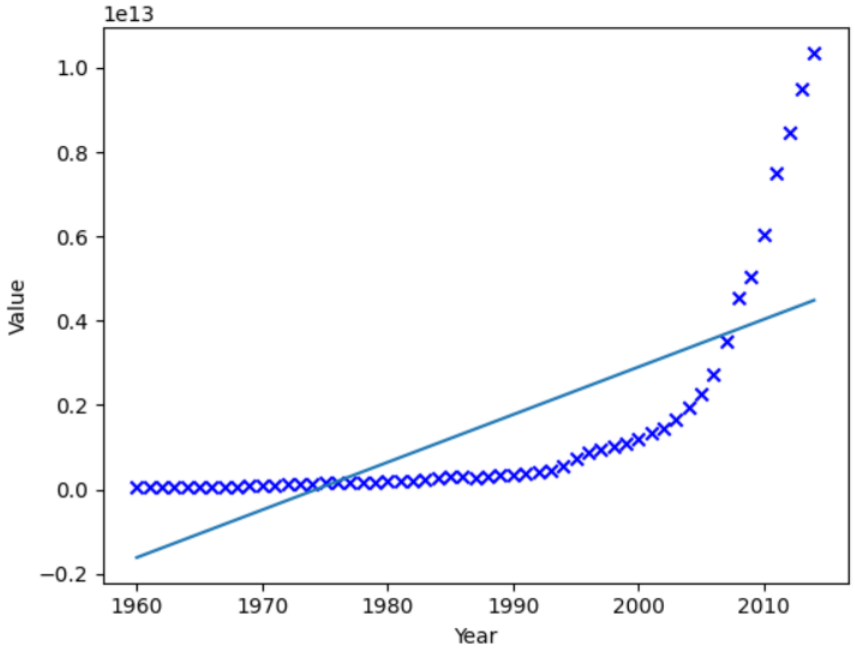
model = LinearRegression(fit_intercept=True)
model.fit(x_reshaped, y_reshaped)
Xnew = np.array([[2025], [2030]])
print(model.predict(Xnew))

theta = [ model.coef_[0], model.intercept_ ]
print(theta)

X_line = np.linspace(x_reshaped.min(), x_reshaped.max(), 100).reshape(-1, 1)
y_line = model.predict(X_line)

plt.plot(X_line, y_line)
```

Результат:



Очевидно, що лінійна модель не найкращим чином наближає ці дані.

Нелінійна модель машинного навчання — це модель, яка здатна описувати складні, нелінійні залежності між вхідними ознаками та цільовою змінною. На відміну від лінійних моделей, вона не обмежується прямою або площиною як межею прогнозування, а може будувати криволінійні поверхні, що точніше відображають закономірності реальних даних.

9.1 Поліноміальна Регресія.

Поліноміальна регресія — це різновид лінійної регресії, в якому взаємозв'язок між x та y моделюється у вигляді полінома n-го ступеня:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + ... + \beta_n x^n + \epsilon$$

Це дозволяє моделювати нелінійні залежності шляхом додавання поліноміальних членів ознак.

Щоб використати поліноміальну регресію, достатньо імпортувати відповідний модуль:

```
from sklearn.preprocessing import PolynomialFeatures
```

Та трансформувати вхідні ознаки в поліноміальні:

```
poly = PolynomialFeatures(degree=5)
X_poly = poly.fit_transform(x_reshaped)
```

Разом код виглядає наступним чином:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

plt = build_plot('../resources/china_gdp.csv', 'Year', 'Value')
data = pd.read_csv('../resources/china_gdp.csv')
x, y = data['Year'], data['Value']
```

```
x_reshaped, y_reshaped = x.values.reshape(-1, 1), y.values.reshape(-1, 1)

poly = PolynomialFeatures(degree=5)
X_poly = poly.fit_transform(x_reshaped)

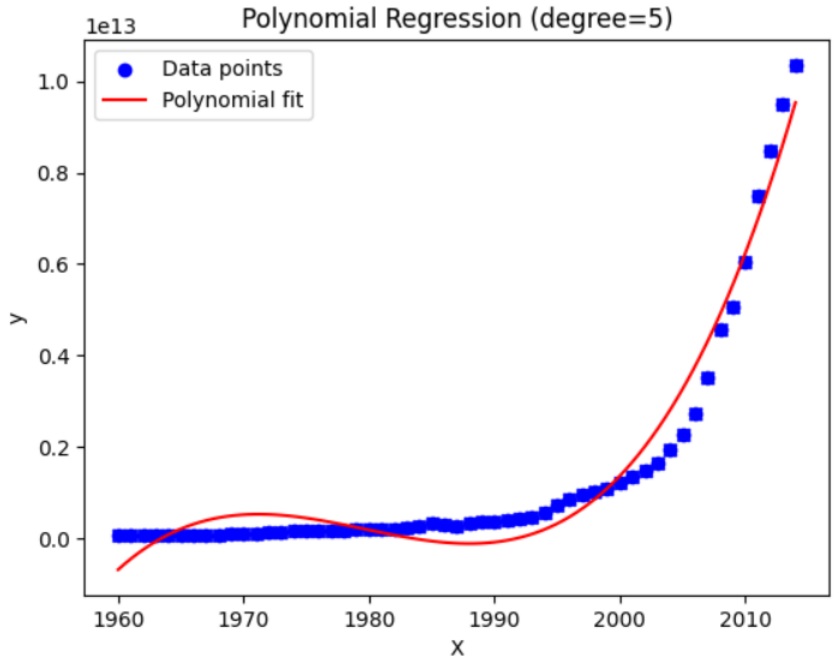
# model fit:
model = LinearRegression()
model.fit(X_poly, y_reshaped)

# forecast and visualization:
X_plot = np.linspace(min(x_reshaped), max(x_reshaped), 100).reshape(-1, 1)
y_pred = model.predict(poly.transform(X_plot))

plt.scatter(x, y, color='blue', label='Data points')
plt.plot(X_plot, y_pred, color='red', label='Polynomial fit')
plt.xlabel("X")
plt.ylabel("y")
plt.title("Polynomial Regression (degree=5)")
plt.legend()
plt.show()

# coefficients
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```

Результат:




```
Intercept: [-2.1028827e+17]
Coefficients: [[ 0.00000000e+00  1.20746941e+03  2.72901110e+05  2.71121389e+08
  -2.05442021e+05  4.15121802e+01]]
```

9.1.1 Узагальнення та перенавчання.

Проблема з канонічною задачею машинного навчання полягає в тому, що нас насправді не цікавить мінімізація цієї цілі на заданому наборі даних.

Нас насправді цікавить, наскільки добре наша функція узагальниться на нових прикладах, які ми не використовували для навчання системи (але які взяті з «того самого розподілу», що й приклади, які ми використовували для навчання)

Поліноми вищого ступеня демонстрували перенавчання: вони фактично мають дуже низькі втрати на даних навчання, але створюють функції, які, як ми очікуємо, не будуть добре узагальнюватися.

У міру ускладнення моделі втрати навчання завжди зменшуються; втрати узагальнення зменшуються до певної точки, а потім починають збільшуватися

Хоча важко кількісно оцінити справжню помилку узагальнення (тобто помилку цих алгоритмів щодо повного розподілу можливих прикладів), ми можемо наблизити її за допомогою *перехресної валідації*.

Основна ідея полягає в тому, щоб розділити набір даних на навчальний набір і набір для перевірки. Навчіть алгоритм на навчальному наборі і оцініть на наборі для перевірки.

Ми називаємо змінні θ параметрами алгоритму машинного навчання. Але є й інші величини, які також впливають на класифікатор: ступінь полінома, кількість регуляризації тощо; їх сукупно називають гіперпараметрами алгоритму.

Основна ідея перехресного валідації: використовувати навчальний набір для визначення параметрів, використовувати резервний набір для визначення гіперпараметрів.

9.1.2 k-кратна перехресна валідація.

k-кратна перехресна валідація (k-folds validation) - це більш складна (але насправді дещо більш поширена) версія перехресної валідації. Розділіть набір даних на k роз'єднаних підмножин (складок); навчіть на $k-1$ і оцініть на залишковій складці; повторіть k разів, утримуючи кожну складку один раз. Повідомте середню похибку для всіх утриманих складок.

Варіанти.

- Перехресна валідація «залиш один»: обмеження k-кратної перехресної валідації, де кожна складка є лише одним прикладом (тож ми навчаємося на всіх інших прикладах, тестуючи цей один приклад)

  Дещо дивно, але для найменших квадратів це можна обчислити ефективніше, ніж k-кратну перехресну валідацію, з такою ж складністю вирішення для оптимального θ за допомогою матричного рівняння.

- Стратифікована перехресна валідація: збереження приблизно рівного відсотка позитивних/негативних прикладів (або будь-якої іншої ознаки) у кожному згині.

☞ Попередження: k-кратна перехресна валідація не завжди є кращою (наприклад, у прогнозуванні часових рядів бажано, щоб набір утриманих даних був повністю сформований після набору навчання)

9.2 Поліноміальна класифікація.

Наведемо приклад поліномільної класифікації на тому ж датасеті пухлин молочної залози.

Завантажуємо датасет (лише дві ознаки для класифікації: середній радіус, середній периметр)

```
data = load_breast_cancer()
X = data.data[:, [3, 25]]
y = data.target
```

Розбиття на тренувальні та тестові дані:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=10)
```

Скейлимо дані:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Додаємо поліноміальні ознаки (нелінійна трансформація):

```
poly = PolynomialFeatures(degree=10)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)
```

Тренуємо логістичну регресію на поліноміальних ознаках:

```
clf = LogisticRegression(max_iter=10000)
clf.fit(X_train_poly, y_train)
```

Евалюємо:

```
y_pred = clf.predict(X_test_poly)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Візуалізація (Межа між класами):

```
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))

Z = clf.predict(poly.transform(np.c_[xx.ravel(), yy.ravel()]))
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.2, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1], c=y_test, cmap='coolwarm',
    edgecolors='k')
plt.xlabel('mean radius (scaled)')
plt.ylabel('mean perimeter (scaled)')
plt.title('Non-linear Classification with Polynomial Logistic Regression')
plt.show()
```

Матриця похибок (confusion matrix):

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
```

```
plt.title("Confusion Matrix")
plt.show()
```

Все разом:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

data = load_breast_cancer()
X = data.data[:, [3, 25]]
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

poly = PolynomialFeatures(degree=10)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

clf = LogisticRegression(max_iter=10000)
clf.fit(X_train_poly, y_train)

y_pred = clf.predict(X_test_poly)
print("Accuracy:", accuracy_score(y_test, y_pred))

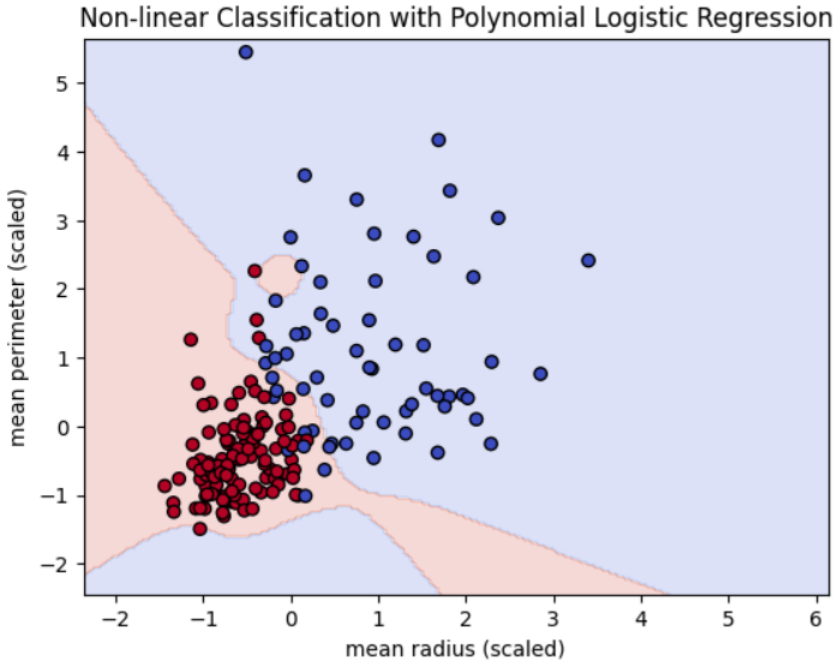
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))

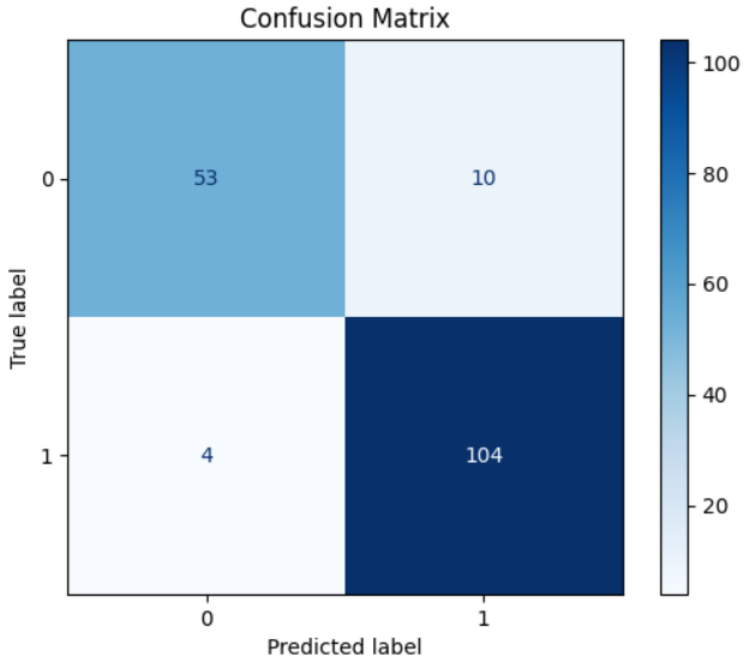
Z = clf.predict(poly.transform(np.c_[xx.ravel(), yy.ravel()]))
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.2, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1], c=y_test, cmap='coolwarm',
    edgecolors='k')
plt.xlabel('mean radius (scaled)')
plt.ylabel('mean perimeter (scaled)')
plt.title('Non-linear Classification with Polynomial Logistic Regression')
plt.show()
```

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```

Результат:





9.3 Метод Опорних Векторів.

Метод опорних векторів (Support Vector Machine) — це алгоритм класифікації, який шукає оптимальну гіперплощину, що розділяє дані двох класів із максимальним зазором (margin).

Нехай ми маємо навчальну вибірку:

$$(x_i, y_i)_{i=1}^n, \quad x_i \in \mathbb{R}^d, \quad y_i \in \{-1, +1\}$$

Ми шукаємо гіперплощину, яка має вигляд:

$$w^T x + b = 0$$

де w — вектор ваг (нормаль до гіперплощини), а b — зсув (bias).

Умова коректної класифікації: Для всіх навчальних точок повинно виконуватись:

$$y_i(w^T x_i + b) \geq 1$$

Це означає, що всі точки знаходяться по правильний бік гіперплощини, причому на відстані не менше 1 від неї.

Відстань (margin): Відстань від точки до гіперплощини дорівнює:

$$\frac{|w^T x + b|}{|w|}$$

Ми хочемо максимізувати цей зазор між класами. Для двох крайніх точок зазор дорівнює:

$$\text{margin} = \frac{2}{|w|}$$

Отже, щоб його максимізувати, потрібно мінімізувати $|w|^2$.

Оптимізаційна задача (лінійно роздільний випадок):

$$\min_{w,b} \quad \frac{1}{2}|w|^2 \text{ за умови:} \qquad y_i(w^T x_i + b) \geq 1, \quad i = 1, \ldots, n$$

Це опукла квадратична задача оптимізації. У реальних даних можливі помилки класифікації, тому вводять змінні послаблення $\xi_i \geq 0$:

$$\min_{w,b,\xi} \quad \frac{1}{2}|w|^2 + C\sum_{i=1}^n \xi_i \text{ за умови:} \qquad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n$$

де $C > 0$ — параметр, що задає компроміс між шириною зазору та кількістю помилок.

Ядровий трюк (kernel trick): Уявимо собі, що перед нами стоїть така задача: потрібно провести класифікацію температури на комфортну та некомфортну для людини. Якщо для людини комфортною температурою є 20 градусів за Цельсієм, то некомфортною є як -10 так і 40 градусів. Провести таку гіперплощину через ці дані, щоб всі точки з комфортною температурою були з однієї сторони, є неможливим.

Для того, щоб провести гіперплощину в цьому випадку, ми можемо застосувати перетворення $y(x) = (x - 20)^2$. Тоді всі некомфортні температури переходять у значення, більші за якесь a, тоді як всі комфортні температури відображаються у точки $x \leq a$. Це і є ядровий трюк.

Щоб працювати з нелінійно роздільними даними, SVM використовує перетворення простору:

$$\phi : \mathbb{R}^d \to \mathbb{R}^m$$

але замість явного обчислення $\phi(x)$ застосовується ядро $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$.

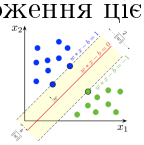
Типові ядра, які використовуються для методу опорних векторів: лінійне ($K(x, x') = x^T x'$), поліноміальне ($K(x, x') = (x^T x' + 1)^d$) та радіально-базисне (RBF, $K(x, x') = \exp(-\gamma|x - x'|^2)$ - це ядро використовується найчастіше).

Рішення у вигляді опорних векторів: у результаті оптимізації маємо:

$$f(x) = \text{sign}\left(\sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b\right)$$

де α_i — коефіцієнти лагранжа. Точки, для яких $\alpha_i > 0$, називаються опорними векторами — саме вони визначають гіперплощину.

☞ Геометрична інтерпретація методу опорних векторів: SVM шукає межу, яка найбільше віддалена від найближчих точок обох класів. Тільки кілька опорних векторів визначають положення цієї межі.



Модуль для класифікації методом опорних векторів імпортується з пакету sklearn:

```
from sklearn.svm import SVC
```

Тип ядра двказується через параметр `kernel`:

```
clf = SVC(kernel='rbf')
clf.fit(X_train_scaled, y_train)
```

Повністю код методу опорних векторів для задачі класифікації на датасеті пухлин молочної залози:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

data = load_breast_cancer()
X = data.data[:, [3, 25]]
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

clf = SVC(kernel='rbf')
clf.fit(X_train_scaled, y_train)

y_pred = clf.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))

x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
```

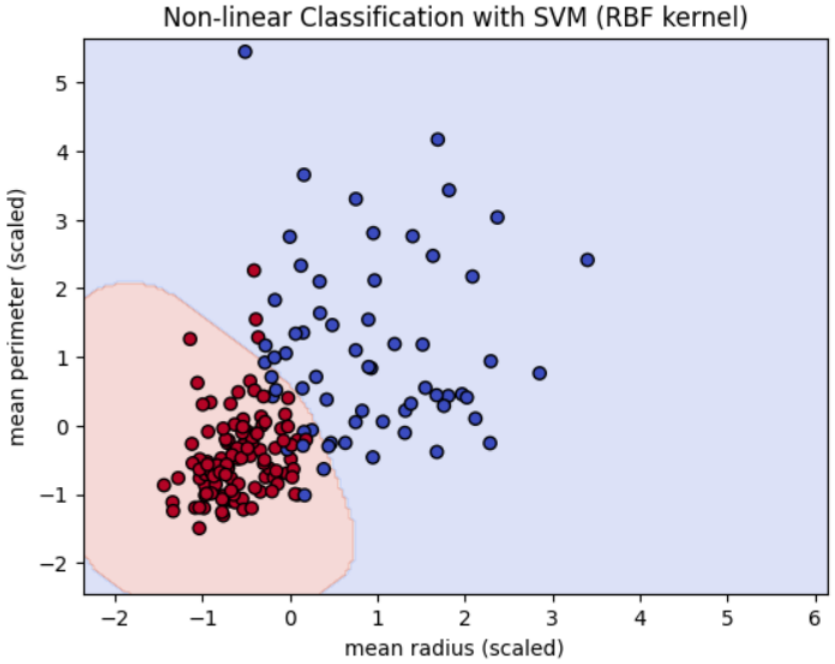
```
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))

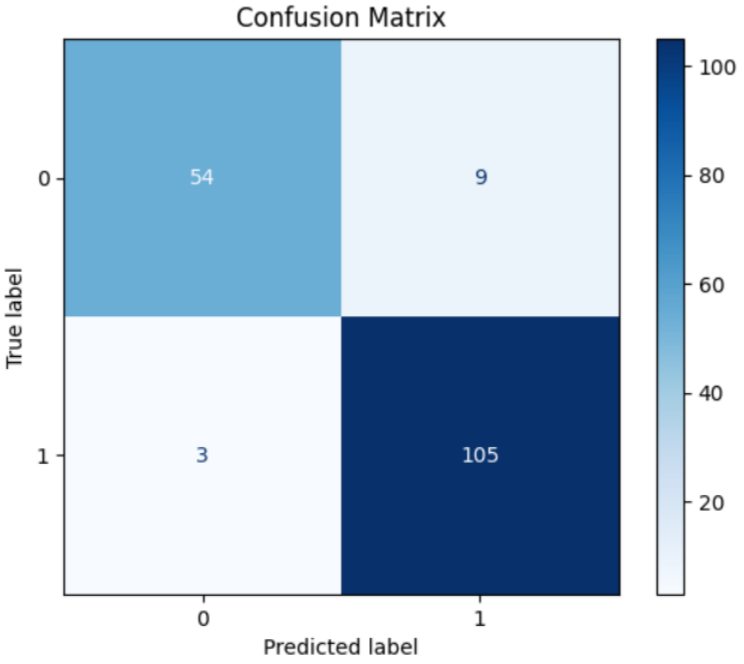
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.2, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1], c=y_test, cmap='coolwarm',
    edgecolors='k')
plt.xlabel('mean radius (scaled)')
plt.ylabel('mean perimeter (scaled)')
plt.title('Non-linear Classification with SVM (RBF kernel)')
plt.show()

ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```

Результат:





9.4 Наївний баєсів класифікатор.

Наївний баєсівський класифікатор (Naive Bayes classifier) — це ймовірнісний метод класифікації, який базується на теоремі Байєса. Він оцінює, до якого класу належить об'єкт x, обчислюючи ймовірність $P(C_k|x)$ для кожного класу C_k.

Теорема Байєса: основна формула

$$P(C_k|x) = \frac{P(x|C_k), P(C_k)}{P(x)}$$

де $P(C_k|x)$ — апостеріорна ймовірність класу після спостереження ознак, $P(x|C_k)$ — правдоподібність (як імовірно отримати ознаки, якщо відомий клас), $P(C_k)$ — апріорна ймовірність класу, $P(x)$ — нормувальний коефіцієнт (спільна ймовірність ознак).

при цьому методі машинного навчання, ми робимо так зване *"наївне"припущення незалежності*: методологічних вважає, що всі ознаки $x_1, x_2, \ldots, x_n$ незалежні одна від одної, якщо відомо, до якого класу належить об'єкт.

$$P(x|C_k) = \prod_{j=1}^{n} P(x_j|C_k)$$

Це "наївне"припущення значно спрощує обчислення, навіть якщо насправді ознаки частково залежать.

Формула класифікації: Підставимо у формулу Байєса:

$$P(C_k|x) \propto P(C_k) \prod_{j=1}^{n} P(x_j|C_k)$$

Оскільки $P(x)$ однакова для всіх класів, її можна не враховувати при порівнянні.

Отже, ми вибираємо клас з максимальною апостеріорною ймовірністю:

$$\hat{C} = \arg\max_{C_k}; P(C_k) \prod_{j=1}^{n} P(x_j|C_k)$$

Наведемо приклад використання наївного баєсівого класифікатора для задачі класифікації пухлин:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

data = load_breast_cancer()
X = data.data[:, [3, 25]]
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

clf = GaussianNB()
clf.fit(X_train_scaled, y_train)

y_pred = clf.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))

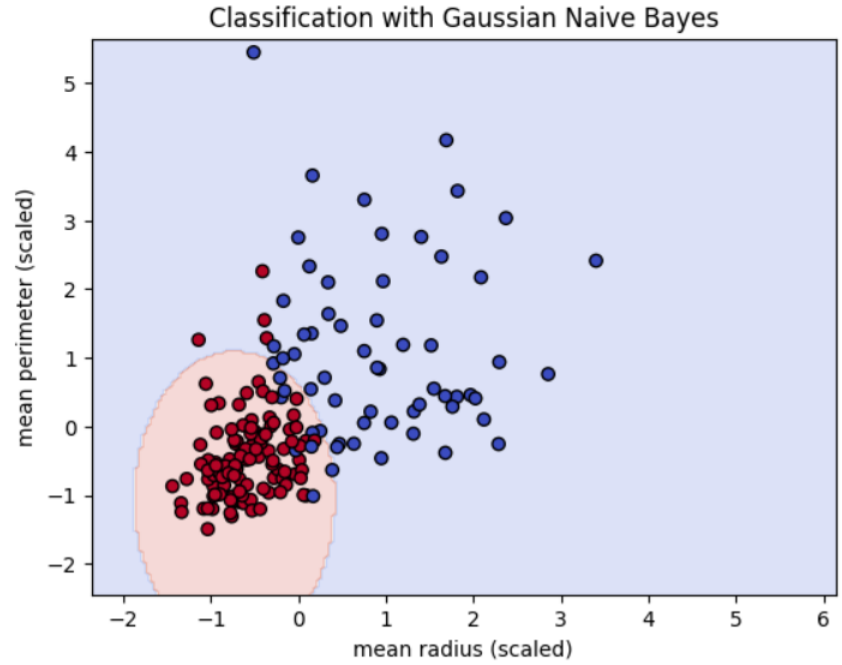
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 200),
    np.linspace(y_min, y_max, 200)
)

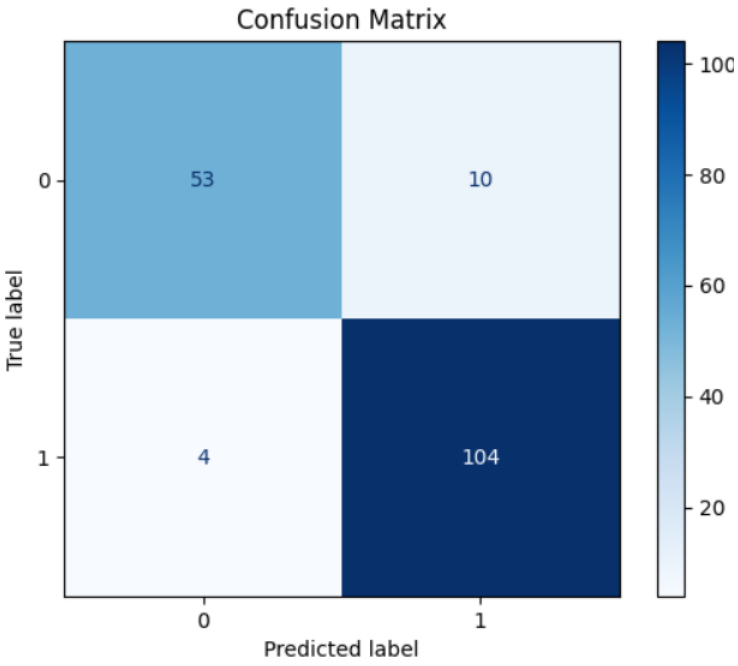
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.2, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1], c=y_test, cmap='coolwarm',
   edgecolors='k')
plt.xlabel('mean radius (scaled)')
plt.ylabel('mean perimeter (scaled)')
plt.title('Classification with Gaussian Naive Bayes')
plt.show()
```

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```

Результат:





9.5 Дерева прийняття рішень.

Алгоритм дерева прийняття рішень (Decision trees) використовується для класифікації або регресії. Його ідея — розділяти простір ознак на підмножини так, щоб отримати групи об'єктів,

максимально однорідні за цільовою змінною.

Дерево прийняття рішень — це ієрархічна структура, де вузол (node) містить умову розділення (наприклад, $x_i < t$), гілки (branches) це варіанти переходу (так/ні), а листки (leaves) — фінальні рішення (клас або числове значення)

Нехай ми маємо навчальну вибірку:

$$(x_i, y_i)_{i=1}^{n}, \quad x_i \in \mathbb{R}^d, ; y_i \in 1, 2, \ldots, K$$

Наша мета — знайти функцію $f(x)$, яка розбиває простір ознак на області R_m, і кожній області відповідає передбачення класу:

$$f(x) = \sum_{m=1}^{M} c_m \cdot I(x \in R_m)$$

де R_m — регіон (листок дерева), c_m — значення або клас, який передбачається для цього регіону, $I(\cdot)$ — індикаторна функція (1, якщо умова виконується).

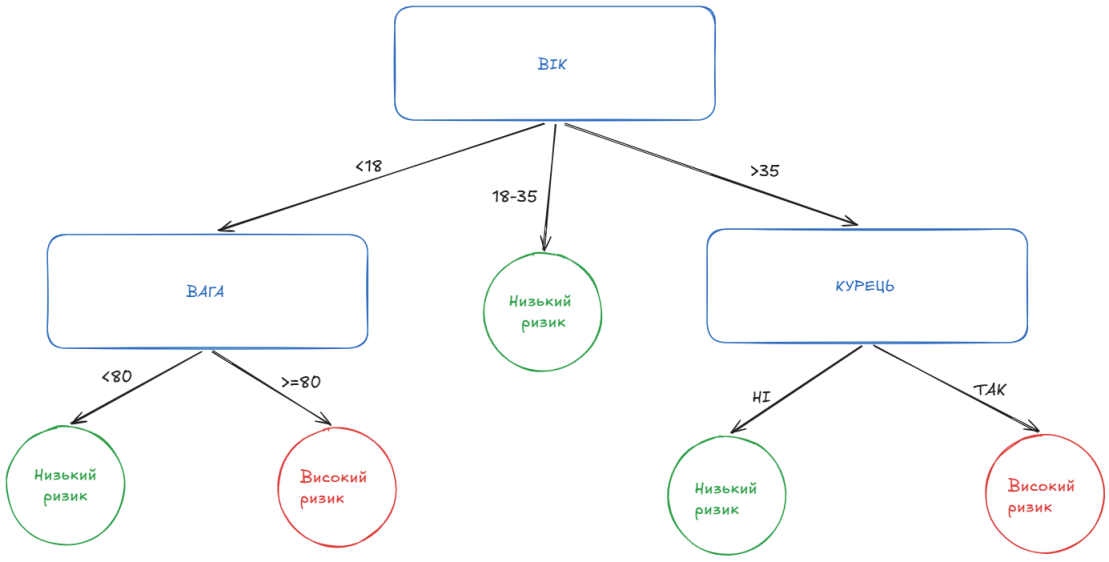
На кожному кроці вибирається ознака x_j та поріг t, які найкраще поділяють дані на дві частини:

$$R_1(j, t) = x \mid x_j \leq t, \quad R_2(j, t) = x \mid x_j > t$$

Ми вибираємо такі j, t, що мінімізують певну функцію втрат L.

Тут можна навести приклад із класифікацією пацієнтів за ризиком серцевих захворювань. Нехай перед нами стоїть задача класифікувати пацієнтів таким чином. Тут логічно будувати дерево, в якому на вузлах пацієнти будуть розділятися за віком (вік, більший за певне число - підвищений ризик), вагою (вага, більша за певне число - також ризик захворювань), та схильністю до куріння. Питання лише в підборі конкретних чисел (гранична вага та вік) таким чином, щоб мінімізувати ентропію (пацієнти, що будуть неправильно класифіковані).

Як бачимо, дерево прийняття рішень є цілком інтуїтивно зрозумілим алгоритмом, а значною перевагою цієї моделі є її інтерпретованість.



Приклад з використанням sklearn:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split

data = load_breast_cancer()

X = data.data[:, [3, 25]]
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)

clf = DecisionTreeClassifier(max_depth=5, criterion='gini', random_state=42)
clf.fit(X_train, y_train)

print("Accuracy on test set:", clf.score(X_test, y_test))

x_min, x_max = X[:, 0].min() - 0.01, X[:, 0].max() + 0.01
y_min, y_max = X[:, 1].min() - 0.01, X[:, 1].max() + 0.01
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                     np.linspace(y_min, y_max, 300))

Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

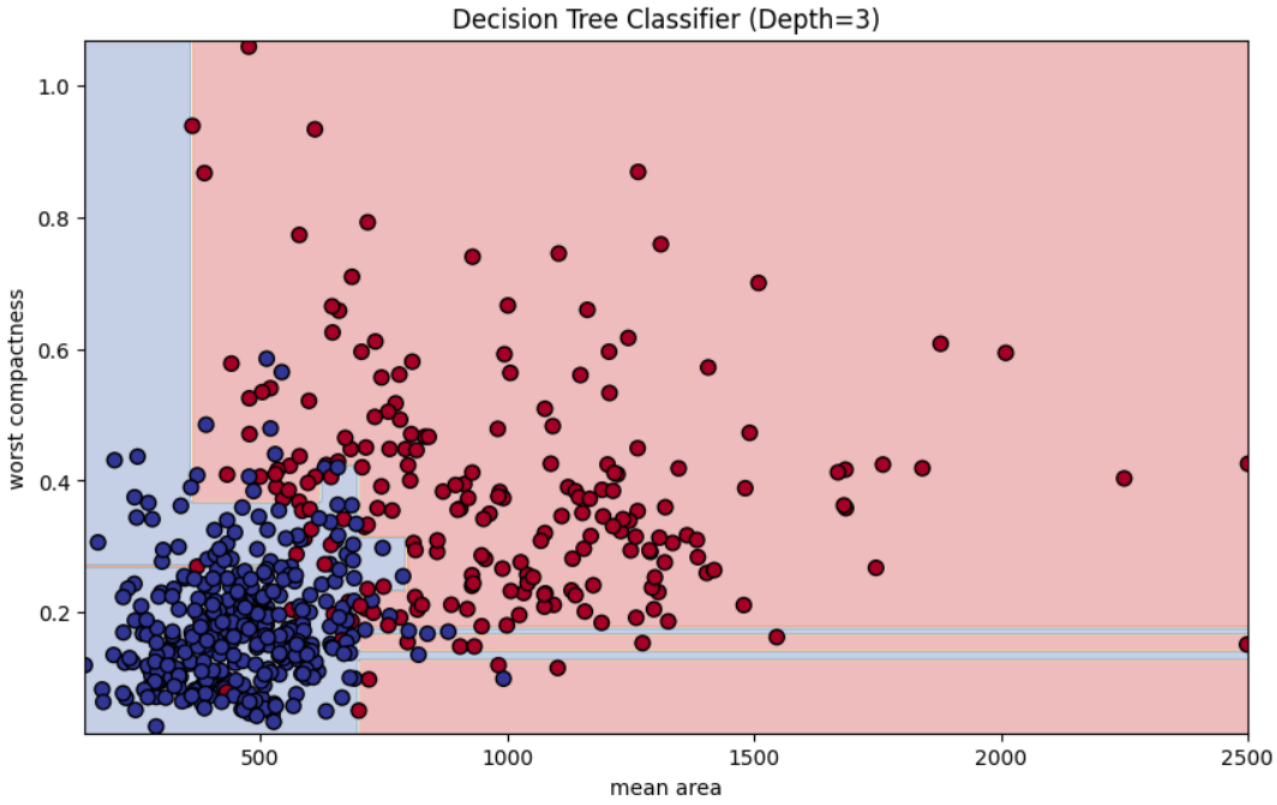
plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.RdYlBu)
scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdYlBu, edgecolor='k',
    s=50)

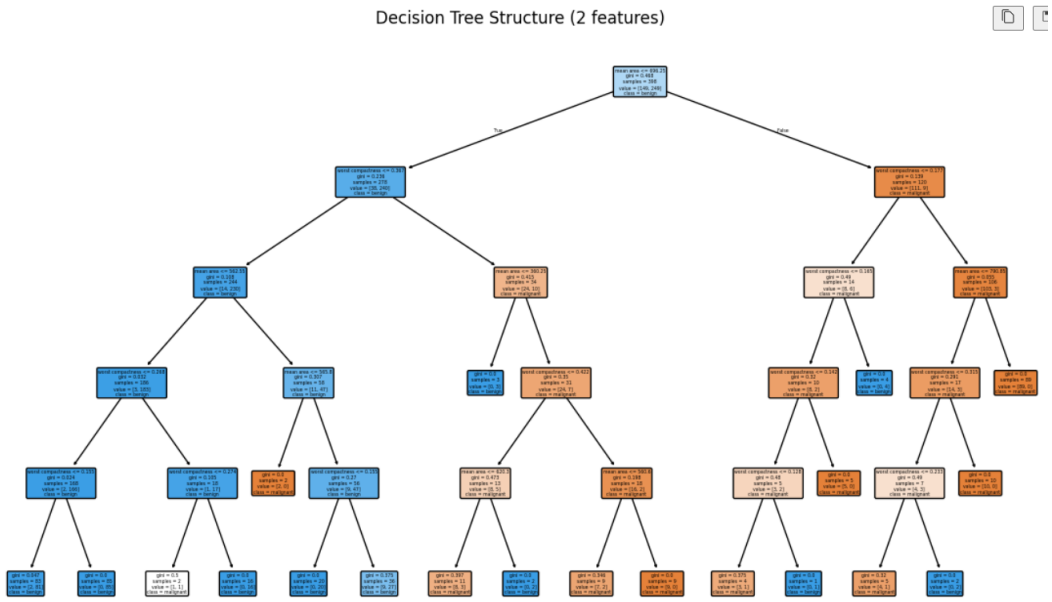
plt.xlabel(data.feature_names[3])
plt.ylabel(data.feature_names[25])
plt.title("Decision Tree Classifier (Depth=3)")
# plt.legend(handles=scatter.legend_elements()[0], labels=data.target_names)
plt.show()

plt.figure(figsize=(14, 8))
plot_tree(clf, feature_names=[data.feature_names[3], data.feature_names[25]],
          class_names=data.target_names, filled=True, rounded=True)
plt.title("Decision Tree Structure (2 features)")
plt.show()
```

Результат:

```
Accuracy on test set: 0.9298245614035088
```





✎ Питання для самоконтролю.

1. Що таке нелінійна модель машинного навчання?
2. Чим нелінійні моделі відрізняються від лінійних?
3. Які переваги мають нелінійні алгоритми?
4. Які недоліки можуть виникати під час використання нелінійних моделей?
5. У яких випадках лінійних моделей недостатньо?

6. Що таке складність моделі?
7. Як складність моделі впливає на її здатність до узагальнення?
8. Що таке поліноміальна регресія?
9. Чому поліноміальна регресія вважається нелінійною моделлю?
10. Як формується поліноміальна ознака другого степеня?
11. Як виглядає рівняння поліноміальної регресії другого порядку?
12. Для чого використовуються поліноміальні ознаки?
13. Як вибір степеня полінома впливає на модель?
14. Які переваги має поліноміальна регресія?
15. Які недоліки має поліноміальна регресія?
16. Що відбувається при надто великому степені полінома?
17. Як поліноміальна регресія може призвести до перенавчання?
18. Що таке узагальнююча здатність моделі?
19. Що таке перенавчання (Overfitting)?
20. Які ознаки свідчать про перенавчання моделі?
21. Що таке недонавчання (Underfitting)?
22. Чим перенавчання відрізняється від недонавчання?
23. Які причини виникнення перенавчання?
24. Як кількість ознак впливає на ризик перенавчання?
25. Які методи використовують для боротьби з перенавчанням?
26. Чому важливо оцінювати модель на даних, які не використовувались під час навчання?
27. Що таке перехресна валідація (Cross-Validation)?
28. Для чого використовується k-кратна перехресна валідація?
29. Як працює метод k-fold cross-validation?
30. Що означає параметр k у k-fold validation?
31. Які переваги має перехресна валідація порівняно зі звичайним поділом даних?
32. Як обчислюється підсумкова якість моделі при k-fold validation?
33. Як вибір значення k впливає на результати оцінювання?
34. Чому k-кратна перехресна валідація допомагає виявляти перенавчання?
35. Що таке поліноміальна класифікація?
36. Як поліноміальні ознаки дозволяють будувати нелінійні межі класифікації?
37. Чим поліноміальна класифікація відрізняється від лінійної?

38. Які переваги має використання поліноміальних ознак у класифікації?
39. Що таке метод опорних векторів (Support Vector Machine)?
40. Яка основна ідея алгоритму SVM?
41. Що таке гіперплощина класифікації в SVM?
42. Що таке опорні вектори?
43. Що називають відступом (margin) між класами?
44. Чому SVM прагне максимізувати margin?
45. Для чого використовується ядро (kernel) в SVM?
46. Які типи ядер найчастіше використовуються в SVM?
47. Як ядровий трюк (Kernel Trick) дозволяє будувати нелінійні класифікатори?
48. Що таке наївний баєсівський класифікатор?
49. На якій математичній теоремі базується цей алгоритм?
50. У чому полягає припущення про незалежність ознак?
51. Чому класифікатор називають «наївним»?
52. Які переваги має наївний баєсівський класифікатор?
53. Які недоліки має наївний баєсівський підхід?
54. У яких задачах особливо часто застосовують Naive Bayes?
55. Чому цей алгоритм популярний у задачах обробки тексту?
56. Що таке дерево прийняття рішень?
57. З яких основних елементів складається дерево рішень?
58. Що таке кореневий вузол дерева?
59. Що таке внутрішній вузол дерева?
60. Що таке лист дерева?
61. Як дерево приймає рішення щодо класифікації об'єкта?
62. Що таке критерій розбиття вузла?
63. Для чого використовується ентропія під час побудови дерева?
64. Що таке Information Gain?
65. Як індекс Джині (Gini Index) використовується при побудові дерева?
66. Які переваги мають дерева прийняття рішень?
67. Які недоліки мають дерева прийняття рішень?
68. Чому дерева схильні до перенавчання?

Розділ 10

Ансамблеве навчання.

Якщо ми використовуємо для прогнозування лише одну модель, вона може мати доволі таки високу варіативність - будь-яка зміна вхідних даних буде призводити до іншого результату. Саме тому в нагоді стає ідея використання набору моделей.

Ансамблеве навчання — техніка машинного навчання, що використовує кілька навчених алгоритмів з метою отримати кращу передбачальну ефективність, ніж можна отримати від кожного алгоритму окремо. На відміну від статистичного ансамблю в статистичній механіці, який зазвичай нескінченний, ансамбль моделей у машинному навчанні складається з конкретної скінченної множини альтернативних моделей, але зазвичай дозволяє існувати істотно гнучкішим структурам.

Ідея ансамблів полягає в тому, що замість того, щоб покладатися на одну "думку ми беремо багато моделей і об'єднуємо їхні прогнози. Тут можна навести аналогію з людьми - уявіть, що потрібно оцінити кількість монет в прозорій банці (нам відомо, що цих монет там 100). Одна людина може дуже сильно помилитися і назвати, наприклад, число '400'. Але якщо запитати 100 людей: '400, 50, 150, 70, 200, 40, ...', то середнє значення часто виявляється набагато ближчим до істини.

10.1 Випадковий ліс.

Випадковий ліс — це ансамблевий метод класифікації або регресії, який об'єднує велику кількість дерев рішень, кожне з яких навчається на випадковій підмножині даних і ознак. Ідея полягає в тому, щоб зменшити перенавчання (overfitting), властиве окремим деревам, і отримати більш стабільну та точну модель.

Нехай маємо навчальну вибірку:

$$D = (x_i, y_i)_{i=1}^{N}, \quad x_i \in \mathbb{R}^m, ; y_i \in 1, 2, \ldots, K$$

Алгоритм випадкового лісу створює T незалежних дерев рішень:

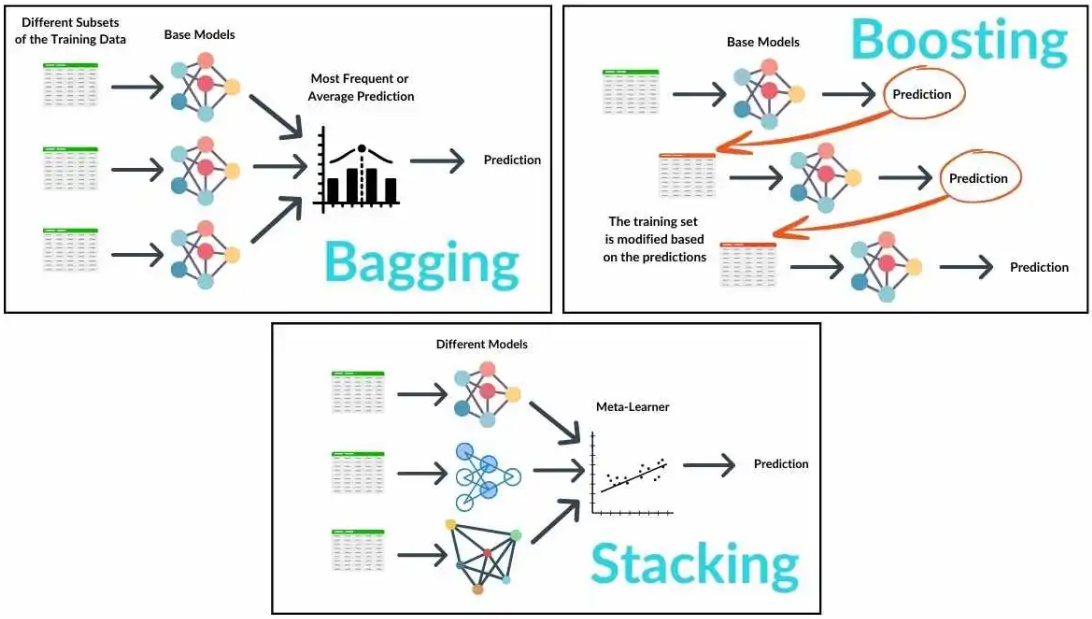
$$\text{RandomForest} = h_1(x), h_2(x), \ldots, h_T(x)$$

Після побудови всіх дерев $h_t(x)$, прогноз робиться голосуванням (для класифікації):

$$\hat{y} = \text{mode} h_1(x), h_2(x), \ldots, h_T(x)$$

або усередненням (для регресії):

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} h_t(x)$$



Для того, щоб застосувати алгоритм Випадкового Лісу, потрібно імпортувати відповідний модуль із бібліотеки sklearn:

```
from sklearn.ensemble import RandomForestClassifier
```

Класифікатор ініціалізується наступним чином:

```
clf = RandomForestClassifier(
    n_estimators=100, max_depth=5, random_state=42
)
```

Серед параметрів, які передаються в модель, можна виділити наступні:

- `n_estimators` - кількість дерев.
- `max_depth` - макимальна глибина дерева.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

data = load_breast_cancer()
X = data.data[:, [3, 25]]
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

clf = RandomForestClassifier(
    n_estimators=100, max_depth=5, random_state=42
)
clf.fit(X_train_scaled, y_train)

y_pred = clf.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))

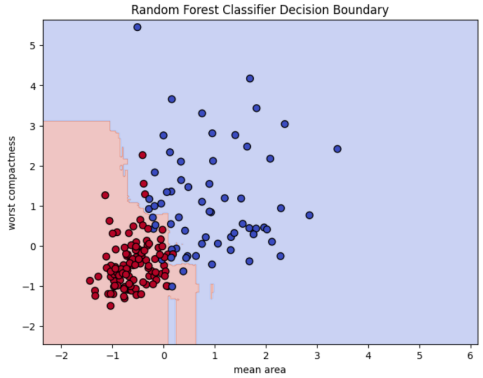
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

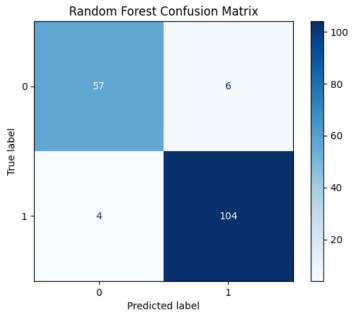
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1],
           c=y_test, cmap='coolwarm', edgecolors='k', s=50)
plt.xlabel(data.feature_names[3])
plt.ylabel(data.feature_names[25])
plt.title("Random Forest Classifier Decision Boundary")
plt.show()

ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("Random Forest Confusion Matrix")
```

Результат:





10.2 XGBoost.

XGBoost (Extreme Gradient Boosting) — це ансамблевий метод машинного навчання, заснований на градієнтному бустингу дерев рішень. На відміну від випадкового лісу, де дерева будуються незалежно одне від одного, в XGBoost кожне нове дерево навчається виправляти помилки попередніх дерев.

Нехай маємо навчальну вибірку: $D = (x_i, y_i)_{i=1}^{N}, \quad x_i \in \mathbb{R}^m$

Модель будується послідовно як сума дерев рішень:

$F_T(x) = \sum_{t=1}^{T} f_t(x)$

де $f_t(x)$ — t-те дерево рішень, а T — загальна кількість дерев. На першій ітерації модель може складатися лише з одного дерева:

$F_1(x) = f_1(x)$

На кожній наступній ітерації додається нове дерево, яке апроксимує залишки (помилки) поточної моделі:

$F_t(x) = F_{t-1}(x) + f_t(x)$

Для задач регресії залишки визначаються як:

$r_i^{(t)} = y_i - F_{t-1}(x_i)$

У загальному випадку XGBoost мінімізує функцію втрат:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{N} l(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^{t} \Omega(f_k)$$

де $l(y_i, \hat{y}_i)$ — функція втрат (наприклад MSE або LogLoss), а $\Omega(f)$ — регуляризаційний доданок для боротьби з перенавчанням.

Для побудови нового дерева використовується градієнтний розклад функції втрат:

$$g_i = \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i}$$

$$h_i = \frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}$$

де g_i та h_i є відповідно першим та другим похідними функції втрат. Після побудови всіх дерев прогноз моделі має вигляд:

$$\hat{y} = \sum_{t=1}^{T} f_t(x)$$

Для задач класифікації отримане значення додатково пропускається через логістичну функцію:

$$P(y=1|x) = \frac{1}{1+e^{-\hat{y}}}$$

та клас визначається за максимальним значенням ймовірності.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score
from xgboost import XGBClassifier

data = load_breast_cancer()
X = data.data[:, [3, 25]] # mean smoothness, worst concave points
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

clf = XGBClassifier(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
clf.fit(X_train_scaled, y_train)

y_pred = clf.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))

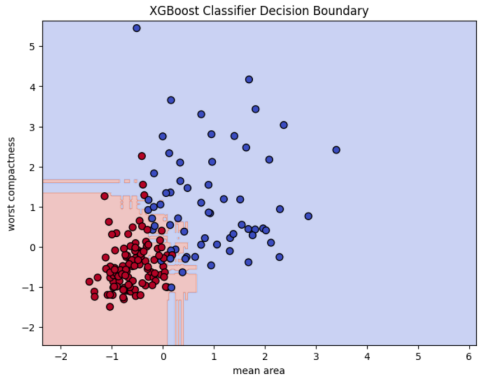
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

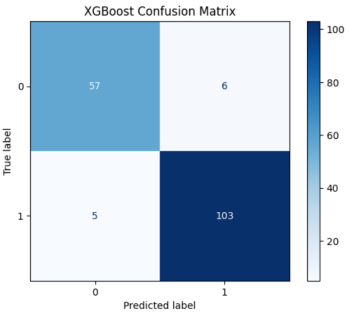
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

```
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1],
            c=y_test, cmap='coolwarm', edgecolors='k', s=50)
plt.xlabel(data.feature_names[3])
plt.ylabel(data.feature_names[25])
plt.title("XGBoost Classifier Decision Boundary")
plt.show()

ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("XGBoost Confusion Matrix")
plt.show()
```

Результат:





10.2.1 Пошук оптимальних гіперпараметрів.

10.3 Компроміс зсуву та дисперсії

Мета будь-якої моделі машинного навчання — знайти закономірність у даних, щоб робити точні передбачення. Однак модель може бути або занадто простою (недонавченою, underfitting) або занадто складною (перенавченою, overfitting).

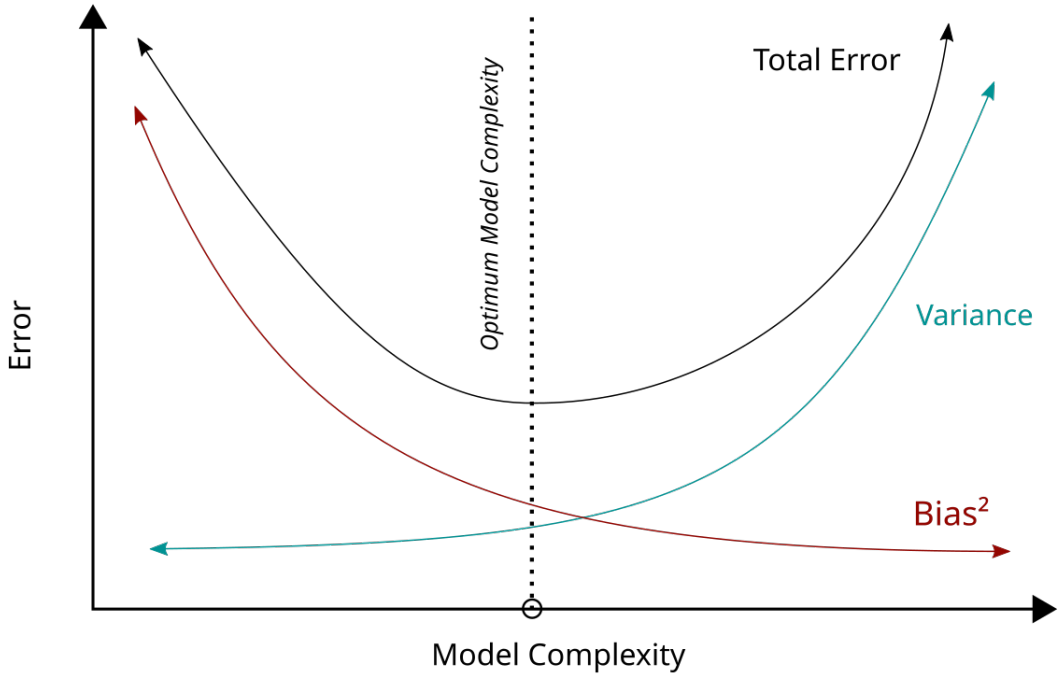
Цей баланс описує компроміс між зсувом і дисперсією.

Мета навчання — знайти модель, у якій $Bias^2$ + Variance мінімальні (шум ми не контролюємо).

☞ Варто запам'ятати, що в переважній більшості випадків, bagging ансамблеві моделі(наприклад, Випадковий Ліс) - зменшують дисперсію (variance), а boosting (XGBoost) - зменшують зсув (bias).

Табл. 10.1: Компоненти розкладу похибки (Bias-Variance Tradeoff)

Компонента	**Формула**	**Пояснення**
Зсув (Bias)	$\text{Bias}[\hat{f}(x)] = \mathbb{E}[\hat{f}(x)] - f(x)$	Наскільки «в середньому» модель відхиляється від істинної залежності. Великий зсув → модель занадто проста.
Дисперсія (Variance)	$\text{Var}[\hat{f}(x)] = \mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right)^2\right]$	Наскільки модель «чутлива» до зміни навчальних даних. Велика дисперсія → модель перенавчається.
Невідворотний шум	$\sigma^2 = \text{Var}(\varepsilon)$	Частина похибки, яку неможливо усунути, бо вона спричинена випадковими факторами в даних.



✎ **Питання для самоконтролю.**

1. Що таке ансамблеве навчання (Ensemble Learning)?
2. Яка основна ідея ансамблевих методів машинного навчання?
3. Які переваги ансамблевих моделей порівняно з окремими алгоритмами?
4. Які основні підходи до побудови ансамблів існують?
5. Що таке бэггінг (Bagging)?
6. Що таке бустинг (Boosting)?
7. Чим бэггінг відрізняється від бустингу?
8. Що таке випадковий ліс (Random Forest)?
9. Як будується окреме дерево у випадковому лісі?
10. Для чого використовується випадковий вибір ознак у Random Forest?
11. Як Random Forest формує остаточне рішення?

12. Які переваги має Random Forest?
13. Які недоліки має Random Forest?
14. Що таке XGBoost?
15. Чим XGBoost відрізняється від Random Forest?
16. Як працює алгоритм градієнтного бустингу?
17. Чому XGBoost часто забезпечує високу точність прогнозування?
18. Які механізми боротьби з перенавчанням використовуються в XGBoost?
19. Що таке гіперпараметри моделі машинного навчання?
20. Чим гіперпараметри відрізняються від параметрів моделі?
21. Які гіперпараметри є найважливішими для Random Forest?
22. Які гіперпараметри найчастіше налаштовують у XGBoost?
23. Що таке оптимізація гіперпараметрів?
24. Які методи оптимізації гіперпараметрів існують?
25. Для чого використовується Grid Search?
26. Для чого використовується Random Search?
27. Чому під час оптимізації гіперпараметрів застосовують крос-валідацію?
28. Що таке компроміс між зсувом (Bias) і дисперсією (Variance)?
29. Як високий зсув впливає на якість моделі?
30. Як висока дисперсія впливає на якість моделі та яким чином ансамблеві методи допомагають досягти оптимального компромісу між Bias і Variance?

Розділ 11

Навчання без учителя.

Навчання без учителя (Unsupervised Learning) - набір алгоритмів машинного навчання, в яких модель навчається без зазделегідь відомих правильних відповідей. Модель отримує лише набір даних, і намагається самостійно знайти в цих даних приховані закономірності.

Найважливішими задачами навчання без учителя є:

- кластеризація — автоматичний поділ об'єктів на групи;
- зменшення розмірності — скорочення кількості ознак без суттєвої втрати інформації;
- виявлення аномалій — пошук незвичайних або підозрілих об'єктів;
- аналіз асоціативних правил — пошук закономірностей між подіями або товарами.

Ці методи широко застосовуються для попереднього аналізу даних, коли інформація про правильні відповіді відсутня.

11.1 Кластеризація методом к–середніх (k-means clustering).

Кластеризація — це процес автоматичного поділу об'єктів на групи (кластери) таким чином, щоб об'єкти всередині одного кластера були максимально схожими між собою, а об'єкти різних кластерів — максимально відмінними.

Метод k–середніх (k-means clustering) — це один із найпопулярніших алгоритмів некерованого навчання (unsupervised learning), який використовується для розбиття даних на k кластерів так, щоб об'єкти всередині одного кластера були схожими між собою, а між різними кластерами — максимально відрізнялися.

Нехай маємо вибірку з n об'єктів:

$$X = x_1, x_2, \ldots, x_n, \quad x_i \in \mathbb{R}^d$$

де d — кількість ознак (вимірів).

Ми хочемо розбити множину X на k кластерів:

$$C_1, C_2, \ldots, C_k$$

так, щоб мінімізувати суму квадратів відстаней точок до центрів своїх кластерів.

Алгоритм мінімізує наступну функцію вартості (відомо як within-cluster sum of squares, WCSS):

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} |x_i - \mu_j|^2$$

де μ_j — центр (середнє) кластера C_j:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

а $|x_i - \mu_j|^2$ — квадрат евклідової відстані між точкою та центром кластера.

Алгоритм ітеративний і складається з таких кроків:

1. Ініціалізація: Вибрати випадковим чином k початкових центрів кластерів $\mu_1, \mu_2, \ldots, \mu_k$.

2. Крок призначення (Assignment step): Для кожної точки x_i визначити найближчий центр:

$$c_i = \arg \min_{j \in 1,\ldots,k} |x_i - \mu_j|^2$$

Тобто точка належить до того кластера, чий центр ближчий.

3. Крок оновлення центрів (Update step) Для кожного кластера C_j оновити центр як середнє всіх точок цього кластера:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

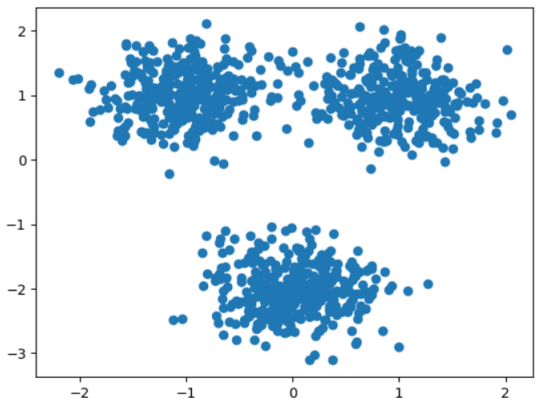
4. Перевірка зупинки: Алгоритм зупиняється, коли центри кластерів перестають змінюватися або зміни стають незначними.

Згенеруємо в python набір даних із трьох нормальних розподілів.

```
import numpy as np
import matplotlib.pyplot as plt

X = m,n = (1000,2)
np.random.seed(0)
X = 0.4*np.random.randn(m,n)
Mu0 = np.array([[0,-2], [-1,1], [1,1]])
X += Mu0[np.random.choice(np.arange(3), m),:]
plt.scatter(X[:,0], X[:,1])
```

```
<matplotlib.collections.PathCollection at 0x1598e4ea110>
```



Приклад виконання алгоритму k-середніх на даному датасеті дано нижче. Тут параметром `n_clusters` передається кількість кластерів, на які потрібно розбити наші дані.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X)
centers = kmeans.cluster_centers_

plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=60, edgecolors='k',
    alpha=0.7)
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, marker='X',
    label='Centroids')
plt.title('K-Means Clustering Example')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.show()
```

Результат:



Алгоритм k-середніх найкраще працює з числовими даними, для яких можна обчислити відстань між об'єктами. Найчастіше використовується евклідова відстань, тому ознаки повинні мати числові значення.

Варто пам'ятати за деякі недоліки цього алгоритму. Результат кластеризації чутливий до початкового вибору центроїдів. Окрім того, метод погано працює з кластерами складної форми.

11.1.1 Вибір k: метод ліктя.

Але яким чином обрати кількість кластерів k?

Метод ліктя — це графічний спосіб вибору оптимальної кількості кластерів.

Для кожного значення k обчислюється сумарна внутрішньокластерна дисперсія (Within Cluster Sum of Squares, WCSS):

$$WCSS = \sum_{i=1}^{k} \sum_{x \in C_i} |x - \mu_i|^2,$$

де C_i — множина об'єктів i-го кластера, μ_i — центр цього кластера.

Після цього будується графік залежності WCSS від кількості кластерів.

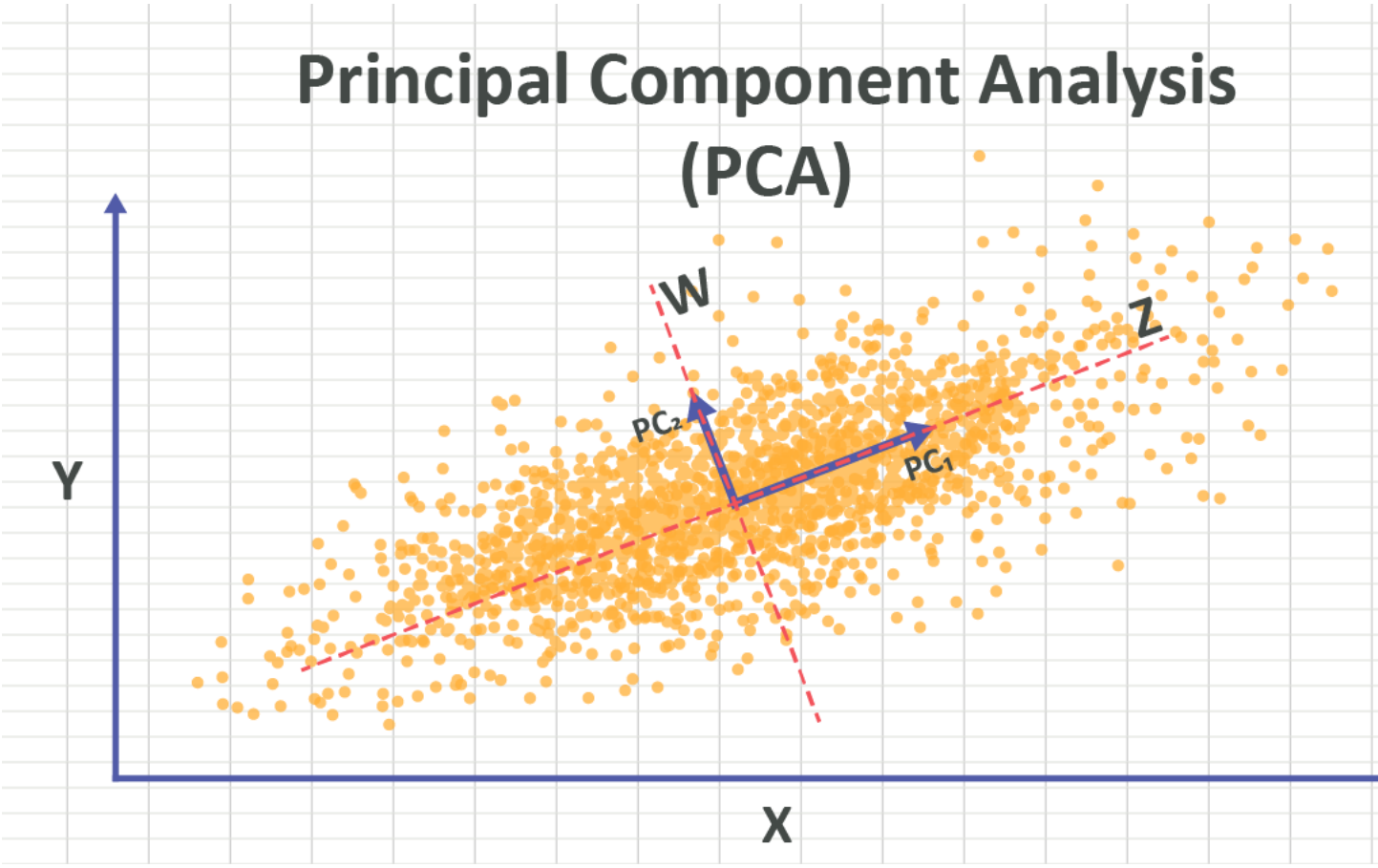
Спочатку зі збільшенням k помилка швидко зменшується, однак після певного моменту покращення стає незначним. Ця точка нагадує вигин ліктя, тому метод і отримав таку назву. Саме

відповідне значення k зазвичай вважають оптимальним.

11.2 Метод головних компонент.

Метод головних компонент (Principal Component Analysis, PCA) — це статистичний метод зменшення розмірності даних. Його ідея полягає в тому, щоб знайти нову систему координат, де:

- перша вісь (перша головна компонента) пояснює максимальну дисперсію даних,
- друга — наступну за величиною дисперсію, і т. д. Так при збереженні головних компонент ми зберігаємо максимум інформації при мінімумі вимірів.



Застосуємо алгоритм головних компонент на відомий нам датасет раку молочної залози. Завантажимо дані та виведемо розмірність.

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer(as_frame=True)
df = cancer.frame
print('Original Dataframe shape :',df.shape)
X = df[cancer['feature_names']]
print('Inputs Dataframe shape :', X.shape)
```

Результат:

```
Original Dataframe shape : (569, 31)
Inputs Dataframe shape : (569, 30)
```

30 вхідних ознак потрібно спроектувати на 2 головні ознаки. Нормалізуємо наші дані:

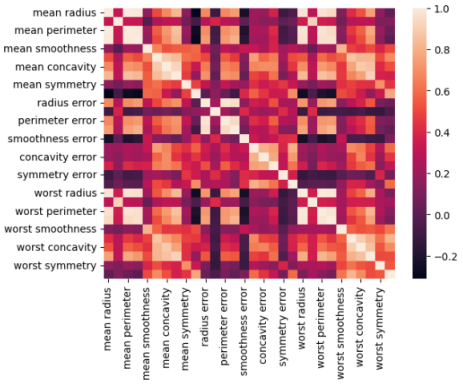
```
X_mean = X.mean()
X_std = X.std()
X_normalized = (X - X_mean) / X_std
```

та побудуємо матрицю коваріацій:

```
c = X_normalized.cov()

import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(c)
plt.show()
```

Результат:



Знайдемо на основі даних про коваріацію власні значення - власні значення матриці коваріації.

```
eigenvalues, eigenvectors = np.linalg.eig(c)
print('Eigen values:\n', eigenvalues)
print('Eigen values Shape:', eigenvalues.shape)
print('Eigen Vector Shape:', eigenvectors.shape)
```

Результат:

```
Eigen values:
 [1.32816077e+01 5.69135461e+00 2.81794898e+00 1.98064047e+00
 1.64873055e+00 1.20735661e+00 6.75220114e-01 4.76617140e-01
 4.16894812e-01 3.50693457e-01 2.93915696e-01 2.61161370e-01
 2.41357496e-01 1.57009724e-01 9.41349650e-02 7.98628010e-02
 5.93990378e-02 5.26187835e-02 4.94775918e-02 1.33044823e-04
 7.48803097e-04 1.58933787e-03 6.90046388e-03 8.17763986e-03
 1.54812714e-02 1.80550070e-02 2.43408378e-02 2.74394025e-02
 3.11594025e-02 2.99728939e-02]
Eigen values Shape: (30,)
Eigen Vector Shape: (30, 30)
```

Проіндексуємо власні значення в порядку спадання, відповідно відсортуємо і власні вектори:

```
idx = eigenvalues.argsort()[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:,idx]

explained_var = np.cumsum(eigenvalues) / np.sum(eigenvalues)
print(explained_var)
```

Результат:

```
[0.44272026 0.63243208 0.72636371 0.79238506 0.84734274 0.88758796
 0.9100953  0.92598254 0.93987903 0.95156881 0.961366   0.97007138
 0.97811663 0.98335029 0.98648812 0.98915022 0.99113018 0.99288414
 0.9945334  0.99557204 0.99657114 0.99748579 0.99829715 0.99889898
 0.99941502 0.99968761 0.99991763 0.99997061 0.99999557 1.       ]
```

Визначимо, скільки компонент ми можемо залишити з достатньо великою варіацією (разом ці компоненти мають пояснювати щонайменше 50% варіації):

```
n_components = np.argmax(explained_var >= 0.50) + 1
print(n_components)
```

Результат:

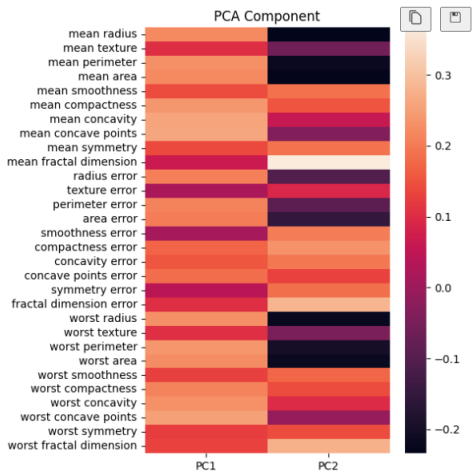
```
2
```

Побужуємо теплову карту:

```
# PCA components
u = eigenvectors[:,:n_components]
pca_component = pd.DataFrame(u, index = cancer['feature_names'], columns =
    ['PC1','PC2'] )

plt.figure(figsize =(5, 7))
sns.heatmap(pca_component)
plt.title('PCA Component')
plt.show()
```

Результат:



Множимо матриці нормалізованих даних на PCA компоненти:

```
Z_pca = X_normalized @ pca_component
Z_pca.rename({'PC1': 'PCA1', 'PC2': 'PCA2'}, axis=1, inplace=True)
print(Z_pca)
```

Результат:

```
          PCA1       PCA2
0     9.184755   1.946870
1     2.385703  -3.764859
2     5.728855  -1.074229
3     7.116691  10.266556
4     3.931842  -1.946359
..         ...        ...
564   6.433655  -3.573673
565   3.790048  -3.580897
566   1.255075  -1.900624
567  10.365673   1.670540
568  -5.470430  -0.670047

[569 rows x 2 columns]
```

Тепер той же самий алгоритм, але використовуємо модуль sklearn.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca.fit(X_normalized)
x_pca = pca.transform(X_normalized)

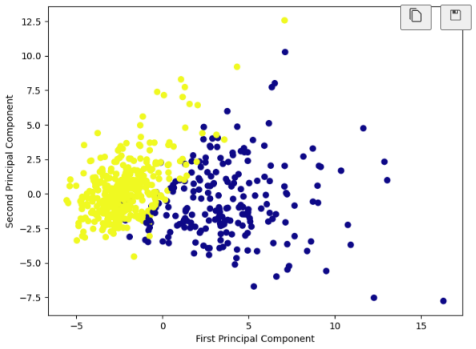
df_pca1 = pd.DataFrame(x_pca, columns=['PC{}'. format(i+1) for i in
   range(n_components)])
print(df_pca1)

plt.figure(figsize=(8, 6))
plt.scatter(x_pca[:, 0], x_pca[:, 1], c=cancer['target'], cmap='plasma')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.show()
```

Результат:

```
          PC1       PC2
0     9.184755   1.946870
1     2.385703  -3.764859
2     5.728855  -1.074229
3     7.116691  10.266556
4     3.931842  -1.946359
..         ...        ...
564   6.433655  -3.573673
565   3.790048  -3.580897
566   1.255075  -1.900624
567  10.365673   1.670540
568  -5.470430  -0.670047

[569 rows x 2 columns]
```



✎ **Питання для самоконтролю.**

1. Що таке навчання без учителя (Unsupervised Learning)?
2. Чим навчання без учителя відрізняється від навчання з учителем?
3. Які основні задачі розв'язуються методами навчання без учителя?
4. У яких сферах застосовується навчання без учителя?
5. Що таке кластеризація?
6. Яка основна ідея алгоритму k-середніх (k-Means)?
7. Як працює алгоритм k-середніх?
8. Що таке центроїд кластера?
9. Як обчислюється центр кластера?
10. Яким чином об'єкти розподіляються між кластерами?
11. Як обирається кількість кластерів k?
12. Що таке метод ліктя (Elbow Method)?
13. Які переваги має алгоритм k-середніх?
14. Які недоліки має алгоритм k-середніх?
15. Як початкове розташування центроїдів впливає на результат кластеризації?
16. Для чого використовується алгоритм k-Means++?
17. Які типи даних найкраще підходять для алгоритму k-середніх?
18. Що таке зменшення розмірності (Dimensionality Reduction)?
19. Для чого застосовується метод головних компонент (PCA)?
20. Яка основна ідея методу головних компонент?
21. Що таке головна компонента?
22. Чому головні компоненти є взаємно ортогональними?
23. Що таке дисперсія ознак і чому вона важлива в PCA?
24. Як PCA дозволяє зменшити кількість ознак без значної втрати інформації?
25. Що таке пояснена дисперсія (Explained Variance)?

26. Як визначити кількість головних компонент, які слід залишити?
27. Чому перед застосуванням PCA часто виконують стандартизацію даних?
28. Які переваги має використання PCA?
29. Які недоліки має метод головних компонент?
30. Як кластеризація та PCA можуть використовуватися разом під час аналізу даних?

Розділ 12

Перевірка статистичних гіпотез.

Перевірка гіпотез — це фундаментальний статистичний метод, який використовується в науці про дані для формування висновків або узагальнень про популяцію на основі вибіркових даних. Він допомагає нам визначити, чи є спостережуваний ефект у наборі даних статистично значущим, чи він міг виникнути випадково.

У науці про дані перевірка гіпотез використовується в різних сферах, зокрема в A/B-тестуванні, регресійному аналізі та оцінці моделей машинного навчання.

12.1 Ключові поняття в тестуванні гіпотез

Нульова гіпотеза (H_0): Нульова гіпотеза — це твердження про відсутність ефекту або різниці. Вона слугує базовим або вихідним припущенням. Наприклад, в A/B-тесті нульова гіпотеза може стверджувати, що дві групи мають однакову середню продуктивність.

Альтернативна гіпотеза (H_1): Альтернативна гіпотеза є протилежною нульовій гіпотезі. Вона передбачає, що існує статистично значущий ефект або різниця. Наприклад, альтернативна гіпотеза може стверджувати, що група A має вищий середній показник, ніж група B.

Тестова статистика: Тестова статистика — це стандартизоване значення, обчислене на основі вибіркових даних, яке використовується для оцінки сили доказів проти нульової гіпотези. До поширених тестових статистик належать t-оцінки та z-оцінки.

P-значення: P-значення представляє ймовірність отримання тестової статистики, що є принаймні такою ж екстремальною, як і спостережувана, припускаючи, що нульова гіпотеза є правдивою. Мале P-значення (зазвичай менше 0,05) вказує на сильні докази проти нульової гіпотези.

Рівень значущості (α): Рівень значущості (часто позначається як α) — це поріг, обраний аналітиком для прийняття рішення про відхилення нульової гіпотези. Зазвичай α встановлюється на рівні 0,05, що означає 5% ймовірність відхилення нульової гіпотези, коли вона є справжньою (помилка I типу).

Помилки типу I та типу II:

Помилка типу I: Неправильне відхилення нульової гіпотези, коли вона є істинною (хибнопозитивний результат).

Помилка типу II: Невідхилення нульової гіпотези, коли вона є хибною (хибнонегативний результат). Потужність тесту: Потужність тесту — це ймовірність правильного відхилення нульової гіпотези, коли вона є хибною. Висока потужність означає менший ризик помилки типу II.

Приклад: У нас є ігровий кубик (гральна кістка). Ми підозрюємо, що він "підкручений" — тобто шістка випадає частіше, ніж мала б.

Ми кидаємо кубик 60 разів, і отримуємо:

кількість шісток: 15

- Нульова гіпотеза H_0: кубик чесний

$$p = \frac{1}{6} \approx 0.1667$$

- Альтернативна гіпотеза H_1: кубик "підкручений на шістку"

$$p > \frac{1}{6}$$

(Тобто — односторонній тест.)
Випадкова величина - Позначимо:

$$X = \text{кількість шісток у 60 кидках}$$

Тоді:

$$X \sim \text{Binomial}(n = 60, p = 1/6)$$

Очікуване значення та дисперсія тоді будуть наступні:

$$\mathbb{E}[X] = np = 60 \times \frac{1}{6} = 10$$

$$\text{Var}(X) = np(1-p) = 60 \times \frac{1}{6} \times \frac{5}{6} \approx 8.33$$

$$\sigma = \sqrt{8.33} \approx 2.89$$

Наближено нормальний тест - для (n=60) можна використати нормальне наближення:

$$Z = \frac{X - \mathbb{E}[X]}{\sigma} = \frac{15 - 10}{2.89} \approx 1.73$$

Рівень значущості - візьмемо рівень $\alpha = 0.05$. Критичне значення для одностороннього тесту:

$$Z = 1.645$$

Прийняття рішення - оскільки $Z = 1.73 > 1.645$, то ми відхиляємо (H_0) на рівні значущості 0.05.

Отже, є статистичні підстави вважати, що кубик дійсно "підкручений" на шістку. Це одновибірковий біноміальний тест.

12.1.1 Перевірка гіпотез у машинному навчанні

У машинному навчанні перевірка гіпотез відіграє важливу роль в оцінці моделей. Наприклад, при порівнянні двох моделей (наприклад, дерева рішень і випадкового лісу) можна використовувати перевірку гіпотез, щоб оцінити, чи є різниця в продуктивності (наприклад, точність, прецизійність) статистично значущою.

Типовим сценарієм є використання перехресного валідації для навчання моделей на різних підмножинах даних, а потім виконання статистичного тесту (наприклад, парного t-тесту) для визначення, чи одна модель постійно перевершує іншу.

12.1.2 Практичний приклад: A/B-тестування з перевіркою гіпотез

Уявіть ситуацію, коли компанія, що займається електронною комерцією, хоче перевірити, чи новий дизайн веб-сайту (B) приносить більше покупок, ніж поточний дизайн (A). Компанія проводить A/B-тестування і збирає дані про кількість покупок, зроблених користувачами, які відвідують кожну версію сайту.

Етапи:

- Нульова гіпотеза (H_0): новий дизайн не впливає на рівень покупок (тобто немає різниці між A і B).
- Альтернативна гіпотеза (H_1): новий дизайн збільшує рівень покупок.
- Рівень значущості: Встановіть $\alpha = 0.05$.
- Збір даних: Припустимо, ви зібрали такі дані:
- Версія A: 500 відвідувачів, 50 покупок.
- Версія B: 600 відвідувачів, 80 покупок.
- Проведення тесту: Використовуйте z-тест для порівняння частоти покупок між двома групами.
- Висновок: На основі p-значення відхиліть або не відхиліть нульову гіпотезу.

Нижче ми наведемо деякі приклади статистичних перевірок.

12.2 Приклади статистичних перевірок.

12.2.1 Одновибірковий t-критерій.

Нехай у нас є дані про ріст студентів. Нашим припущенням є те, що є певний середній ріст студентів (у прикладі нижче - 183 сантиметри). Проведемо одновибірковий t-критерій (One-sample t-test), щоб перевірити цю гіпотезу.

Для цього спочатку потрібно імпортувати пакет для роботи із статистикою `import scipy.stats as s` В метод `stats.ttest_1samp` передаємо параметри: дані `data` та середнє значення, яке ми припускаємо:

```
import scipy.stats as stats
import numpy as np

data = [182, 180, 190, 184]

t_statistic, p_value = stats.ttest_1samp(a=data, popmean=183)
print(t_statistic , p_value)
```

Результат:

```
0.46291004988627577 0.6749411569291093
```

Оскільки $p = 0.675 > 0.05$, ми не можемо відхилити нульову гіпотезу. Отримані дані не дають статистично значущих підстав стверджувати, що середній зріст відрізняється від 183 см.

Інакше кажучи, вибірка $182, 180, 190, 184$ узгоджується з припущенням про середнє значення 183.

12.2.2 Однофакторний дисперсійний аналіз.

Уявімо, що ми хочемо з'ясувати, чи впливає певний фактор на деякий показник. Наприклад, інтернет-магазин провів три різні рекламні кампанії, і хоче перевірити, чи вони вплинули на кількість продажів. Або ж три методики викладання впливають на успішність студентів. Постає питання: чи дійсно середні значення цих груп відрізняються, чи різниця виникла випадково?

Ідея дуже проста. Якщо середні значення всіх груп однакові, то відмінності між ними повинні бути приблизно такими ж, як випадкові коливання всередині кожної групи. Якщо ж середні істотно різняться, то різниця між групами буде значно більшою за випадковий шум. Тому ANOVA порівнює два типи дисперсії.

Нульовою гіпотезою тут є те, що усі середні однакові. Альтернатива (H_1) - принаймні одне середнє відрізняється.

☞ Зверніть увагу: ANOVA не говорить, яка саме група відрізняється. Вона лише повідомляє, що хоча б одна середня інша.

Візьмемо дані з трьох незалежних груп, та проведемо однофакторний дисперсійний аналіз (One-way ANOVA):

```
group1 = [2.9, 3.0, 2.5, 2.6, 3.2, 2.8]
group2 = [3.8, 2.7, 4.0, 3.9, 3.2, 3.1]
group3 = [3.1, 3.4, 3.7, 3.0, 3.1, 3.6]

f_statistic, p_value = stats.f_oneway(group1, group2, group3)

print(f"F-statistic: {f_statistic}")
print(f"P-value: {p_value}")
```

Результат:

```
F-statistic: 4.434477379095165
P-value: 0.03068555150519455
```

Оскільки $p = 0.0307 < 0.05$ нульова гіпотеза відхиляється. Існують статистично значущі відмінності між середніми значеннями хоча б двох груп.

12.2.3 Одновибірковий критерій хі-квадрат.

Застосуємо одновибірковий критерій хі-квадрат (One-sample Chi-Squared test) до нашого прикладу із гральним кубиком. Розподіл результатів кидків грального кубика - від 1 до 6 - через масив в numpy під назвою `observed`:

```
observed = np.array([30, 10, 5, 5, 5, 5])

avg = observed.sum()

expected = np.array([avg/6, avg/6, avg/6, avg/6, avg/6, avg/6])

chi2_stat, p_value = stats.chisquare(f_obs=observed, f_exp=expected)

print(f"Chi-Square Statistic: {chi2_stat}")
print(f"P-value: {p_value}")
```

Результат:

```
Chi-Square Statistic: 50.0
P-value: 1.3857973367009573e-09
```

Оскільки $p << 0.05$ нульова гіпотеза впевнено відхиляється. Отриманий розподіл практично неможливо пояснити випадковими коливаннями - ігровий кубик є зваженим.

12.2.4 Критерій незалежності хі-квадрат.

Приклад даних: таблиця співвідношень зі спостережуваними частотами. Наприклад, це можуть бути дані про вподобання споживачів щодо двох брендів у різних вікових групах. Застосуємо критерій незалежності хі-квадрат (Chi-Square Test of Independence):

```
observed = np.array([[1,2,2,5],
                     [1, 3, 2, 1]])

chi2_stat, p_value, dof, expected = stats.chi2_contingency(observed)

print(f"Chi-Square Statistic: {chi2_stat}")
print(f"P-value: {p_value}")
print(f"Degrees of Freedom: {dof}")
print(f"Expected Frequencies:\n{expected}")
```

Результат:

```
Chi-Square Statistic: 2.412380952380953
P-value: 0.49133407367391724
Degrees of Freedom: 3
Expected Frequencies:
[[1.17647059 2.94117647 2.35294118 3.52941176]
 [0.82352941 2.05882353 1.64705882 2.47058824]]
```

Оскільки $p = 0.491 > 0.05$, ми не можемо відхилити нульову гіпотезу. Статистично значущого зв'язку між віковою групою та вибором бренду не виявлено.

Спостережувані відмінності можуть бути пояснені випадковими коливаннями вибірки.

12.2.5 Коли вибрати який тест?

✎ **Питання для самоконтролю.**

1. Що таке статистична гіпотеза?
2. Чим нульова гіпотеза (H_0) відрізняється від альтернативної гіпотези (H_1)?
3. Яка основна мета перевірки статистичних гіпотез?
4. Що таке рівень статистичної значущості (α)?
5. Що таке p-значення (p-value)?
6. Як приймається рішення за результатами статистичного тесту?
7. Що таке помилка першого роду (Type I Error)?
8. Що таке помилка другого роду (Type II Error)?
9. Чому перевірка статистичних гіпотез є важливою в машинному навчанні?

Табл. 12.1: Вибір статистичних критеріїв

Тип даних / Кількість груп	**1 група**	**2 групи**	**2 і більше груп**
Категоріальні дані	Тест для частки (наближення Z-критерієм) Критерій хі-квадрат	Тест для частки (наближення Z-критерієм) Критерій хі-квадрат	Критерій хі-квадрат
Неперервні дані	Z-критерій та його варіанти T-критерій та його варіанти	Z-критерій та його варіанти T-критерій та його варіанти	Дисперсійний аналіз ANOVA (F-критерій, однофакторний або двофакторний)
Порушення класичних припущень	Знаковий критерій (Sign Test) Критерій знакових рангів (Signed Rank Test)	Критерій Вілкоксона–Манна–Уїтні Парний t-критерій Критерій Мак-Немара	Критерій Краскела–Уолліса

10. У яких задачах машинного навчання застосовують статистичні критерії?
11. Що таке A/B-тестування?
12. Які етапи проведення A/B-тесту?
13. Які показники найчастіше аналізують під час A/B-тестування?
14. Що таке одновибірковий t-критерій Стьюдента?
15. За яких умов застосовується одновибірковий t-критерій?
16. Які припущення необхідно виконати для використання t-критерію?
17. Що таке однофакторний дисперсійний аналіз (One-Way ANOVA)?
18. Для чого використовується однофакторний дисперсійний аналіз?
19. Які гіпотези перевіряє ANOVA?
20. Чому не рекомендується замість ANOVA виконувати багаторазові t-тести?
21. Що таке F-статистика в дисперсійному аналізі?
22. Що таке критерій хі-квадрат узгодженості (Goodness-of-Fit Test)?
23. У яких випадках застосовується критерій узгодженості хі-квадрат?
24. Що таке очікувані та спостережувані частоти?
25. Що перевіряє критерій незалежності хі-квадрат?
26. Які дані необхідні для застосування критерію незалежності хі-квадрат?
27. Що таке таблиця спряженості (Contingency Table)?
28. Які припущення необхідно виконати для застосування критеріїв хі-квадрат?
29. Як інтерпретувати результати перевірки статистичної гіпотези?
30. Які обмеження слід враховувати під час використання статистичних критеріїв у задачах інтелектуального аналізу даних?

Розділ 13

Рекомендаційні Системи.

Рекомендаційна система - це система, яка автоматично аналізує інформацію про користувачів певного сервісу та об'єкти, якими вони користуються (товари, фільми, ін.) і пропонує користувачам ті об'єкти, в яких вони можуть бути зацікавленими. Тобто, ця система може передбачити, який рейтинг поставить той чи інший користувач певному об'єкту на основі вподобань інших користувачів та вподобань самого цього користувача.

Тут можна згадати приклад Youtube (алгоритми Youtube самі пропонують користувачам відео на основі їхніх вподобань), інтернет-магазини, Netflix та інші стрімінгові платформи, книжкові сервіси та багато інших. Всі ці онлайн-платформи імплементують алгоритми рекомендаційних систем для аналізу даних, і саме про ці алгоритми ми і поговоримо далі.

Контентно-орієнтована фільтрація: це метод рекомендації, який аналізує характеристики самих об'єктів. Основна ідея полягає у тому, що якщо користувачеві подобалися певні об'єкти, то йому, ймовірно, сподобаються інші об'єкти зі схожими характеристиками (жаер фільму, автор книги, інш.). Серед недоліків цього підходу можна виділити те, що необхідно детально описати кожен об'єкт.

13.1 Колаборативна фільтрація.

Колаборативна фільтрація (Collaborative Filtering) — це підхід у системах рекомендацій, який прогнозує, які об'єкти (товари, фільми, книги тощо) можуть сподобатися користувачу, виходячи з подібності між користувачами або між самими об'єктами.

Існує декілька підходів до обчислення оцінки:

Підходи "користувач – користувач"(User-Based Collaborative Filtering): знайти користувачів, які найбільш схожі на мене (на основі тільки тих елементів, які оцінені для нас обох), і передбачити оцінки для інших елементів на основі середнього значення

Підходи "елемент – елемент (Item-Based Collaborative Filtering)": знайти елементи, найбільш схожі на даний елемент (на основі всіх користувачів, які оцінили обидва елементи), і передбачити оцінки для інших користувачів на основі середнього значення

Підходи матричної факторизації: знайти деяку низькорангову декомпозицію матриці X, яка узгоджується з спостережуваними значеннями

Маємо:

- множину користувачів $U = u_1, u_2, \ldots, u_m$,
- множину об'єктів $I = i_1, i_2, \ldots, i_n$.

Створюється матриця оцінок:

$$R = [r_{ui}]_{m \times n}$$

де r_{ui} — рейтинг, який користувач u поставив об'єкту i, якщо користувач не оцінив об'єкт — r_{ui} відсутній (невідомий).

Передбачити невідомий рейтинг:

$$\hat{r_{ui}} \approx r_{ui}$$

для пари (користувач, об'єкт), якої ще немає у матриці.

13.1.1 Фільтрація на основі подібності користувачів.

Розглянемо підхід на основі подібності користувачів (User-based Collaborative Filtering)

Основна ідея: Шукаємо користувачів, схожих на користувача u, і прогнозуємо його рейтинг на основі оцінок цих схожих користувачів.

Кроки:

1. Обчислити схожість між користувачами, наприклад, за допомогою косинусної подібності:

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} r_{ui} r_{vi}}{\sqrt{\sum_{i \in I_{uv}} r_{ui}^2} \sqrt{\sum_{i \in I_{uv}} r_{vi}^2}}$$

де I_{uv} — множина об'єктів, які оцінили обидва користувачі.

2. Прогноз рейтингу:

$$\hat{r_{ui}} = \bar{r_u} + \frac{\sum_{v \in N(u)} \text{sim}(u, v)(r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |\text{sim}(u, v)|}$$

де $N(u)$ — набір найбільш схожих користувачів (сусідів), $\bar{r}_u$ — середній рейтинг користувача u.

Окремо можна винести Кореляцію Пірсона, яка дещо за формулою відрізняється від косинусної подібності:

$$\text{sim}(x, y) = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$$

Табл. 13.1: Матриця оцінок користувачів та фільмів

Користувач / Фільм	Фільм A	Фільм B	Фільм C	Фільм D
U1	5	4	?	2
U2	3	?	4	1
U3	?	2	5	4
U4	4	3	?	?

На практиці кожен користувач взаємодіє лише з дуже невеликою частиною всіх доступних об'єктів. Наприклад, онлайн-магазин може містити мільйони товарів, але конкретний покупець придбає лише кілька десятків із них. Аналогічно користувач Netflix перегляне лише незначну частину величезної бібліотеки фільмів. Саме тому, на практиці, ця матриця переважно є розрідженою. Ви можете нагадати, як працювати з розрідженими матрицями, в розділі 4.

Запишемо ці дані в numpy масив `ratings_matrix` та перетворимо цю матрицю в датафрейм:

```
import numpy as np
import pandas as pd

ratings_matrix = np.array([
    [5, 3, 0, 1],
    [4, 0, 0, 1],
```

```
    [1, 1, 0, 5],
    [1, 0, 0, 4],
    [0, 1, 5, 4],
])

ratings_df = pd.DataFrame(ratings_matrix, columns=["Item1", "Item2", "Item3",
    "Item4"])
```

Обчислимо кореляційну матрицю Пірсона:

```
user_similarity = ratings_df.T.corr(method='pearson')

print("User-User Pearson Correlation Matrix:\n", user_similarity)
```

Результат:

```
User-User Pearson Correlation Matrix:
          0         1         2         3         4
0  1.000000  0.774291 -0.186441 -0.178683 -0.978839
1  0.774291  1.000000  0.019854  0.162791 -0.628768
2 -0.186441  0.019854  1.000000  0.972828  0.221028
3 -0.178683  0.162791  0.972828  1.000000  0.258904
4 -0.978839 -0.628768  0.221028  0.258904  1.000000
```

Передбачимо рейтинг користувача для якогось із фільмів:

```
def predict_rating(user_index, item_index, ratings_df, similarity_matrix):
    user_means = ratings_df.replace(0, np.nan).mean(axis=1).values

    item_ratings = ratings_df.iloc[:, item_index]

    user_similarities = similarity_matrix[user_index]

    non_zero_indices = item_ratings[item_ratings > 0].index
    similarities = user_similarities[non_zero_indices]
    ratings = item_ratings[non_zero_indices]
    means = user_means[non_zero_indices]

    if len(similarities) == 0:
        return 0

    ratings_weighted = ratings - means
    weighted_sum = np.dot(similarities, ratings_weighted)

    sum_of_similarities = np.sum(np.abs(similarities))

    predicted = weighted_sum / sum_of_similarities if sum_of_similarities != 0
        else 0

    return ratings.mean() + predicted

print(predict_rating(4, 0, ratings_df, user_similarity))
```

Тут ми спочатку отримуємо рейтинг за айтем від усіх інших користувачів:

```
item_ratings = ratings_df.iloc[:, item_index]
```

Отримуємо подібність користувача з іншими користувачами:

```
user_similarities = similarity_matrix[user_index]
```

Фільтруємо користувачів за тими, хто надав рейтинг цьому айтему і вилучаємо самоподібність:

```
non_zero_indices = item_ratings[item_ratings > 0].index
similarities = user_similarities[non_zero_indices]
ratings = item_ratings[non_zero_indices]
means = user_means[non_zero_indices]
```

Якщо жоден інший користувач надав рейтинг цьому айтему, ставимо 0Ж

```
if len(similarities) == 0:
    return 0
```

І зрештою, обчислюємо зважену суму рейтингів, нормалізовану за подібностями користувачів:

```
ratings_weighted = ratings - means
weighted_sum = np.dot(similarities, ratings_weighted)

sum_of_similarities = np.sum(np.abs(similarities))

predicted = weighted_sum / sum_of_similarities if sum_of_similarities != 0 else 0
```

Результат:

```
1.0331985858244794
```

☞ На практиці рекомендаційні системи поєднують декілька методів побудови рекомендацій, таким чином утворюючи гібридну рекомендаційну систему (Hybrid Recommendation System). Саме гібридний підхід використовують більшість сучасних платформ, таких як Amazon і YouTube.

Гібридні системи мають низку переваг: вища точність рекомендацій, часткове розв'язання проблеми холодного старту; менша чутливість до розрідженості даних; більша різноманітність рекомендованих об'єктів; можливість використовувати різні джерела інформації одночасно.Завдяки цьому гібридні системи вважаються сучасним стандартом побудови рекомендаційних сервісів.

13.1.2 Холодний старт.

Проблема холодного старту (Cold Start) виникає тоді, коли рекомендаційна система не має достатньо інформації для формування якісних рекомендацій. Це може бути або коли у нас новий користувач (система іще не знає його вподобань), або новий об'єкт (товар/фільм/книга іще не мають оцінок). Може бути також зовсім нова система вцілому. Через нестачу даних алгоритми колаборативної фільтрації працюють менш ефективно.

Для подолання проблеми холодного старту використовують кілька підходів.

Найпоширенішими є:

- запропонувати новому користувачеві оцінити декілька об'єктів під час реєстрації;
- використовувати інформацію про характеристики об'єктів (контентно-орієнтована фільтрація);
- застосовувати демографічні дані користувачів;
- рекомендувати популярні або трендові об'єкти;
- використовувати гібридні рекомендаційні системи;
- аналізувати поведінку користувача (кліки, перегляди, пошукові запити) навіть без явних оцінок.

Завдяки поєднанню цих методів сучасні системи значно зменшують негативний вплив холодного старту.

13.2 Матрична факторизація.

13.3 Метрики рекомендаційних систем.

Для оцінювання рекомендаційних систем використовують різні метрики. Поговоримо про деякі з них.

Precision (точність), як і в задачах класифікації із машинного навчання, показує, яка частина рекомендованих об'єктів дійсно виявилася корисною для користувача.

$$Precision = \frac{TP}{TP + FP}$$

де TP - релевантні рекомендовані об'єкти, FP - нерелевантні.

Recall показує, яку частину всіх релевантних об'єктів система змогла рекомендувати.

Mean Average Precision (MAP) — це середнє значення показника Average Precision для всіх користувачів.

Ця метрика враховує правильність рекомендацій та порядок, у якому вони показуються. Якщо релевантні об'єкти знаходяться на початку списку рекомендацій, значення MAP буде високим. Тому MAP широко використовується для оцінювання пошукових систем і рекомендаційних платформ.

Normalized Discounted Cumulative Gain (NDCG) оцінює якість ранжування рекомендованих об'єктів.

Особливість цієї метрики полягає в тому, що вона надає більшу вагу тим рекомендаціям, які знаходяться на початку списку. Таким чином, система отримує вищу оцінку, якщо найрелевантніші рекомендації показуються першими. NDCG особливо важлива для сервісів, де користувач переглядає лише перші кілька рекомендацій.

Під час оцінки якості рекомендаційної моделі часто використовується крос-валідація: дані розбиваються на декілька частин, і на кожній ітерації одна частина використовується для тестування, а решта — для навчання.

☞ Сучасні рекомендаційні системи дедалі частіше використовують методи машинного навчання та глибокого навчання, зокрема градієнтний бустинг, нейронні мережі, автоенкодери (Autoencoders), рекурентні нейронні мережі (RNN), трансформери (Transformers), графові нейронні мережі (Graph Neural Networks, GNN) та моделі навчання з підкріпленням (Reinforcement Learning);

13.4 Етичні проблеми та сучасні підходи.

Попри численні переваги, рекомендаційні системи породжують низку етичних питань.

Основними проблемами є порушення конфіденційності користувачів та збір великої кількості персональних даних. Сучасні рекомендаційні системи дедалі частіше проєктуються з урахуванням принципів етичного використання штучного інтелекту.

Також варто пам'ятати за соціальну відповідальність. Сучасні рекомендаційні системи в соціальних платформах часто приводять до формування інформаційних бульбашок (Filter Bubble), що в свою чергу приводить до росту соціальної напруги між різними групами населення.

Останніми роками рекомендаційні системи активно інтегрують технології штучного інтелекту та великі мовні моделі (Large Language Models, LLM).

Поєднання рекомендаційних систем із сучасними моделями штучного інтелекту дозволяє створювати більш точні, гнучкі та персоналізовані сервіси, здатні враховувати як довгострокові вподобання, так і поточний контекст взаємодії користувача із системою.

✎ **Питання для самоконтролю.**

1. Що таке рекомендаційна система?
2. Які основні завдання розв'язують рекомендаційні системи?
3. У яких сферах застосовуються рекомендаційні системи?
4. Які переваги використання рекомендаційних систем для користувачів і бізнесу?
5. Які основні типи рекомендаційних систем існують?
6. Що таке контентно-орієнтована фільтрація (Content-Based Filtering)?
7. На яких даних базуються контентно-орієнтовані рекомендації?
8. Які переваги та недоліки контентно-орієнтованого підходу?
9. Що таке колаборативна фільтрація (Collaborative Filtering)?
10. Чим User-Based Collaborative Filtering відрізняється від Item-Based Collaborative Filtering?
11. Як визначається схожість між користувачами?
12. Як визначається схожість між об'єктами?
13. Для чого використовується косинусна подібність у рекомендаційних системах?
14. Які інші міри схожості застосовуються в рекомендаційних системах?
15. Що таке матриця «користувач – об'єкт» (User–Item Matrix)?
16. Чому матриця оцінок зазвичай є розрідженою (Sparse Matrix)?
17. Що таке проблема «холодного старту» (Cold Start)?
18. Які існують способи розв'язання проблеми холодного старту?
19. Що таке гібридні рекомендаційні системи?
20. Які переваги мають гібридні рекомендаційні системи?
21. Для чого використовується матрична факторизація (Matrix Factorization)?
22. Як метод сингулярного розкладу (SVD) застосовується в рекомендаційних системах?

23. Які метрики використовуються для оцінювання рекомендаційних систем?
24. Що таке Precision та Recall у задачах рекомендації?
25. Що таке метрика Mean Average Precision (MAP)?
26. Для чого використовується метрика Normalized Discounted Cumulative Gain (NDCG)?
27. Як оцінюється якість рекомендацій за допомогою крос-валідації?
28. Які сучасні методи машинного навчання застосовуються для побудови рекомендаційних систем?
29. Які етичні проблеми можуть виникати під час використання рекомендаційних систем?
30. Які сучасні тенденції розвитку рекомендаційних систем пов'язані зі штучним інтелектом і великими мовними моделями (LLM)?

Розділ 14

Часові Ряди.

Часовий ряд — це послідовність спостережень, впорядкованих у часі. Кожне значення відповідає певному моменту часу або часовому інтервалу.

Серед прикладів часових рядів можна навести щоденний курс валют, продажі магазину, температуру повітря, кількість відвідувачів веб-сторінки, ціни на акції та багато іншого.

Головною відмінністю від звичайного набору даних є наявність часової залежності між спостереженнями. У звичайних наборах даних кожен запис можна розглядати незалежно від інших. Наприклад, під час класифікації клієнтів порядок їх записів у таблиці не має значення. У часових рядах ситуація інша: значення сьогодні часто залежить від значень учора, минулого тижня або навіть кілька місяців тому.

14.1 Основи роботи з часовими рядами. Стаціонарність часового ряду.

Часовий ряд дає можливість передбачити значення наступної (майбутньої) точки (змінної), наприклад, передбачити ціну акції через годину, день, тиждень, базуючись на розмірності даних.

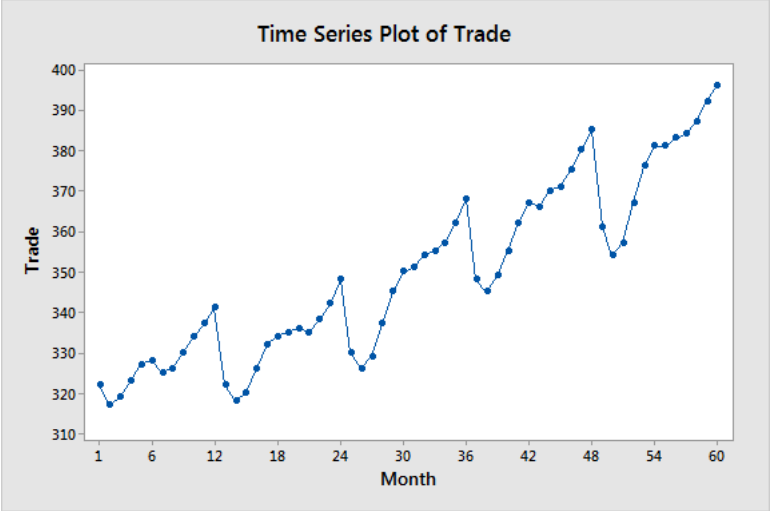
Ряд складається з двох частин - сигналу та шуму. Сигнал - це математично залежні змінні, які можна проаналізувати та прибрати з часового ряду. Шум навпаки складається з рандомних значень, частіше за все нормального розподілу, та візуально схожий на звуковий шум.

Патерни сигналу:

- Сезонність (seasonality) - циклічні коливання змінної в залежності від часу. Наприклад, зниження температури взимку.
- Тренд (trend/level) - це наклон ряду до горизонтальної осі, довгострокова тенденція зміни значень. Тренд може бути зростаючим або спадаючим, якщо тренд не змінюється, то часовий ряд розташований горизонтально.
- Автокореляція - це “пам’ять” ряду, значення змінної в конкретний час напряму залежить від попередніх значень змінної.

Якщо ряд не містить патерни, його називають стаціонарним. Стаціонарним (Stationary) є ряд без тренду та сезонності, середнє та стандартне відхилення якого є константами. Окремим патерном є часові ряди, які не можна зробити стаціонарними, прибравши тренд та/або сезонність. Такі ряди називають difference - stationary.

Для стаціонарного часового ряду незмінними залишаються: математичне сподівання (середнє значення), дисперсія, стандартне відхилення, та коваріація або автокореляція між значеннями для однакового часового лага. Саме сталість цих характеристик дозволяє будувати надійні статистичні моделі прогнозування.



Розглянемо наступний часовий ряд:

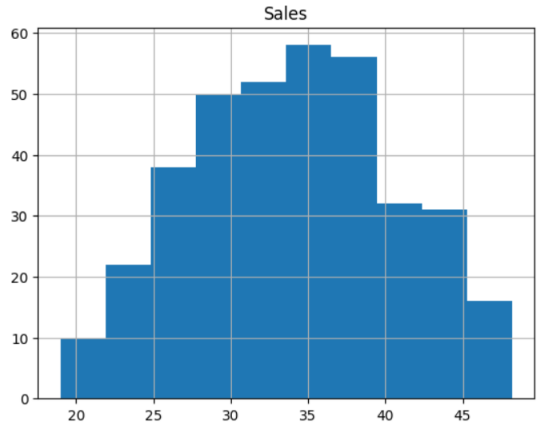
```
import pandas as pd

data = pd.read_csv('../resources/sales.csv')
display(data.head(10))

# data = data.drop('Unnamed: 0', axis=1)
data.hist()
```

Результат:

```
Unnamed: 0  Date  Sales
0  0  2021-10-01  29.109547
1  1  2021-10-02  30.720435
2  2  2021-10-03  30.305173
3  3  2021-10-04  27.390893
4  4  2021-10-05  22.841123
5  5  2021-10-06  19.002444
6  6  2021-10-07  23.904594
7  7  2021-10-08  27.544137
8  8  2021-10-09  30.567059
9  9  2021-10-10  30.087033
```

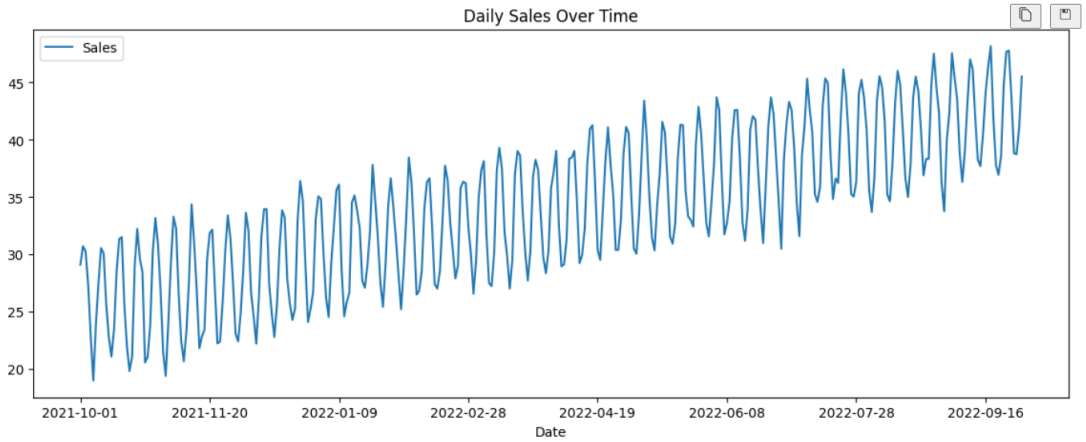


Зобразимо даний часовий ряд:

```
data.plot(x='Date', y='Sales', kind='line', title='Daily Sales Over Time',
```

```
    figsize=(14, 5))
```

Результат:



Виведемо, якільки є відсутніх даних:

```
# data has no missing values
print(data.describe())
print("NAN values:",data.isna().sum())
```

Результат:

```
            Sales
count  365.000000
mean    34.093942
std      6.572683
min     19.002444
25%     29.290376
50%     33.971567
75%     38.647294
max     48.190967
NAN values: Date     0
Sales    0
dtype: int64
```

На графіку вище чітко видно сезонну закономірність, яка повторюється тиждня, та висхідний тренд. Давайте детальніше розглянемо тренд та сезонну закономірність:

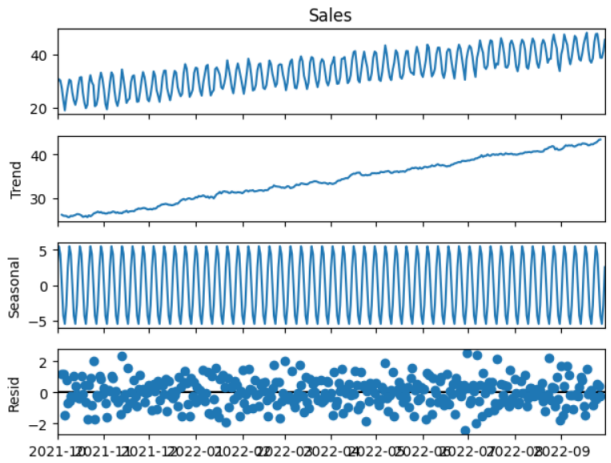
Шум (resid) - це часовий ряд після прибирання з нього тренду та сезонності. Це випадкові коливання значень, які не можна пояснити трендом, сезонністю або циклічністю. Причинами шуму можуть бути випадкові події або похибки у вимірюваннях. Шум не містить корисної закономірності та ускладнює аналіз даних - чим більший рівень шуму, тим складніше моделі знайти справжні закономірності.

```
data['Date'] = pd.to_datetime(data['Date'])
data = data.set_index('Date')
data = data.asfreq('D')

from statsmodels.tsa.seasonal import seasonal_decompose

decomposed = seasonal_decompose(data['Sales'])
decomposed.plot()
```

Результат:



Диференціювання — це метод перетворення часового ряду, при якому кожне нове значення замінюється різницею між поточним і попереднім значенням. За допомогою диференціювання багато часових рядів можна зробити стаціонарними.

14.1.1 Доповнений тест Дікі - Фуллера (ADF).

Тест ADF - це тест для перевірки стаціонарності часового ряду на основі unit root - це процес, який через диференціювання визначає чи є часовий ряд стаціонарним. Тест працює за принципом приймання або відкидання нульової гіпотези, яка полягає у тому, що ряд не є стаціонарним. Приймати чи відкидати нульову гіпотезу вирішується за значенням p (вірогідність неправильності нульової гіпотези): якщо значення є більшим за визначену константу, то нульова гіпотеза приймається (ряд не є стаціонарним), якщо меншим - гіпотеза відкидається (ряд є стаціонарним). Традиційно, константа = 0.05 (5

Розглянемо приклад дослідження числового ряду на стаціонарність за допомогою доповненого теста Дікі - Фуллера. Ми застосовуватимо можливості модуля statsmodels в python.

```
from statsmodels.tsa.stattools import adfuller
import matplotlib.pyplot as plt

# data = data.drop('Unnamed: 0', axis=1)

def adfuller_test(df):
    adf=adfuller(df)
    statistic= adf[0]
    p_value=adf[1]
    critical_values=adf[4]
    return p_value,statistic,critical_values

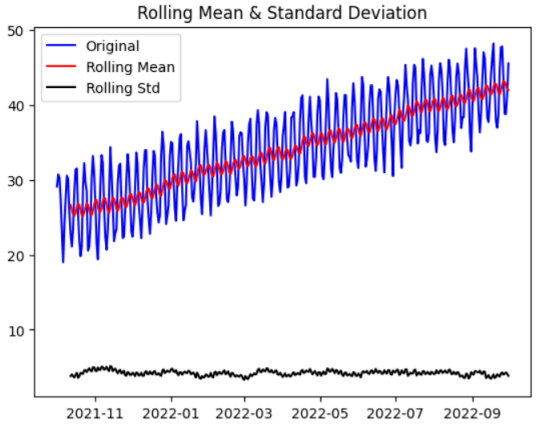
def stat_check(df,p_value_check=0.05):
    movingAverage = df.rolling(window=12).mean()
    movingSTD = df.rolling(window=12).std()
    orig = plt.plot(df, color='blue', label='Original')
    mean = plt.plot(movingAverage, color='red', label='Rolling Mean')
    std = plt.plot(movingSTD, color='black', label='Rolling Std')
    plt.legend(loc='best')
    plt.title('Rolling Mean & Standard Deviation')
```

```
    plt.show(block=False)

    p_value,statistic,critical_values=adfuller_test(df)
    if (p_value<p_value_check):
        print('Time series is stationary, p value:{}'.format(p_value))
    else:
        print('Time series is NOT stationary, p value:{}'.format(p_value))
    print('Statistic:{}'.format(statistic))
    for key, value in critical_values.items():
        print('Critial Values:')
        print(f'  {key}, {value}')

stat_check(data)
```

Результат:




```
Time series is NOT stationary, p value:0.979126294138459
Statistic:0.34045531718527483
Critial Values:
   1%, -3.4492815848836296
Critial Values:
   5%, -2.8698813715275406
Critial Values:
   10%, -2.5712138845950587
```

p-value показує ймовірність отримати спостережувані результати за умови, що нульова гіпотеза є істинною. Якщо p-value $< 0,05$, нульову гіпотезу відхиляють, і часовий ряд вважають стаціонарним; якщо p-value $\geq 0,05$, немає достатніх підстав відхилити нульову гіпотезу, тому ряд вважають нестаціонарним. Тест Дікі—Фуллера зазвичай виконують перед побудовою моделей ARIMA.

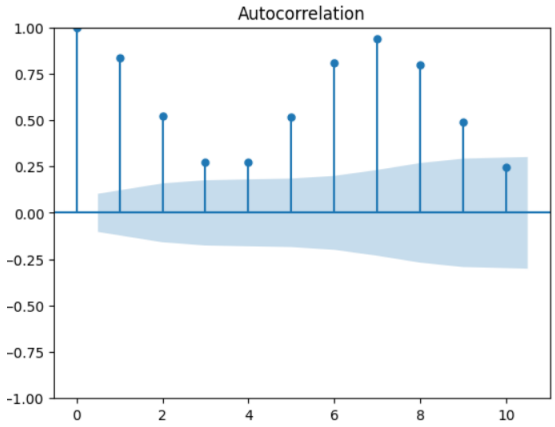
14.2 Автокореляція.

Кореляція - це зв'язок змінних між собою, автокореляція - зв'язок змінної з власними попередніми значенням. Оскільки ряд має тільки одну змінну, аналізувати потрібно тільки автокореляцію. Перед тим, як графік автокореляції розкриється у всій красі, розкажу про поняття lags (лаги) - це значення змінної n - кількість часу назад, оскільки розмірністю нашого часового ряду є день, то lag -1 означає значення 1 днів тому для поточної змінної, -2 - два дні тому, тощо. На наведеному нижче графіку кількість лагів дорівнює 10.

```
from statsmodels.graphics.tsaplots import plot_acf

plot_acf(data,lags=10)
plt.show()
```

Результат:



Декомпозиція - це процес перетворення нестаціонарного часового ряду у стаціонарний шляхом прибирання з нього сезонності та тренду.

Реконструкція часового ряду - це процес зворотнього додавання до ряду тренду та сезонності, якщо цього не зробити, ряд буде паралельним осі x та буде мало корисним як прогнозування.

14.3 Функції втрати.

Показниками якості моделей та прогнозування для часових рядів є функції втрати та метрики похибки. Найчастіше використовують такі функції втрати як MAE (L1 norm) - "Mean Square Error MSE(L2 norm) - "Mean Absolute Error".

MAE надає кожній помилці однакову"“вагу тобто є робастною до викидів та шуму функцією, але одночасно з цим занижує "вплив"великих помилок.

Формула:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

MSE в свою чергу навпаки, дає більшим помилкам більшу вагу, що робить функцію більш чутливою до викидів. Якщо модель повинна бути чутлива до помилок, обирають саме цю функцію.

Формула:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) є квадратним коренем із середньої квадратичної помилки. Ця метрика також сильніше реагує на великі помилки, однак її перевага полягає в тому, що вона вимірюється в тих самих одиницях, що й прогнозований показник. Саме тому RMSE широко використовується для порівняння моделей прогнозування часових рядів.

Формула:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Mean Absolute Percentage Error (MAPE) або середня абсолютна відносна помилка показує середню величину помилки у відсотках. Наприклад, якщо MAPE дорівнює 5 %, це означає, що прогноз у середньому відхиляється від реальних значень на п'ять відсотків. Завдяки такому представленню результати легко інтерпретувати та порівнювати між різними наборами даних. Водночас MAPE не рекомендується використовувати, якщо реальні значення можуть наближатися до нуля.

Формула:

$$MAPE = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_1 - \hat{y}_i}{y_i}\right|$$

```
import numpy as np

def calculate_mape(actual, predicted):

    if([isinstance(actual, np.ndarray) or isinstance(predicted, np.ndarray)]):
        actual, predicted = np.array(actual), np.array(predicted)
    else:
        actual, predicted = np.Int(actual), np.Int(predicted)

    return round(np.mean(np.abs((
      actual - predicted) / actual)) * 100, 2)
```

14.4 ARIMA.

ARIMA (AutoRegressive Integrated Moving Average) — один із найпопулярніших методів класичного прогнозування часових рядів. Його використовують, коли дані мають автокореляцію, тренд, але не обов'язково мають сезонність.

ARIMA дозволяє моделювати складні часові залежності й робити прогноз на майбутні значення.

Модель ARIMA описується трійкою параметрів:

$$ARIMA(p, d, q)$$

де p – кількість авторегресивних лагів (AR, або іншими словами, кількість попередніх значень), d – порядок диференціювання (I, Integrated) щоб зробити ряд стаціонарним, q – порядок ковзного середнього (MA, кількість попередніх похибок).

Поточне значення залежить від попередніх:

$$y_t = \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

Авторегресійна частина (AR(p)) - поточне значення залежить від попередніх. Інтегрована частина (I(d)) - диференціювання робить ряд стаціонарним:

$$y_t^d = y_t - y_{t-1}$$

або для d>1 – повторюється кілька разів. Стаціонарність важлива, бо AR і MA працюють саме з нею.

Ковзне середнє (MA(q)):

$$y_t = \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q} + \varepsilon_t$$

Це модель залежності від попередніх випадкових шоків (помилок).

Повна модель ARIMA - поєднання всіх частин:

$$ARIMA(p,d,q) = AR(p) + I(d) + MA(q)$$

Після диференціювання:

$$y'_t = \phi_1 y'_{t-1} + ... + \phi_p y'_{t-p} + \theta_1 \varepsilon_{t-1} + ... + \theta_q \varepsilon_{t-q} + \varepsilon_t$$

Починаємо із $y'_0 = y_0$

```
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(data['Sales'], order=(7,1,7))
model_fit = model.fit()

print(model_fit.summary())
forecast = model_fit.forecast(steps=30)

print("Forecasted values:")
print(forecast)

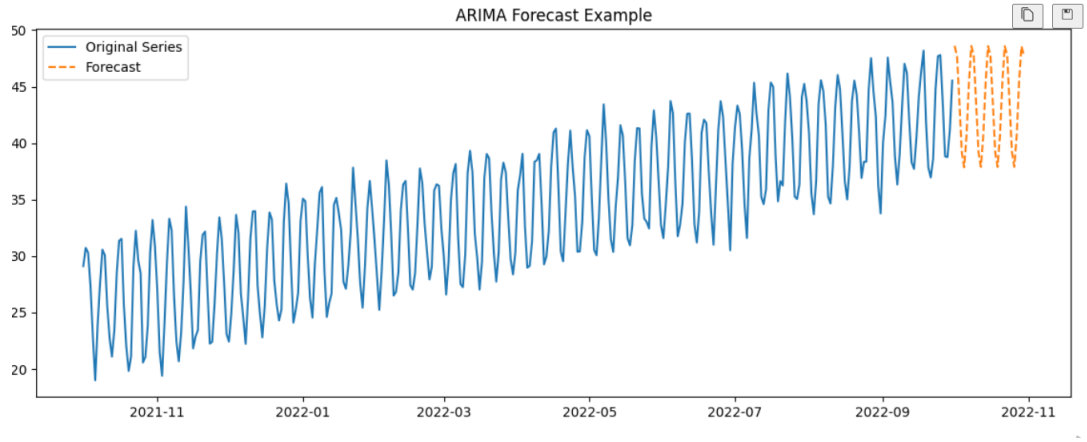
plt.figure(figsize=(14, 5))
plt.plot(data['Sales'], label="Original Series")
plt.plot(forecast.index, forecast, label="Forecast", linestyle='--')
plt.legend()
plt.title("ARIMA Forecast Example")
plt.show()
```

Результат:

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                  Sales   No. Observations:                  365
Model:                 ARIMA(7, 1, 7)   Log Likelihood                -536.758
Date:                Fri, 21 Nov 2025   AIC                           1103.517
Time:                        12:02:09   BIC                           1161.974
Sample:                    10-01-2021   HQIC                          1126.751
                         - 09-30-2022
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -1.5086      1.264     -1.193      0.233      -3.986       0.969
ar.L2         -1.0640      0.866     -1.228      0.219      -2.762       0.634
ar.L3         -0.8506      0.554     -1.536      0.124      -1.936       0.235
ar.L4         -1.3599      0.572     -2.376      0.018      -2.482      -0.238
ar.L5         -1.4469      1.224     -1.182      0.237      -3.846       0.952
ar.L6         -0.7478      0.760     -0.984      0.325      -2.238       0.742
ar.L7         -0.1451      0.231     -0.629      0.529      -0.597       0.307
```

```
ma.L1          0.5698      1.260      0.452      0.651      -1.900       3.039
ma.L2         -0.2415      0.583     -0.414      0.679      -1.385       0.902
ma.L3          0.0016      0.369      0.004      0.997      -0.721       0.724
ma.L4          0.6490      0.221      2.934      0.003       0.216       1.083
ma.L5          0.3343      0.746      0.448      0.654      -1.129       1.797
ma.L6         -0.4259      0.335     -1.270      0.204      -1.083       0.232
ma.L7         -0.3330      0.435     -0.765      0.444      -1.186       0.520
sigma2         1.1157      0.091     12.310      0.000       0.938       1.293
===================================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):                 0.39
Prob(Q):                              0.95   Prob(JB):                         0.82
Heteroskedasticity (H):               0.92   Skew:                             0.08
Prob(H) (two-sided):                  0.67   Kurtosis:                         3.02
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
Forecasted values:
2022-10-01    48.616060
2022-10-02    47.693508
2022-10-03    43.710640
2022-10-04    39.170891
2022-10-05    37.856026
2022-10-06    40.696951
2022-10-07    45.373877
2022-10-08    48.592830
2022-10-09    47.813319
2022-10-10    43.598087
2022-10-11    39.219554
2022-10-12    37.894455
2022-10-13    40.630354
2022-10-14    45.411763
2022-10-15    48.584311
2022-10-16    47.786612
2022-10-17    43.630119
2022-10-18    39.215936
2022-10-19    37.889536
2022-10-20    40.644359
2022-10-21    45.394903
2022-10-22    48.578806
2022-10-23    47.792791
2022-10-24    43.627127
2022-10-25    39.225426
2022-10-26    37.895271
2022-10-27    40.638293
2022-10-28    45.391572
2022-10-29    48.573010
2022-10-30    47.790048
Freq: D, Name: predicted_mean, dtype: float64
```



☞ ARIMA доцільно використовувати, якщо часовий ряд є стаціонарним або його можна зробити стаціонарним за допомогою диференціювання. ARIMA підходить також для часових рядів що не мають складних нелінійних залежностей.

Проте ARIMA звісно має свої обмеження із моделюванням нелінійних залежностей. Точність для стаціонарних рядів без нелінійних залежностей, точність для довгострокового прогнозування стрімко падає.

Якщо часовий ряд має складну структуру або залежить від багатьох факторів, часто ефективнішими є методи машинного навчання або глибокого навчання.

14.5 Передбачення із використанням методів машинного навчання.

Застосуємо ансамблеву модель випадкового лісу для передбачення значення часового ряду. Тут ознаками будуть лагові значення (попередні n значень часового ряду), цільовою змінною буде поточне значення, яке потрібно передбачити. Таким чином модель навчається прогнозувати наступне значення за кількома попередніми.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor

def create_lags(series, n_lags=10):
    df = pd.DataFrame({"y": series})
    for i in range(1, n_lags + 1):
        df[f"lag_{i}"] = df["y"].shift(i)
    return df.dropna()

N_LAGS = 10
df_lags = create_lags(data['Sales'], N_LAGS)

X = df_lags.drop("y", axis=1)
y = df_lags["y"]

model = RandomForestRegressor(n_estimators=500, random_state=42)
model.fit(X, y)

steps = 30
```

```
last_values = list(data['Sales'].values[-N_LAGS:])
forecast_values = []

for _ in range(steps):
    x_input = np.array(last_values[-N_LAGS:]).reshape(1, -1)
    y_pred = model.predict(x_input)[0]

    forecast_values.append(y_pred)
    last_values.append(y_pred)

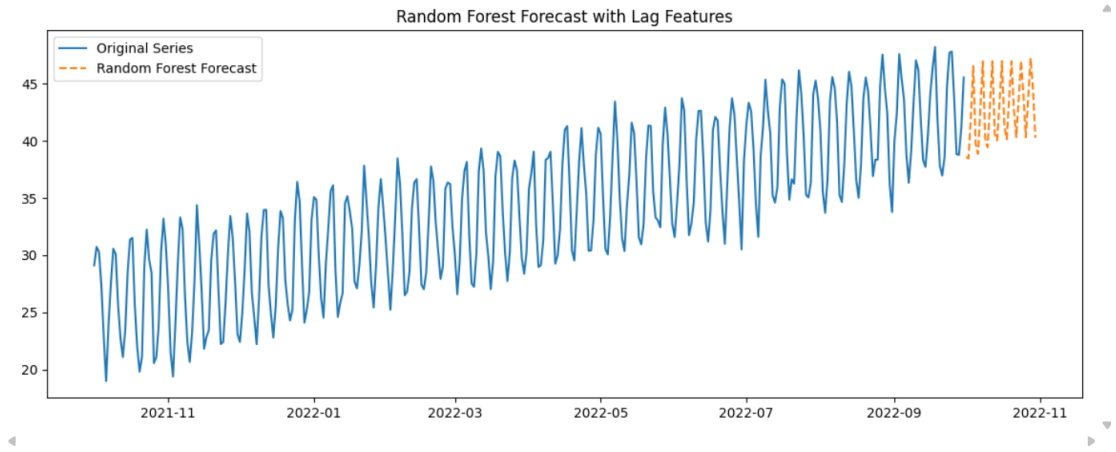
last_date = data.index[-1]
future_index = pd.date_range(start=last_date, periods=steps+1, freq='D')[1:]

forecast = pd.Series(forecast_values, index=future_index)

print("Forecast:")
plt.figure(figsize=(14, 5))
plt.plot(data['Sales'], label="Original Series")
plt.plot(forecast.index, forecast, label="Random Forest Forecast", linestyle='--')
plt.legend()
plt.title("Random Forest Forecast with Lag Features")
plt.show()
```

Результат:

```
Forecast:
```



Для задач із великою кількістю даних і складними закономірностями найкращі результати часто демонструють LSTM, GRU та Transformer-моделі.

У звичайних задачах машинного навчання випадкове перемішування даних допомагає уникнути зміщення вибірки. Для часових рядів цього робити не можна, оскільки спостереження мають природний часовий порядок. Якщо випадково перемішати дані, модель може отримати інформацію з майбутнього під час навчання, що призведе до так званого витоку даних (Data Leakage).

14.6 Експоненційне зглажування.

Експоненційне згладжування — це клас методів прогнозування часових рядів, у яких останні спостереження отримують більшу вагу, ніж старі. Ваги зменшуються експоненційно, звідки й

назва. Завдяки цьому модель швидше реагує на зміни в даних і водночас згладжує випадкові коливання. На відміну від простого ковзного середнього, ваги попередніх спостережень зменшуються експоненційно, що й дало назву цьому методу.

Нехай y_t — фактичне значення часового ряду, а $\hat{y}_t$ — прогноз.

Тоді просте експоненційне згладжування (SES) визначається як:

$$\hat{y}_t = \alpha y_{t-1} + (1-\alpha)\hat{y}_{t-1}$$

де $0 < \alpha < 1$ — коефіцієнт згладжування, велике α - модель сильніше реагує на нові дані, мале α - прогноз згладжений, повільно змінюється

Експоненційне згладжування найкраще підходить для часових рядів, у яких зберігаються відносно стабільні закономірності. Просте згладжування використовується для стаціонарних рядів без тренду, метод Хольта — для рядів із трендом, а метод Хольта—Вінтерса — для даних, що містять як тренд, так і сезонність. Ці методи широко застосовуються для короткострокового прогнозування економічних показників, продажів, попиту на товари, споживання електроенергії та інших процесів.

Модель Холта (Holt's Linear Trend): якщо в ряді є тренд, ми додаємо окрему модель для тренду:

$$\hat{y}_t = \ell_{t-1} + b_{t-1}\ \ell_t \ \ = \alpha y_t + (1-\alpha)(\ell_{t-1} + b_{t-1})\ b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1}$$

де ℓ_t — рівень, b_t — тренд, $\alpha, \beta \in (0,1)$

```
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing

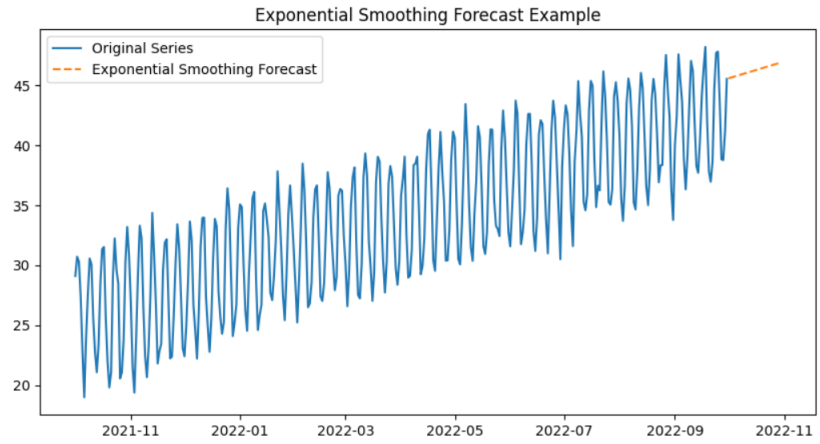
model = ExponentialSmoothing(
    data['Sales'],
    trend="add",
    seasonal=None
)

model_fit = model.fit()
forecast = model_fit.forecast(30)

print("Forecast:")
plt.figure(figsize=(10, 5))
plt.plot(data['Sales'], label="Original Series")
plt.plot(forecast.index, forecast, label="Exponential Smoothing Forecast",
   linestyle="--")
plt.legend()
plt.title("Exponential Smoothing Forecast Example")
plt.show()
```

Результат:

```
Forecast:
```



☞ Варто пам'ятати, що методи машинного навчання потребують значно більшої кількості навчальних даних для правильного прогнозу. Окрім того, вони часто є менш інтерпретованими, та мають високу обчислювальну складність. У сучасному інтелектуальному аналізі даних вибір моделі залежить від конкретної задачі. Для невеликих і добре структурованих часових рядів часто достатньо моделей ARIMA або експоненційного згладжування. Якщо ж часовий ряд має складні нелінійні залежності або залежить від великої кількості зовнішніх факторів, доцільніше використовувати методи машинного навчання чи глибокого навчання, такі як XGBoost, LSTM або Transformer.

✎ **Питання для самоконтролю.**

1. Що таке часовий ряд?
2. Чим часовий ряд відрізняється від звичайного набору даних?
3. Які приклади часових рядів можна зустріти в інтелектуальному аналізі даних?
4. Які основні компоненти містить часовий ряд?
5. Що таке тренд у часовому ряді?
6. Що таке сезонність?
7. Що таке шум у часовому ряді?
8. Як шум впливає на точність прогнозування?
9. Що таке стаціонарний часовий ряд?
10. Чому стаціонарність є важливою під час побудови моделей прогнозування?
11. Які характеристики стаціонарного часового ряду залишаються незмінними з часом?
12. Як можна перетворити нестаціонарний часовий ряд на стаціонарний?
13. Що таке диференціювання (Differencing)?
14. Для чого використовується тест Дікі—Фуллера (Augmented Dickey–Fuller Test)?
15. Як інтерпретувати результати тесту Дікі—Фуллера?
16. Що означає p-value у тесті Дікі—Фуллера?
17. Що таке автокореляція?

18. Як автокореляція використовується під час аналізу часових рядів?
19. Що показує функція автокореляції (ACF)?
20. Для чого використовується часткова автокореляційна функція (PACF)?
21. Яку роль відіграють графіки ACF та PACF при виборі параметрів моделі ARIMA?
22. Що таке модель ARIMA?
23. Що означають параметри p, d та q у моделі ARIMA?
24. У яких випадках доцільно використовувати модель ARIMA?
25. Які обмеження має модель ARIMA?
26. Що таке експоненційне згладжування?
27. Чим просте експоненційне згладжування відрізняється від методів Хольта та Хольта—Вінтерса?
28. Для яких часових рядів найкраще підходить експоненційне згладжування?
29. Які функції втрат використовуються для оцінювання якості прогнозування часових рядів?
30. Що таке Mean Absolute Error (MAE)?
31. Що таке Mean Squared Error (MSE)?
32. Що таке Root Mean Squared Error (RMSE)?
33. Що таке Mean Absolute Percentage Error (MAPE)?
34. Чому для прогнозування часових рядів застосовують методи машинного навчання?
35. Які алгоритми машинного навчання найчастіше використовуються для прогнозування часових рядів?
36. Як формується навчальна вибірка для моделей машинного навчання при роботі з часовими рядами?
37. Чому під час прогнозування часових рядів не можна випадково перемішувати дані перед навчанням моделі?
38. Які переваги та недоліки мають класичні статистичні моделі (ARIMA, експоненційне згладжування) порівняно з сучасними методами машинного навчання?

Розділ 15

Моделювання Кризових Явищ.

Кризове явище - це різка або суттєва зміна стану системи, яка призводить до негативних наслідків і може порушити її нормальне функціонування. З точки зору інтелектуального аналізу даних кризові явища розглядаються як події, що можуть бути виявлені або передбачені на основі аналізу історичних даних.

Серед прикладів кризових явищ можна визначити фінансові кризи та економічні спади, епідемії, техногенні аварії, природні катастрофи та масові кібератаки. Основною метою аналізу є своєчасне виявлення ознак наближення кризи та прогнозування її розвитку.

Передбачення криз дозволяє завчасно приймати управлінські рішення та мінімізувати можливі збитки. Завдяки прогнозуваннюю можна своєчасно виявляти ризики та планувати заходи врегулювання.

15.1 Передбачення фінансових криз.

Для прогнозування економічних криз можна використовувати великі обсяги фінансових даних. Зокрема, для цього добре підійдуть ціни акцій та фондові індекси, валютні курси та процентні ставки, рівні безробіття та інфляції, ВВП та кредитна активність, та багато інших. Поєднання різних джерел даних дозволяє підвищити точність прогнозування.

У прогнозуванні фінансових криз використовуються різні задачі машинного навчання.

Найпоширенішими є:

- класифікація — визначення, чи існує ризик виникнення кризи;
- регресія — прогнозування числових економічних показників;
- аналіз часових рядів — прогнозування майбутніх значень економічних індикаторів;
- кластеризація — виявлення груп країн або компаній із подібною фінансовою поведінкою;
- виявлення аномалій — пошук нетипових змін, що можуть передувати кризі.

Кожна з цих задач допомагає досліджувати кризові процеси з різних сторін. Для прогнозування криз використовують як класичні статистичні методи, так і сучасні алгоритми машинного навчання.

Методи машинного навчання мають низку переваг порівняно з традиційними економічними моделями. Вони дозволяють працювати з великими обсягами даних, знаходити складні нелінійні залежності та приховані закономірності в фінансових даних.

В той же час, прогнозування фінансових криз є складною задачею через особливості економічних процесів. Кількість історичних криз достатньо невелика - у нас просто замало даних.

Фінансові дані часто містять велику кількість шуму, а моделі що вчаться на цих даних підпадають під ризик перенавчання. Крім того, на економічні процеси можуть суттєво впливати політичні рішення, міжнародні конфлікти та інші непередбачувані події.

Будь-яка модель машинного навчання навчається на наявних даних. Якщо фінансові дані є неповними, неточними або містять помилки, модель також формуватиме неправильні прогнози.

Завантажемо дані з ринку $S\&P$ 500 цінних паперів, дані про дохідність облігацій (як економічний індикатор), та об'єднаємо ці датасети за датами.

```
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

sp500_data = yf.download('^GSPC', start='2000-01-01', end='2023-01-01')

print(sp500_data.head(10))
sp500_data['Return'] = sp500_data['Close'].pct_change() # Calculate daily returns

bond_yield_data = yf.download('^TNX', start='2000-01-01', end='2023-01-01')
bond_yield_data['Yield'] = bond_yield_data['Close']

data = sp500_data[['Return']].merge(bond_yield_data[['Yield']], left_index=True,
    right_index=True)
data.dropna(inplace=True)

sp500_data[:10]
```

Результат:

```
Price             Close         High          Low         Open      Volume
Ticker            ^GSPC        ^GSPC        ^GSPC        ^GSPC       ^GSPC
Date
2000-01-03  1455.219971  1478.000000  1438.359985  1469.250000   931800000
2000-01-04  1399.420044  1455.219971  1397.430054  1455.219971  1009000000
2000-01-05  1402.109985  1413.270020  1377.680054  1399.420044  1085500000
2000-01-06  1403.449951  1411.900024  1392.099976  1402.109985  1092300000
2000-01-07  1441.469971  1441.469971  1400.729980  1403.449951  1225200000
2000-01-10  1457.599976  1464.359985  1441.469971  1441.469971  1064800000
2000-01-11  1438.560059  1458.660034  1434.420044  1457.599976  1014000000
2000-01-12  1432.250000  1442.599976  1427.079956  1438.560059   974600000
2000-01-13  1449.680054  1454.199951  1432.250000  1432.250000  1030400000
2000-01-14  1465.150024  1473.000000  1449.680054  1449.680054  1085900000
[*********************100%**********************]  1 of 1 completed
```

Нам потрібно визначити, що таке фінансова криза. Загальноприйнятий підхід полягає у виділенні періодів екстремального спаду на ринку, коли прибутковість фондового ринку значно падає за короткий проміжок часу.

Визначимо кризову ситуацію, якщо прибутковість опуститься нижче певного порогового значення (наприклад, -5 % за один день). Перевіримо розподіл кризових моментів.

```
data['Crisis'] = (data['Return'] < -0.05).astype(int)

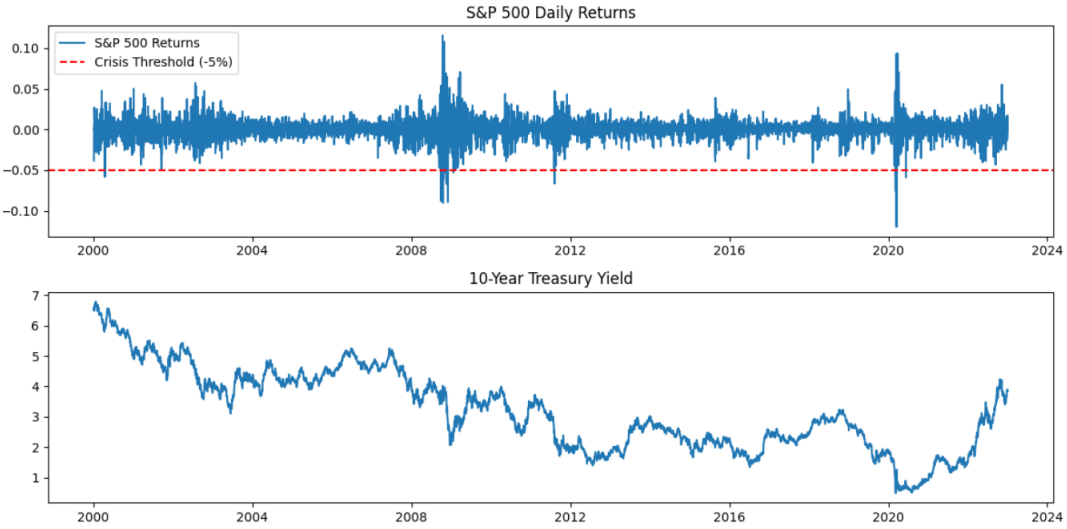
print(data['Crisis'].value_counts())

plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(data.index, data['Return'], label='S&P 500 Returns')
plt.axhline(y=-0.05, color='r', linestyle='--', label='Crisis Threshold (-5%)')
plt.legend()
plt.title('S&P 500 Daily Returns')

plt.subplot(2, 1, 2)
plt.plot(data.index, data['Yield'], label='10-Year Treasury Yield')
plt.title('10-Year Treasury Yield')
plt.tight_layout()
plt.show()
```

Результат:

```
Crisis
0    5761
1      19
Name: count, dtype: int64
```



Ми розділимо дані на навчальні та тестові набори, щоб оцінити ефективність нашої моделі.

Визначаємо характеристики (економічні показники) та ціль (мітку «Криза»), та розділяємо дані на тестові та тренувальні:

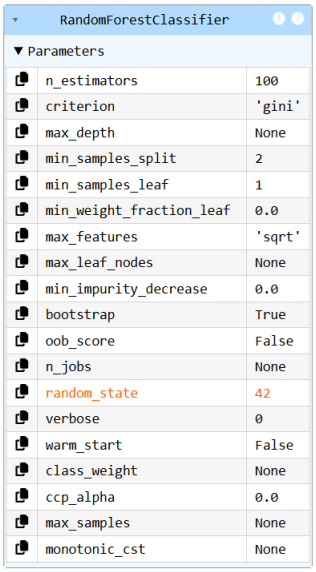
```
X = data[['Return', 'Yield']]
y = data['Crisis']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    shuffle=False)
```

Ми будемо використовувати RandomForestClassifier для моделювання взаємозв'язку між економічними показниками та кризовими подіями.

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

Результат:



Після навчання моделі ми оцінюємо її ефективність за допомогою матриці плутанини та звіту про класифікацію.

```
y_pred = model.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

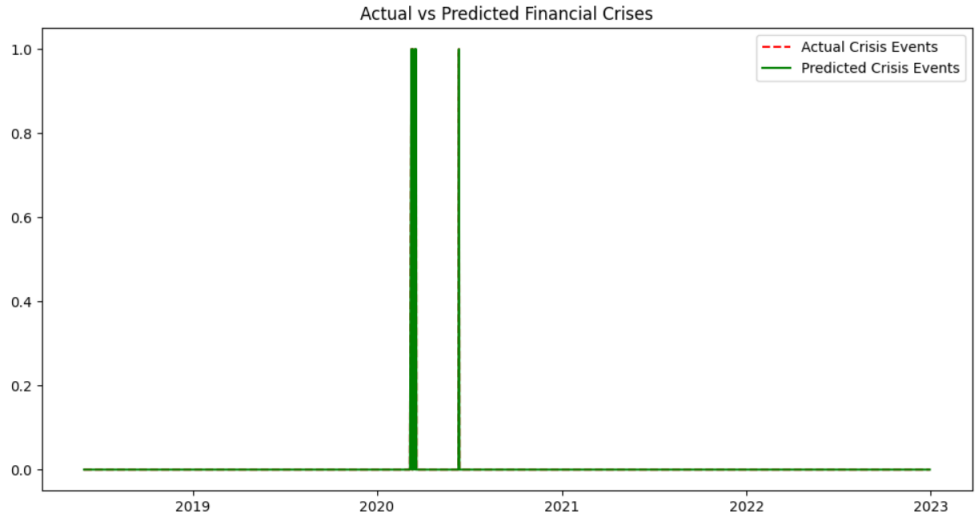
plt.figure(figsize=(12, 6))
plt.plot(y_test.index, y_test.values, label='Actual Crisis Events', color='r',
    linestyle='--')
plt.plot(y_test.index, y_pred, label='Predicted Crisis Events', color='g')
plt.title('Actual vs Predicted Financial Crises')
plt.legend()
plt.show()
```

Результат:

```
[[1151    0]
 [   0    5]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1151
           1       1.00      1.00      1.00         5

    accuracy                           1.00      1156
   macro avg       1.00      1.00      1.00      1156
weighted avg       1.00      1.00      1.00      1156
```



15.2 Моделювання розповсюдження епідемій.

Епідеміологічне моделювання - це побудова математичних або комп'ютерних моделей, які описують поширення інфекційних захворювань серед населення. Такі моделі дозволяють досліджувати механізми передачі інфекції, оцінювати швидкість її поширення, прогнозувати кількість хворих у майбутньому та аналізувати ефективність різних протиепідемічних заходів. Епідеміологічне моделювання широко застосовується в медицині, громадському здоров'ї та державному управлінні для прийняття обґрунтованих рішень під час спалахів інфекційних хвороб.

SIR-модель — це одна з найвідоміших математичних моделей для опису поширення інфекційних захворювань у популяції. Вона дозволяє зрозуміти, як хвороба поширюється, коли досягає піку та за яких умов епідемія згасає. Модель була запропонована Кермаком та Мак-Кендріком у 1927 році і досі є основою епідеміологічного моделювання.

SIR — це абревіатура від трьох станів, у яких може знаходитися кожна людина в популяції:

- S (Susceptible) — сприйнятливі: Люди, які ще не хворіють, але можуть заразитися.
- I (Infectious) — інфіковані: Люди, які хворіють і здатні передавати інфекцію іншим.
- R (Recovered/Removed) — одужалі (або видалені): Люди, які або одужали й отримали імунітет, або були ізольовані, або померли (тобто не взаємодіють із іншими).

Популяція вважається закритою, тобто:

$$N = S(t) + I(t) + R(t) = const$$

Система диференціальних рівнянь: SIR-модель описується трьома рівняннями:

$$\frac{dS(t)}{dt} = -\beta \frac{S(t) * I(t)}{N}$$

$$\frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

тут β (бета) - коефіцієнт інфікування - показує, як часто відбуваються контакти та з якою ймовірністю вони ведуть до зараження; γ (гама) - коефіцієнт одужання - це швидкість переходу з $I \to R$. Якщо середня тривалість хвороби — D днів, то:

$$\gamma \approx \frac{1}{D}$$

Коефіцієнт зараження β визначає, наскільки швидко інфекція поширюється серед населення. Якщо його значення велике, кожен інфікований заражає більше людей, тому кількість хворих швидко зростає, а пік епідемії настає раніше. Якщо ж значення коефіцієнта невелике, поширення інфекції відбувається повільніше, а епідемічна крива стає більш пологою.

Коефіцієнт одужання γ визначає, як швидко інфіковані люди перестають бути джерелом зараження. Чим більшим є цей коефіцієнт, тим швидше люди одужують, а отже, тим менше часу вони можуть передавати інфекцію іншим. У результаті зменшується кількість одночасно хворих людей і скорочується тривалість епідемії.

Базове репродуктивне число R_0 - найважливіший параметр епідемії:

$$R_0 = \frac{\beta}{\gamma}$$

Якщо $R_0 > 1$ — хвороба поширюється, епідемія. Якщо $R_0 < 1$ — хвороба згасає.

Інтуїтивно це означає, скільки людей у середньому заразить один хворий у незахищеній популяції.

Параметри моделі безпосередньо визначають форму епідемічної кривої. Збільшення коефіцієнта зараження призводить до швидкого зростання кількості інфікованих і більш високого піку захворюваності. Збільшення коефіцієнта одужання, навпаки, зменшує висоту піку та прискорює завершення епідемії. Комбінація цих параметрів дозволяє моделювати різні сценарії розвитку захворювання та оцінювати можливий вплив карантинних обмежень або вакцинації.

Динаміка епідемії, типовий перебіг для SIR: на початку багато S та мало I, інфекція швидко росте. Максимальне значення I досягається доволі швидко, потім воно спадає. З часом S зменшується, тому інфікування сповільнюється. Коли кількість сприйнятливих падає нижче певного порогу, хвиля епідемії природно згасає. Збільшується група R — тих, хто набув імунітету.

Інтерпретація кривих: на графіку зазвичай видно, як S(t) плавно зменшується; I(t) різко зростає, досягає піку, потім спадає, та R(t) зростає та стабілізується.

Пік I(t) залежить від параметрів β та γ — і саме його намагаються "сплющити"карантинними заходами.

Спочатку імпортуємо необхідні для роботи модулі та задамо параметри моделі: чисельність населення N, початкове число інфікованих I0, початкове число виздоровівших/вилучених R0, початкове число тих, хто може бути зараженим S0, коефіцієнти інфікування та виздоровлення beta та gamma:

```
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt

N = 1000
I0 = 1
R0 = 0
S0 = N - I0 - R0
beta = 0.5
gamma = 0.1
```

Визначимо часовий проміжок:

```
t = np.linspace(0, 160, 160)
```

Систему диференціальних рівнянь визначимо за допомогою наступного методу:

```
def sir_model(y, t, N, beta, gamma):
    S, I, R = y
    dSdt = -beta * S * I / N
    dIdt = beta * S * I / N - gamma * I
    dRdt = gamma * I
    return dSdt, dIdt, dRdt
```

Зрештою, сама SIR модель рівнянь за часом t:

```
y0 = S0, I0, R0

ret = odeint(sir_model, y0, t, args=(N, beta, gamma))
S, I, R = ret.T
```

Повністю разом із побідовую результатів на графіку:

```
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt

N = 1000
I0 = 1
R0 = 0
S0 = N - I0 - R0
beta = 0.5
gamma = 0.1

t = np.linspace(0, 160, 160)

def sir_model(y, t, N, beta, gamma):
    S, I, R = y
    dSdt = -beta * S * I / N
    dIdt = beta * S * I / N - gamma * I
    dRdt = gamma * I
    return dSdt, dIdt, dRdt

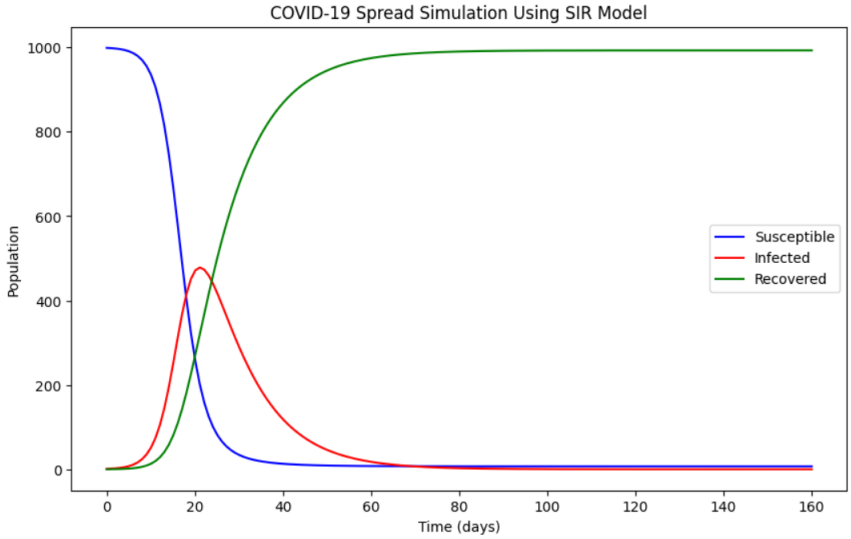
y0 = S0, I0, R0

ret = odeint(sir_model, y0, t, args=(N, beta, gamma))
S, I, R = ret.T

plt.figure(figsize=(10, 6))
plt.plot(t, S, 'b', label='Susceptible')
plt.plot(t, I, 'r', label='Infected')
plt.plot(t, R, 'g', label='Recovered')
```

```
plt.xlabel('Time (days)')
plt.ylabel('Population')
plt.title('COVID-19 Spread Simulation Using SIR Model')
plt.legend()
plt.show()
```

Результат:



Класична модель SIR базується на низці спрощувальних припущень. Вона вважає, що всі люди однаково контактують між собою, не враховує вікову структуру населення, географічне розташування, різницю в поведінці людей, повторні зараження та зміну параметрів поширення інфекції з часом. Крім того, модель не враховує вплив вакцинації, мутацій вірусу чи державних обмежувальних заходів, якщо вони не вводяться окремо.

Методи машинного навчання дозволяють враховувати значно більшу кількість факторів, ніж класичні математичні моделі. Вони можуть аналізувати мобільність населення, погодні умови, соціально-економічні показники, статистику вакцинації, дані соціальних мереж та інші джерела інформації. Поєднання моделей SIR із методами машинного навчання дає змогу автоматично оцінювати параметри моделі, підвищувати точність прогнозування та швидше адаптуватися до нових умов розвитку епідемії.

Сучасний розвиток штучного інтелекту значно розширює можливості моделювання кризових явищ. Використання великих даних дозволяє аналізувати інформацію з численних джерел у режимі реального часу, а сучасні алгоритми машинного навчання здатні виявляти складні закономірності, які важко помітити традиційними методами. Особливо перспективними є глибокі нейронні мережі, Transformer-моделі, графові нейронні мережі та генеративний штучний інтелект. У майбутньому очікується поєднання класичних математичних моделей із методами машинного навчання, що забезпечить більш точне прогнозування криз, швидшу адаптацію моделей до нових умов і можливість підтримки прийняття рішень у реальному часі.

✎ **Питання для самоконтролю.**

1. Що таке кризове явище з точки зору аналізу даних?
2. Які типи кризових явищ можна моделювати за допомогою методів машинного навчання?
3. Чому прогнозування кризових явищ є важливим завданням інтелектуального аналізу даних?
4. Які дані використовуються для прогнозування фінансових криз?

5. Які економічні показники можуть бути ознаками наближення фінансової кризи?
6. Які задачі машинного навчання застосовуються для прогнозування фінансових криз?
7. Які алгоритми машинного навчання найчастіше використовуються для прогнозування кризових явищ?
8. Які переваги має використання машинного навчання порівняно з традиційними економічними моделями?
9. Які труднощі виникають під час побудови моделей прогнозування фінансових криз?
10. Чому якість даних є критично важливою для прогнозування кризових явищ?
11. Що таке епідеміологічне моделювання?
12. Для чого використовується модель SIR?
13. Які групи населення виділяє модель SIR?
14. Що означає категорія Susceptible (S) у моделі SIR?
15. Що означає категорія Infected (I) у моделі SIR?
16. Що означає категорія Recovered (R) у моделі SIR?
17. Які параметри використовуються в моделі SIR?
18. Як коефіцієнт зараження впливає на поширення епідемії?
19. Як коефіцієнт одужання впливає на перебіг епідемії?
20. Що таке базове репродуктивне число R_0 і як воно інтерпретується?
21. Як зміна параметрів моделі SIR впливає на форму епідемічної кривої?
22. Які обмеження має класична модель SIR?
23. Як методи машинного навчання можуть доповнювати математичні моделі поширення епідемій?
24. У яких практичних сферах використовуються моделі прогнозування кризових явищ?
25. Які перспективи розвитку моделювання кризових явищ пов'язані зі штучним інтелектом, аналізом великих даних та сучасними методами машинного навчання?

Розділ 16

Використання нейронних мереж для аналізу даних.

16.1 Вступ до нейронних мереж. Архітектура нейронної мережі.

У попередніх розділах ми розглянули набір алгоритмів машинного навчання, та їхні можливості при аналізі даних. Вони розв'язують значну кількість практичних задач, проте вони мають і свої обмеження. У багатьох реальних задачах залежності між змінними можуть бути настільки складними, що їх неможливо описати за дпопомогою простих математичних моделей.

Нейронна мережа - це модель прогнозування, створена на основі принципів роботи мозку. Мозок можна уявити як сукупність нейроннів, з'єднаних між собою. Кожен нейрон аналізує дані інших нейроннів, що приходять до нього, виконує певні обчислення, і активується (якщо результат обчислення вищий за певне порогове значення), або не активується. Схожим чином організована робота штучних нейронних мереж.

Нейронні мережі широко використовуються для класифікації, регресії, прогнозування часових рядів, кластеризації, розпізнавання об'єктів, аналізу тексту та зображень. Наприклад, нейронна мережа може визначати категорію зображення, прогнозувати ціну нерухомості, визначати тональність тексту або прогнозувати майбутні значення часового ряду.

Однією з основних відмінностей є здатність нейронних мереж автоматично формувати складні представлення даних. У багатьох класичних алгоритмах якість результату значною мірою залежить від того, наскільки вдало дослідник вибрав і підготував ознаки.

☞ В більшості задач, з якими стикається початківець в галузі аналізу даних, нейронні мережі не є правильним вибором. Більшість нейронних мереж є "чорними скриньками вивчення їхніх деталей не дає глибокого розуміння того, як саме воони вирішують задачу. Крім того, великі нейронні мережі може бути складно навчити.

Типова нейронна мережа складається з вхідного шару, одного або декількох прихованих шарів та вихідного шару. Вхідний шар отримує ознаки об'єктів. Приховані шари виконують послідовні перетворення цих ознак і формують внутрішнє представлення даних. Вихідний шар формує кінцевий результат, наприклад клас або числовий прогноз. Між нейронами різних шарів встановлюються зв'язки, кожен з яких має власний ваговий коефіцієнт.

Штучний нейрон — це базовий обчислювальний елемент нейронної мережі. Він отримує декілька вхідних значень, множить кожне з них на відповідну вагу, додає зміщення та передає отриманий результат через функцію активації.

У спрощеному вигляді роботу нейрона можна представити як

$$z = \sum_{i=1}^{n} w_i x_i + b,$$

після чого формується вихід

$$y = f(z),$$

де x_i - вхідні ознаки, w_i - ваги, b - зміщення, а f - функція активації.

Вагові коефіцієнти визначають, наскільки сильно кожна вхідна ознака впливає на результат роботи нейрона. Якщо абсолютне значення ваги є великим, відповідний вхід може мати значний вплив на результат. Якщо вага близька до нуля, вплив відповідної ознаки є меншим. Під час навчання мережі ваги поступово змінюються за допомогою алгоритму оптимізації, щоб зменшити значення функції втрат.

Зміщення (bias) дозволяє нейрону зміщувати результат своєї лінійної комбінації незалежно від значень вхідних ознак. Без зміщення нейрон був би змушений використовувати функцію, яка проходить через початок координат. Додавання b робить модель більш гнучкою та дозволяє краще адаптуватися до даних.

Функція активації перетворює результат роботи нейрона та додає моделі нелінійність. Це принципово важливо для аналізу складних даних. Якщо всі шари нейронної мережі виконували б лише лінійні перетворення, то навіть дуже глибока мережа фактично залишалася б еквівалентною одній лінійній моделі. Серед поширених функцій активації можна назвати ReLU, sigmoid та tanh. Наприклад, функція ReLU визначається як

$$f(x) = max(0, x).$$

Завдяки функціям активації нейронна мережа може моделювати складні нелінійні залежності між ознаками та результатом, що є однією з її ключових переваг в інтелектуальному аналізі даних.

Типова нейронна мережа складається з вхідного шару, одного або декількох прихованих шарів та вихідного шару. Вхідний шар отримує ознаки об'єкта, приховані шари виконують послідовне перетворення даних, а вихідний шар формує результат роботи моделі. Кількість прихованих шарів і нейронів у них визначає архітектуру мережі.

Вхідний шар приймає підготовлені дані, які подаються на вхід нейронної мережі. Кожен вхід зазвичай відповідає певній ознаці об'єкта. Наприклад, під час прогнозування ціни квартири вхідними ознаками можуть бути площа, кількість кімнат і поверх. Вхідний шар фактично передає ознаки до першого прихованого шару, де починається їх обробка.

Приховані шари виконують послідовні перетворення вхідних даних та дають змогу мережі виявляти складні закономірності. Кожен шар може формувати нове представлення інформації на основі результатів попереднього шару. Наприклад, під час аналізу зображення перші шари можуть виявляти прості структури, а глибші — складніші об'єкти та їх комбінації.

Вихідний шар формує кінцевий результат роботи нейронної мережі. Його структура залежить від поставленої задачі. Для регресії вихідний шар може містити один нейрон, який повертає числове значення. Для бінарної класифікації часто використовується один вихід із sigmoid-активацією, а для багатокласової класифікації — кілька виходів із softmax-активацією.

16.2 Навчання нейронної мережі.

16.3 Згорткові нейронні мережі (CNN).

16.4 Transformer-моделі.

16.5 Переваги та недоліки нейронних мереж для аналізу даних. Сучасні напрями розвитку нейронних мереж.

Нейронні мережі автоматично знаходять складні залежності у великих масивах інформації та здатні формувати внутрішні представлення даних без необхідності вручну задавати всі правила їх обробки. Саме тому вони часто використовуються для аналізу текстів, зображень, аудіо, табличних даних та інших типів інформації.

✎ **Питання для самоконтролю.**

1. Що таке штучна нейронна мережа?
2. Чому нейронні мережі є важливими для інтелектуального аналізу даних?
3. Які задачі аналізу даних найчастіше розв'язуються за допомогою нейронних мереж?
4. Чим нейронні мережі відрізняються від класичних алгоритмів машинного навчання?
5. Які переваги мають нейронні мережі під час роботи зі складними даними?
6. Які недоліки нейронних мереж порівняно з традиційними алгоритмами?
7. З яких основних компонентів складається штучна нейронна мережа?
8. Що таке штучний нейрон?
9. Яку роль відіграють вагові коефіцієнти нейрона?
10. Для чого використовується зміщення (bias)?
11. Яке призначення функції активації?
12. Які шари входять до архітектури нейронної мережі?
13. Яке призначення вхідного шару?
14. Для чого використовуються приховані шари?
15. Яку функцію виконує вихідний шар?
16. Що таке пряме поширення сигналу (Forward Propagation)?
17. Як нейронна мережа автоматично формує нові ознаки під час навчання?
18. Що означає поняття «глибока нейронна мережа» (Deep Neural Network)?
19. Що означає навчання нейронної мережі?
20. Для чого використовується функція втрат?
21. Що таке зворотне поширення помилки (Backpropagation)?
22. Як градієнтний спуск використовується під час навчання нейронної мережі?

23. Яку роль відіграє швидкість навчання (Learning Rate)?
24. Що таке епоха навчання (Epoch)?
25. Чим Mini-Batch Gradient Descent відрізняється від Batch Gradient Descent?
26. Що таке згорткова нейронна мережа (CNN)?
27. Чому CNN особливо ефективні для аналізу зображень?
28. Що таке операція згортки (Convolution)?
29. Для чого використовуються згорткові фільтри?
30. Яке призначення шару Pooling?
31. У яких задачах аналізу даних, крім комп'ютерного зору, застосовуються CNN?
32. Що таке Transformer-модель?
33. Які недоліки рекурентних нейронних мереж привели до появи Transformer-моделей?
34. Що таке механізм уваги (Attention)?
35. Що таке Self-Attention?
36. Які переваги Transformer-моделей порівняно з RNN?
37. У яких задачах інтелектуального аналізу даних використовуються Transformer-моделі?
38. Який зв'язок існує між Transformer-моделями та великими мовними моделями (LLM)?
39. Які переваги мають нейронні мережі під час аналізу великих обсягів даних?
40. Які обмеження виникають під час навчання нейронних мереж?
41. Чому нейронні мережі часто називають моделями з низькою інтерпретованістю?
42. У яких випадках доцільніше використовувати класичні алгоритми машинного навчання замість нейронних мереж?
43. Що таке Transfer Learning?
44. Які сучасні напрями розвитку нейронних мереж існують?
45. Яку роль нейронні мережі відіграють у розвитку сучасного штучного інтелекту?